Six Systems Projects for CS:APP

A C/C++ build-as-you-read companion to
Computer Systems: A Programmer's Perspective
aimed at LLM inference, KV-cache, and accelerator work

Prepared for Utkarsh Kumar (UK)

October 2026

Contents

How to use this document **5**

1 Shared infrastructure **7**
1.1 Toolchain . . . 7
1.2 The C toolchain, concept by concept . . . 7
1.2.1 What `gcc foo.c` actually runs . . . 7
1.2.2 Make, properly . . . 9
1.2.3 Objects, archives, shared libraries, and the linker . . . 9
1.2.4 Debuggers, sanitizers, and analyzers . . . 10
1.3 Repository layout . . . 11
1.4 The test harness `ck.h` . . . 12
1.5 Makefile skeleton . . . 12
1.6 Documentation conventions . . . 13
1.7 How assignments are specified . . . 13
1.8 How to write the blog posts . . . 14
1.8.1 Who the reader is . . . 14
1.8.2 The anatomy of a main post (2000–3500 words) . . . 14
1.8.3 Figure standards . . . 15
1.8.4 Writing rules . . . 15
1.8.5 Workflow and cadence . . . 15
1.8.6 Pre-publication checklist . . . 16
1.9 Git fundamentals (taught once, used everywhere) . . . 16
1.9.1 The object model . . . 16
1.9.2 Refs, HEAD, and the three trees . . . 17
1.9.3 The baseline workflow . . . 17
1.9.4 Commit discipline . . . 17

2 Project 1: `qnum` — a low-precision numerics library **18**
2.1 Goal . . . 18
2.2 CS:APP reading map . . . 18
2.3 Mathematical background . . . 19
2.3.1 Binary floating-point formats . . . 19
2.3.2 Rounding by integer arithmetic . . . 19
2.3.3 Narrowing the exponent range (fp32 → fp16/fp8) . . . 20
2.3.4 Integer quantization . . . 21
2.3.5 Block floating point . . . 21

2.3.6 Comparing floats: ULPs . . . 22
2.3.7 The safetensors container . . . 22
2.4 Architecture . . . 22
2.4.1 Directory layout . . . 22
2.4.2 Public API . . . 23
2.4.3 Implementation notes . . . 24
2.5 Assignments . . . 24
2.6 Test plan . . . 26
2.7 Measurements and blog briefs . . . 27
2.8 Git lesson and push checklist . . . 29

3 Project 2: `cforward` — an optimized single-core transformer forward pass 30
3.1 Goal . . . 30
3.2 CS:APP reading map . . . 30
3.3 Mathematical background . . . 31
3.3.1 The model . . . 31
3.3.2 Cost model . . . 32
3.3.3 Why one accumulator is slow (§ 5.7) . . . 32
3.3.4 Cache blocking for GEMM (§ 6.6) . . . 33
3.3.5 The memory mountain (§ 6.6.1) . . . 33
3.4 Architecture . . . 33
3.4.1 Directory layout . . . 33
3.4.2 Public API . . . 33
3.4.3 Weight layout . . . 34
3.4.4 The optimization ladder for `matvec` . . . 35
3.5 Assignments . . . 35
3.6 Test plan . . . 37
3.7 Measurements and blog briefs . . . 37
3.8 Git lesson and push checklist . . . 39

4 Project 3: `ltrace-ml` — an interposition tracer and sampling profiler 40
4.1 Goal . . . 40
4.2 CS:APP reading map . . . 41
4.3 Background . . . 41
4.3.1 Symbol resolution and why preloading works . . . 41
4.3.2 Capturing the caller . . . 41
4.3.3 Offline symbolization . . . 42
4.3.4 Sampling profiler . . . 42
4.3.5 Process control for the launcher (§ 8.4) . . . 42
4.3.6 What CUDA looks like from here . . . 43
4.4 Architecture . . . 43
4.4.1 Directory layout . . . 43
4.4.2 Public API and formats . . . 43
4.4.3 Hook pattern (pseudocode) . . . 44
4.5 Assignments . . . 45
4.6 Test plan . . . 46
4.7 Measurements and blog briefs . . . 47
4.8 Git lesson and push checklist . . . 49

5 Project 4: `pagedkv` — a memory allocator and a paged KV cache with prefix caching 50
5.1 Goal . . . 50
5.2 CS:APP reading map . . . 51
5.3 Mathematical background . . . 51
5.3.1 Allocator design . . . 51
5.3.2 KV-cache geometry . . . 51
5.3.3 Sharing: reference counts and copy-on-write . . . 52
5.3.4 Prefix cache: radix tree over blocks . . . 52
5.3.5 Reuse distance and optimal replacement . . . 53
5.3.6 Pages and the TLB . . . 54
5.4 Architecture . . . 54
5.4.1 Directory layout . . . 54
5.4.2 Public API . . . 54
5.4.3 Attention over paged blocks . . . 56
5.5 Assignments . . . 56
5.6 Test plan . . . 58
5.7 Measurements and blog briefs . . . 59
5.8 Git lesson and push checklist . . . 60

6 Project 5: `tinyjit` — an AVX2 JIT compiler for fused tensor operations 62
6.1 Goal . . . 62
6.2 CS:APP reading map . . . 63
6.3 Background . . . 63
6.3.1 The IR . . . 63
6.3.2 Lowering to a loop . . . 63
6.3.3 Register allocation . . . 64
6.3.4 The System V x86-64 calling convention (§ 3.7) . . . 64
6.3.5 Instruction encoding . . . 64
6.3.6 Executable memory . . . 65
6.3.7 Transcendentals in SIMD . . . 66
6.3.8 Why fusion pays . . . 66
6.3.9 Writing a relocatable ELF object (stretch) . . . 66
6.4 Architecture . . . 66
6.5 Assignments . . . 67
6.6 Test plan . . . 69
6.7 Measurements and blog briefs . . . 70
6.8 Git lesson and push checklist . . . 72

7 Project 6: `servlet` — a continuous-batching inference server 73
7.1 Goal . . . 73
7.2 CS:APP reading map . . . 73
7.3 Background . . . 74
7.3.1 Non-blocking I/O and the reactor . . . 74
7.3.2 HTTP/1.1 and SSE . . . 74
7.3.3 Threads and queues (§ 12.3–12.7) . . . 74
7.3.4 Continuous batching . . . 75
7.3.5 Performance model . . . 75

7.3.6 Tokenizer 76
7.4 Architecture 76
7.5 Assignments 77
7.6 Test plan 79
7.7 Measurements and blog briefs 80
7.8 Git lesson and push checklist 82

A Git command reference **83**

B Performance measurement cheat sheet **85**

C Reference constants **86**

D Blog series overview **88**

How to use this document

This document specifies six C/C++ projects to build while reading CS:APP (Bryant & O'Hallaron, 3rd ed.). The projects are ordered so that each one lands on the chapters you are reading at the time, and they compose: by the end you will have a small CPU-side LLM inference stack whose every layer you wrote and measured yourself—a numerics library, an optimized forward pass, a tracer and profiler, a paged KV-cache with a prefix cache, a tensor JIT, and a continuous-batching server.

Everything that is *not* typing code has been done here: the architecture of each project (directory layout, public headers, data structures, algorithms in pseudocode), the mathematics behind it, the measurements that matter, the test plan, a blog plan per project, and a progressive Git curriculum ending in a CI-tested, versioned, multi-repository release.

Left to you

Four things are left to you for every project, and the document is organized so that nothing else is:

1. **Implementation** of the specified headers, in C (projects 1–5) or C++20 (project 6).
2. **Testing**: writing the tests enumerated in each "Test plan" section and making them pass under the sanitizers.
3. **Documentation**: the README, the `docs/` notes, and the blog post whose outline is given.
4. **Pushing**: committing and publishing with the Git workflow taught in that project's Git lesson.

Schedule

Two months per project at 1–2 hours a day, with project 6 allowed to run long. Read the CS:APP chapters listed *before* the milestones that depend on them; the mapping is deliberate.

#	Project	CS:APP chapters	Language	Months
1	`qnum` — low-precision numerics	2 (bits, IEEE 754), 3 (x86-64 asm)	C	1–2
2	`cforward` — optimized transformer forward pass	5 (optimization), 6 (memory hierarchy)	C	3–4
3	`ltrace-ml` — interposition tracer + sampling profiler	7 (linking), 8 (exceptional control flow)	C	5–6
4	`pagedkv` — allocator + paged KV cache + prefix cache	9 (virtual memory)	C	7–8
5	`tinyjit` — AVX2 JIT for fused tensor ops	3 (revisit), 7 (relocations, ELF)	C	9–10
6	`servlet` — continuous-batching inference server	10 (I/O), 11 (network), 12 (concurrency)	C++20	11–12

When to start each project: chapter gates

The table above says which chapters a project *uses*; this one says when to *start* it. A gate is the reading that must be finished before the first assignment, because the first assignment already depends on it. The remaining chapters of a project's map can be read while its early assignments are under way; the per-assignment gates in the repository's `ROADMAP.md` say exactly when. Read § 1 before anything and skim § 4.1–4.5 (pipelining) before § 5; neither has a project of its own.

#	Start after (gate)	Read while building	Lab first
1	§ 1, § 2 (all)	§ 3.1–3.6, § 3.11 before A1.6	Data Lab
2	§ 5, § 6.1–6.7	§ 6 rest; A2.1 may begin right after P1	Cache Lab
3	§ 7 (all), § 8.1–8.4	§ 8.5 before A3.3; skim § 12.3 before A3.4	Shell Lab
4	§ 9 (all)	— (A4.1 *is* Malloc Lab, done properly)	—
5	P4 done; re-read § 3.2–3.7, § 3.11	§ 7.4–7.7 again before A5.6	Bomb/Attack Lab (if skipped)
6	§ 10, § 11	§ 12 before A6.3	Proxy Lab (optional)

Conventions used in this document

- `monospace` is a file, command, symbol, or identifier. § 6.4 is a CS:APP section.
- Header listings give the *public* API you must implement; signatures are normative, internal helpers are yours.
- Pseudocode is written to be translated line-for-line; where a detail is left open it is stated as a choice.
- Boxes: **Research hook** ties the project to your research; **Blog plan** gives the post to write; **Git lesson** is the version-control material for that project; **Pitfall** is a mistake worth avoiding before you make it.

How the projects compose

Consumer	Uses
`cforward` (P2)	`qnum` (P1) for bf16/int8/int4/BFP weights
`ltrace-ml` (P3)	`cforward` as its first tracing target
`pagedkv` (P4)	`cforward`'s attention kernel reads from paged blocks
`tinyjit` (P5)	replaces `cforward`'s elementwise kernels; emits P1's quantize/dequantize ops
`servlet` (P6)	`cforward` + `pagedkv` + `qnum` as Git submodules; profiled with `ltrace-ml`

Chapter 1

Shared infrastructure

Everything in this chapter is set up once, in project 1, and reused by the other five. Do it properly the first time: a solid test harness and build system is what makes the later projects' measurements trustworthy.

1.1 Toolchain

- **Compilers.** GCC 12+ and Clang 16+, both installed; every project must build with both (the differences are themselves data for you). C projects use `-std=c17`; project 6 uses `-std=c++20`.
- **Warning flags, always on:** `-Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wstrict-prototypes -Werror`. Fixing `-Wconversion` warnings in a numerics library will teach you more about CS:APP chapter 2 than the exercises will.
- **Build variants** (selected by `BUILD=`): `debug` (`-O0 -g3`), `release` (`-O2 -g -fno-omit-frame-pointer -march=native`), `asan` (`-O1 -g -fsanitize=address,undefined`), `tsan` (`-O1 -g -fsanitize=thread`). Keep frame pointers in release builds: project 3's stack walker depends on them, and the cost is negligible.
- **Tools you will use constantly:** `perf` (with `kernel.perf_event_paranoid` ≤ 1), `valgrind` (`--tool=memcheck`, `--tool=cachegrind`), `objdump -d -M intel`, `readelf -a`, `nm -C`, `ldd`, `strace`, `gdb`, `clang-format`, `python3` with `numpy` and `torch` for reference values.
- **Hardware notes.** Record once, in `docs/machine.md`: CPU model, core count, base/turbo frequency, L1d/L2/L3 sizes (`lscpu`, `getconf -a | grep CACHE`), DRAM type and channel count, measured STREAM bandwidth, measured peak FP32 FLOP/s (project 2 gives you the microbenchmarks). Every performance number you publish refers back to this file.

1.2 The C toolchain, concept by concept

You know C++ and CUDA, so this section is not about the language. It is about the *tools around* a C program—what each one does to your bytes, in the order they do it—because the projects lean on them constantly and CS:APP chapter 7 assumes you already have the vocabulary.

1.2.1 What `gcc foo.c` actually runs

The compiler driver runs four programs in sequence, and you can stop after any of them:

Stage	Program	Stop with	What it does
Preprocess	`cpp`	`-E` → `foo.i`	Textual: `#include` pastes headers in, `#define` substitutes, `#if` selects. No types, no semantics.
Compile	`cc1`	`-S` → `foo.s`	C to assembly for one *translation unit* (the `.i` file). All optimization happens here.
Assemble	`as`	`-c` → `foo.o`	Assembly to a *relocatable object*: machine code plus a symbol table plus relocation records for addresses it does not know yet.
Link	`ld` (via `collect2`)	(default) → `a.out`	Merges objects and libraries, resolves symbols, applies relocations, lays out segments. Chapter 7 is about this stage.

Run `gcc -v foo.c` once to see the four commands with their real flags. Run `gcc -E` on a file that includes `<stdio.h>` and look at how many lines come out; that is why header hygiene matters. Run `gcc -S -O2 -fverbose-asm` and read the output next to `objdump -d` of the `.o`: the first is what the compiler meant, the second is what the assembler produced, and the addresses in the second are all zero-based because relocation has not happened yet.

Translation units and headers. A `.c` file plus everything it includes is one translation unit, compiled in isolation. A header is a *promise* about things defined elsewhere (function prototypes, `extern` variables, types, macros); the definition lives in exactly one `.c` file. `static` at file scope makes a symbol invisible to other units (internal linkage); without it, the symbol is global and the linker will complain if two units define it. Include guards (`#ifndef FOO_H`) or `#pragma once` prevent double inclusion within one unit. Your `include/<project>/` directory holds the public promises; `src/` holds the definitions and may have private headers of its own.

Flags you will type a thousand times. `-I dir` adds an include search directory; `-D NAME=val` defines a macro from the command line (how `-DTJ_VERSION` works in project 5); `-O0/-O1/-O2/-O3/-Os` set optimization level (`-O2` is the measurement default; `-O3` adds aggressive vectorization and inlining that sometimes helps and sometimes hurts—measure); `-g` embeds DWARF debug information (does not slow code down; always keep it); `-march=native` lets the compiler use every instruction your CPU has (AVX2, FMA) and makes the binary non-portable; `-fno-omit-frame-pointer` keeps `rbp` as a frame pointer so stack walkers work; `-fPIC` generates position-independent code, required for shared libraries; `-std=c17` and `-pedantic` pin the language; `-Wall -Wextra` and friends turn on diagnostics that the standard does not require but that catch real bugs; `-Werror` makes them fatal so they cannot accumulate.

What `-O2` does to your code. Inlining (small functions disappear), constant propagation and dead-code elimination (your benchmark loop may vanish if its result is unused—use `volatile` sinks or `asm volatile("" :: "r"(x))` barriers), loop-invariant code motion, strength reduction, register allocation (locals never touch memory), and auto-vectorization when it can prove there is no aliasing. CS:APP chapter 5 is largely about what stops the optimizer: memory aliasing (two pointers might point at the same place, so stores cannot be reordered), function calls (might have side effects), and floating-point reassociation (not allowed without `-ffast-math`). `restrict` on pointer parameters promises no aliasing and is often the difference between a vectorized loop and a scalar one.

1.2.2 Make, properly

`make` is a dependency engine: given rules of the form *target: prerequisites, recipe*, it rebuilds a target when any prerequisite is newer than it (by file modification time), recursively. Everything else is convenience syntax.

```
target: prereq1 prereq2        # a rule
    recipe-command               # recipes are indented with a TAB, not spaces
```

Variables. `CC = gcc` (recursively expanded: evaluated when used) vs `CC := gcc` (simply expanded: evaluated now); `CC ?= gcc` sets only if unset (lets `make CC=clang` override from the command line); `CFLAGS += -g` appends. Conventional names matter because the built-in rules use them: `CC`, `CFLAGS`, `CPPFLAGS` (preprocessor: `-I`, `-D`), `LDFLAGS` (linker: `-L`), `LDLIBS` (libraries: `-lm`).

Pattern rules and automatic variables. `$(OUT)/%.o: src/%.c` says "any `.o` in `build/` is made from the same-stem `.c` in `src/`". Inside a recipe: `$@` is the target, `$<` the first prerequisite, `$^` all prerequisites, `$*` the stem matched by `%`. `$(wildcard src/*.c)` lists files; `$(SRC:src/%.c=$(OUT)/%.o)` rewrites a list by pattern (substitution reference); `$(patsubst ...)`, `$(filter ...)`, `$(shell ...)` are the other functions you will need.

Order-only prerequisites. `$(OUT)/%.o: src/%.c | $(OUT)` means the directory must exist before the recipe runs but its timestamp does not trigger rebuilds (directories change mtime whenever a file inside them does, which would otherwise rebuild everything every time).

Phony targets. `.PHONY: all test clean fmt` declares names that are not files, so `make clean` runs even if a file named `clean` exists and is never considered up to date.

Header dependencies. Make does not parse C, so by default editing a header rebuilds nothing. `-MMD -MP` makes the compiler write `foo.d` alongside `foo.o` containing `foo.o: foo.c a.h b.h` (plus empty rules for each header so deleting one does not break the build); `-include $(OBJ:.o=.d)` pulls those in. This is the single most common cause of "it works after `make clean`".

Conditionals and variants. `ifeq ($(BUILD),asan) ... else ... endif` selects flags; because `OUT` includes `$(BUILD)`, the variants have separate object directories and never contaminate each other.

Running it. `make -j$(nproc)` builds in parallel (your rules must declare true dependencies for this to be safe); `make -n` prints the commands without running them; `make -B` forces a rebuild; `make --debug=b` explains why each target is rebuilt; `make -p` dumps the database of rules including the built-in ones (read it once).

Why not CMake? CMake generates Makefiles (or Ninja files) from a higher-level description and handles cross-platform and multi-configuration builds well; for six single-platform libraries with a dozen files each, a hand-written Makefile is shorter, teaches more, and has no generator step. Project 6 may adopt CMake when the submodules make the Makefile awkward; by then you will know what it is generating.

1.2.3 Objects, archives, shared libraries, and the linker

Relocatable object (`.o`). Sections (`.text` code, `.data` initialized globals, `.bss` zero-initialized globals that occupy no file space, `.rodata` constants, `.symtab`, `.rela.text` relocations, DWARF

`.debug_*`); `readelf -S` lists them, `nm` lists symbols with a letter (`T` defined in text, `U` undefined, `D` data, `t` local text, `W` weak), `objdump -d` disassembles, `objdump -r` shows relocations.

Static library (`.a`). An `ar` archive of `.o` files with an index; `ar rcs lib.a *.o`. The linker pulls out only the members that resolve currently-undefined symbols, *in command-line order*, which is why `-lfoo` must come *after* the objects that use it and why circular dependencies between archives need the library listed twice. `libqnum.a` linked into `cforward` means `cforward`'s binary contains a copy of the code it used, and nothing else.

Shared library (`.so`). Built from `-fPIC` objects with `gcc -shared -o libfoo.so`; code is not copied into the executable but mapped at run time by the dynamic linker (`ld-linux.so`), which resolves `DT_NEEDED` entries by searching `LD_LIBRARY_PATH`, the executable's `RUNPATH` (`-Wl,-rpath,$ORIGIN/../lib`), and the system cache (`ldconfig`). `ldd prog` shows what it will load; `LD_DEBUG=libs ./prog` shows the search. Functions are called through the PLT/GOT indirection (§ 7.12), lazily bound on first call unless `LD_BIND_NOW` is set; `LD_PRELOAD` is project 3's whole premise. Versioned names (`libfoo.so.1.2.3` ← `libfoo.so.1` ← `libfoo.so`) exist so that a program linked against ABI version 1 keeps working after an upgrade; you will not need them, but you will see them in `ldd` output.

Symbol resolution rules (§ 7.6). Strong symbols (functions, initialized globals) may be defined once; weak symbols (uninitialized globals, `__attribute__((weak))`) yield to a strong one; two strong definitions are an error; two weak ones pick arbitrarily (compile with `-fno-common` to avoid the surprises). `-Wl,--wrap=sym`, `-Wl,-Map=out.map` (a full layout listing), `-Wl,--gc-sections` with `-ffunction-sections` (drop unused functions) are the linker flags worth knowing; `-Wl,` passes what follows to `ld`.

Executables. `readelf -l` shows program headers (segments: what the loader maps, with permissions); `readelf -h` the entry point; a PIE (position-independent executable, the default now) is itself relocated at load time to a random base (ASLR), which is why project 3 subtracts the load base before symbolizing.

1.2.4 Debuggers, sanitizers, and analyzers

gdb. `gdb --args ./prog arg`; `break file.c:123`, `run`, `next`/`step`/`finish`, `print expr` (and `print/x`, `print *ptr@8` for arrays), `backtrace` (`bt`), `frame 2`, `info registers`, `x/8xg $rsp` (examine memory), `watch var` (hardware watchpoint; stops when it changes), `display var`, `layout asm`/`layout src` (TUI). For a crash: `ulimit -c unlimited`, then `gdb ./prog core`. Reverse execution (`record`, `reverse-next`) is slow but occasionally the only way to answer "who wrote this value?". Build with `-g -O0` to debug logic, `-g -O2` to debug what you ship (variables may be "optimized out"—that is real, not a bug).

Sanitizers are compiler instrumentation plus a runtime, enabled per build: AddressSanitizer (`-fsanitize=address`: heap/stack/global overflows, use-after-free, leaks; $\sim$ 2× slowdown, 2–3× memory), UndefinedBehaviorSanitizer (`-fsanitize=undefined`: signed overflow, misaligned access, shift out of range, null dereference—cheap, always combine with ASan), ThreadSanitizer (`-fsanitize=thread`: data races and some deadlocks; 5–15× slowdown; incompatible with ASan in the same binary), MemorySanitizer (clang only: uninitialized reads). Sanitizers cannot see inside memory you manage yourself (project 4's allocator), which is why that project has a shadow checker, and ASan's own allocator replaces `malloc`, which matters for project 3 (test the tracer in non-ASan builds too). `ASAN_OPTIONS=detect_leaks=1:abort_on_error=1` and

`TSAN_OPTIONS=halt_on_error=1` are the environment knobs you will use.

Valgrind is binary instrumentation, no rebuild needed, 20–50× slower: `memcheck` (default) finds uninitialized reads and invalid accesses, including in code you did not compile; `cachegrind` simulates the cache hierarchy and gives per-line miss counts (`cg_annotate`); `callgrind` gives call-graph profiles (`kcachegrind` to view); `massif` profiles heap usage over time; `helgrind` and `drd` find races (slower than TSan, different blind spots). Use memcheck on every project once; cachegrind in project 2; massif if an allocator question arises in project 4.

perf reads hardware counters and samples (Appendix B); `strace -f -e trace=mmap,openat ./prog` shows system calls (what project 3's tracer sees from the other side; use it to confirm `mmap` sizes in projects 2 and 4); `ltrace` is the library-call analogue (and the name your project 3 tool nods to).

Static analysis and formatting. `clang-tidy` with a modest check set (`bugprone-*`, `performance-*`, `readability-*`) catches a class of mistakes before running anything; `cppcheck` is a second opinion; `scan-build make` runs clang's static analyzer over the whole build. `clang-format` with a committed `.clang-format` ends formatting discussions; `make fmt` runs it, and the pre-commit hook from project 3 checks it.

Inspecting binaries, a habit to build. After every optimization in project 2, `objdump -d -M intel --no-show-raw-insn build/release/kernels.o | less` and find your loop. After every linking question in project 3, `readelf -r`, `nm -D` (dynamic symbols), `readelf -d` (dynamic section: `NEEDED`, `RUNPATH`). After writing bytes in project 5, `objdump -D -b binary -m i386:x86-64`. `size prog` for section sizes, `strings`, `file`, `hexdump -C` for the first 64 bytes of any file you are parsing (the ELF magic, the safetensors header length).

1.3 Repository layout

One repository per project (they become six portfolio pieces and six blog posts), all following the same skeleton:

```
<project>/
  README.md             what it is, how to build, how to test, headline numbers
  LICENSE               MIT or Apache-2.0
  Makefile              targets: all test bench fmt clean; BUILD=debug|release|asan|
    tsan
  include/<project>/    public headers (the API specified in this document)
  src/                  implementation
  tests/                ck-based unit tests, one file per module; test_*.c
  bench/                microbenchmarks and measurement scripts
  scripts/              python3 reference generators, plotting, post-processing
  docs/                 machine.md, design notes, blog draft (blog.md)
  .github/workflows/    CI (added in project 6, back-ported to the others)
  .clang-format         shared style; run 'make fmt' before every commit
  .gitignore            build/, *.o, *.so, *.pdf, data/, __pycache__/
```

Build outputs go to `build/$(BUILD)/` so that debug and sanitizer builds coexist. Model weights and traces live in `data/` and are never committed (they belong in `.gitignore`); a `scripts/fetch_data.sh` reproduces them.

1.4 The test harness `ck.h`

Write a single-header unit test framework in project 1 and copy it into every later project. Specification:

```c
/* include/ck.h -- minimal C test harness (spec) */
#define CK_TEST(name)  /* defines `static void name(void)` and registers it */
#define CK_ASSERT(cond)                    /* fail with file:line and #cond   */
#define CK_ASSERT_EQ_U64(a, b)             /* prints both values on failure   */
#define CK_ASSERT_EQ_I64(a, b)
#define CK_ASSERT_EQ_PTR(a, b)
#define CK_ASSERT_EQ_STR(a, b)
#define CK_ASSERT_NEAR(a, b, tol)          /* |a-b| <= tol, prints a, b, |a-b| */
#define CK_ASSERT_ULP(a, b, n)             /* float a,b within n ULPs (P1 helper) */
#define CK_SKIP(reason)                    /* mark test skipped, e.g. no F16C   */
int ck_run_all(int argc, char **argv);     /* --filter=substr, --list, -v       */
                                           /* returns 0 iff all tests pass      */
```

Design requirements:

- Registration uses `__attribute__((constructor))` so a test file needs no central list. When you reach CS:APP § 7.7, look at how this works: constructors are function pointers placed in the ELF `.init_array` section (`readelf -S` and `objdump -s -j .init_array` on a test binary) and run by the dynamic loader before `main`.
- Each test runs in a `fork()`ed child (from project 3 onward, once you have read § 8.4); a crashing test then reports "SIGSEGV in test X" instead of killing the runner. Until then, run tests in-process.
- `CK_ASSERT_ULP` compares floats by reinterpreting their bits as signed integers (ordered for finite values of the same sign) and taking the difference; this is itself a chapter 2 exercise.
- The runner's exit code drives `make test`, which drives CI in project 6.

1.5 Makefile skeleton

```
BUILD ?= release
CC    ?= gcc
WARN   = -Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wstrict-prototypes -Werror
ifeq ($(BUILD),debug)    ; CFLAGS = -std=c17 -O0 -g3 $(WARN)
else ifeq ($(BUILD),asan); CFLAGS = -std=c17 -O1 -g -fsanitize=address,undefined -fno-
    omit-frame-pointer $(WARN)
else ifeq ($(BUILD),tsan); CFLAGS = -std=c17 -O1 -g -fsanitize=thread $(WARN)
else                     ; CFLAGS = -std=c17 -O2 -g -fno-omit-frame-pointer -march=
    native $(WARN)
endif
CPPFLAGS = -Iinclude
OUT  = build/$(BUILD)
SRC  = $(wildcard src/*.c)
OBJ  = $(SRC:src/%.c=$(OUT)/%.o)
TSRC = $(wildcard tests/*.c)
TBIN = $(OUT)/run_tests

all: $(OUT)/lib<project>.a
$(OUT)/%.o: src/%.c | $(OUT)
```

```
    $(CC) $(CPPFLAGS) $(CFLAGS) -MMD -MP -c $< -o $@
$(OUT)/lib<project>.a: $(OBJ)
    ar rcs $@ $^
$(TBIN): $(TSRC) $(OUT)/lib<project>.a
    $(CC) $(CPPFLAGS) $(CFLAGS) $^ -o $@ -lm
test: $(TBIN) ; $(TBIN) $(ARGS)
bench: ...   ; fmt: clang-format -i src/*.c include/**/*.h tests/*.c
clean: ; rm -rf build
$(OUT): ; mkdir -p $@
-include $(OBJ:.o=.d)
```

The semicolon-separated `ifeq` lines above are shorthand for readability; write them as proper multi-line conditionals. The `-MMD -MP` flags generate header dependency files so that editing a header rebuilds what includes it—a small taste of what the linker chapter formalizes.

1.6 Documentation conventions

Every README answers five questions in order: what is this; one headline measurement (a number and a plot); how to build; how to test; how to reproduce the measurement. `docs/design.md` records decisions and the alternatives rejected. The blog draft lives at `docs/blog.md` and is written *measurement-first*: the post is organized around one figure and the explanation of why it looks the way it does, not around the feature list.

1.7 How assignments are specified

Each project chapter breaks the work into assignments numbered `A<project>.<n>`. An assignment has five parts, always in the same order, so that you can tell at a glance whether you are done:

Goal
: One or two sentences: what exists at the end that did not exist before.

Steps
: The order of work. Follow it; the order is chosen so that each step has a test before the next step depends on it.

Acceptance
: The checklist that must be true before the assignment is closed. Every item is mechanically checkable—a test that passes, a file that exists, a number within a range—because "it seems to work" is how bugs survive into the next project.

Budget
: Hours, assuming 1–2 focused hours per sitting. If you are past 1.5× the budget, stop and write down in `docs/design.md` what is harder than expected; that note usually reveals a design mistake worth fixing before continuing. The budgets add up to roughly 55–70 hours per project.

Feeds
: Which blog section or figure this assignment produces. If an assignment feeds nothing, it is infrastructure; keep it small.

Assignments close with a commit (or a merged PR from project 4 on) whose message starts with the assignment id, e.g. `A2.3: quantized matvec via qnum (int8, int4)`. `git log --grep=^A2` then lists a project's history by assignment. Keep a `docs/log.md` with one dated line per sitting:

what you did, what broke, what you measured. It costs two minutes and it is the raw material for the "what went wrong" section of each post, which is the section readers remember.

1.8 How to write the blog posts

Each project yields one main post and one short companion post (the "b" post). The briefs in each chapter say what each post argues and which figures it carries; this section says how every post is built, so that the series reads as one body of work.

1.8.1 Who the reader is

Write for a competent engineer or graduate student in ML systems who has not read CS:APP and has not built the thing. They can read C, know what a cache and a GPU are, and want to leave with (a) one mechanism they now understand from the bottom, (b) one number they can quote, and (c) code they could run. Do not write for the reviewer of your next paper, and do not write for beginners; both audiences are served by the same post if the mechanism is explained from first principles and the numbers are real.

1.8.2 The anatomy of a main post (2000–3500 words)

1. **Title** ($\leq$ 10 words): a claim or a question, never a topic. "A roofline for LLM decode, measured from scratch" says what you did; "Notes on CPU inference performance" says nothing. Subtitle: the single headline number.
2. **Lede** ($\leq$ 120 words): what you built, the one-sentence result, and why someone working on inference should care. No history of the project, no "in this post I will".
3. **The central figure**, in the first screen, with a caption that states the *finding* ("Decode throughput stops improving once weights are 4 bits: the bandwidth ceiling, not the FMA units, sets the limit"), not the axes.
4. **Setup** ($\approx$ 200 words): machine (link `docs/machine.md`), model and weights, compiler and flags, the Git tag the numbers came from, how many runs and what statistic. Readers who skip this section still trust you more because it is there.
5. **Mechanism** (600–1200 words): the mathematics, *derived*, not quoted. One equation per idea, every symbol defined where it first appears, and a worked numerical example with your machine's numbers right after each equation. If the chapter's math section has a proof (the three-case RNE argument, the online-softmax invariant), the post contains it in full; proofs are what make these posts different from the dozens that paraphrase documentation.
6. **Implementation** (300–600 words): one to three code excerpts of at most 25 lines each, chosen because they show the idea (the RNE expression, the SPSC publish/consume pair, the frame-pointer walk), never for completeness. Each excerpt links to the file at the tagged commit. Describe what the compiler did with it when that is the point (`objdump` snippets of $\leq$ 15 lines).
7. **Results** (400–800 words): the remaining figures, each introduced by what to look at, followed by what it confirms and what surprised you. Report medians with spread and say how many runs. Every speedup appears next to the absolute numbers it came from.
8. **What went wrong** (150–300 words): the bug, the wrong assumption, the measurement that lied. Be specific ("the RoPE pairing convention cost two evenings; the logits matched to 10^{-2}, which is exactly the signature of a permuted rotary dimension"). This section is the most read and the most linked.

9. **What this predicts elsewhere** (150–300 words): take the mechanism to the GPU, to a FLOPs-poor accelerator, to the serving stack you actually use. Make one falsifiable prediction. This is where the post connects to your research without claiming more than you measured.
10. **Reproduce** (a boxed section): the exact commands, expected wall-clock time, expected output or figure. If it cannot be reproduced from the tagged repo and `scripts/fetch_data.sh`, the post is not finished.

The companion "b" post (1000–1800 words) keeps items 1–3, 5, 8 and 10 and drops the rest; it exists to give one narrow idea a page of its own so it can be linked on its own.

1.8.3 Figure standards

- One message per figure. If you need the word "and" to describe what a figure shows, split it.
- Axes labelled with units; log axes whenever the data spans more than two decades (bytes, working-set sizes, latency); the derived bound or model drawn as a dashed line *on the same axes* as the measurement.
- Caption states the finding and names the tag the data came from. Legends inside the plot area only when they do not cover data.
- Colourblind-safe palette (Okabe–Ito or viridis), the same across the whole series; markers distinguish series in addition to colour.
- Export SVG (preferred) or PNG at 2× resolution; width at most the blog's text column. Every figure's data (CSV) and plotting script live in `scripts/plots/` in the repo, and the figure file name matches the script name.
- Error bars or shaded spread whenever a point is a statistic over runs; say what the bar is.

1.8.4 Writing rules

- Every claim is measured (with the figure), derived (with the equation), or cited (with the link). No fourth category.
- Past tense for what you did, present tense for what is true. First person singular is fine.
- Explain in the order of ideas, not the order you discovered them; the chronology goes in "what went wrong".
- Numbers carry their uncertainty and their conditions ("38.1 ± 0.4 GB/s, single core, 2 MB pages").
- Never "simply", "just", "obviously". If it were, the post would not be needed.
- Define jargon the first time (TTFT, ULP, PLT). A one-line glossary at the end is better than parentheses throughout.
- Cut the paragraph that explains why the topic is important in general. The lede already did that with a specific reason.

1.8.5 Workflow and cadence

1. At the project's first assignment, write the post's title, thesis, and the central figure's axes into `docs/blog.md`. You are writing down a hypothesis; the project will test it.
2. As assignments complete, drop figures and "what went wrong" notes into the draft the same day. Do not write prose yet.
3. When the last measurement assignment closes, write the prose in two sittings: mechanism and implementation first, then results and predictions. Leave it for three days, then read it aloud once and cut 20%.

4. Have one person who did not build it read it; ask only "where did you stop understanding?" Fix that place.
5. Publish: copy to the blog's `_posts/` with the date, link the Git tag and the `reproduce` commands, and add the post to the series index page. Tag the repo `blog-<n>` at the exact commit the numbers came from if it differs from the release tag.
6. Post the link in one or two places where ML-systems people read (a relevant Discord/Slack, the `r/MachineLearning` or Hacker News crowd if the result is strong). Answer every substantive comment; corrections go into the post with a dated note, never silently.

1.8.6 Pre-publication checklist

1. The central figure appears in the first screen and its caption states a finding.
2. Every equation's symbols are defined and at least one has a worked example with real numbers.
3. Every number has units, conditions, and (if a statistic) spread.
4. Every code excerpt is ≤ 25 lines and links to a tagged file.
5. The "what went wrong" section names a specific bug or wrong assumption.
6. There is one falsifiable prediction about another platform.
7. The reproduce box has been executed from a fresh clone on the tagged commit.
8. The post links the repo, the tag, `docs/machine.md`, and the previous post in the series.
9. A reader who skips all prose and reads only figure captions gets the argument.
10. The post is under 3500 words (main) or 1800 words (companion) after the 20% cut.

1.9 Git fundamentals (taught once, used everywhere)

The Git lessons in projects 1–6 build on this base. Appendix A is the command reference.

1.9.1 The object model

Git is a content-addressed filesystem with a commit graph on top. There are four object types, each identified by the SHA-1 of its contents (SHA-256 repositories exist but are rare):

blob
: the bytes of one file, with no name. Two identical files anywhere in history are one blob.

tree a directory: a list of (mode, type, hash, name) entries pointing to blobs and subtrees.

commit
: a pointer to a root tree, zero or more parent commits, author/committer with timestamps, and a message. A commit's hash therefore covers the entire snapshot and its entire history: change anything and every descendant hash changes.

tag (annotated) a named, signed-able pointer to a commit with a message.

Objects live in `.git/objects/ab/cdef...` (first two hex digits are the directory), zlib-compressed, and are later packed into delta-compressed *packfiles*. Inspect them yourself after your first commit:

```
git cat-file -p HEAD          # the commit
git cat-file -p HEAD^{tree}   # its root tree
git cat-file -p HEAD:src/x.c  # a blob by path
echo -n 'hello' | git hash-object --stdin   # content addressing by hand
```

1.9.2 Refs, HEAD, and the three trees

A *branch* is a 41-byte file `.git/refs/heads/<name>` containing a commit hash; nothing more. `HEAD` is a symbolic ref (`ref: refs/heads/main`) naming the current branch; a detached HEAD holds a hash directly. Remote-tracking branches `refs/remotes/origin/main` record where the remote was at your last fetch.

Three "trees" are involved in every command: the *working tree* (your files), the *index* or staging area (`.git/index`, the proposed next commit), and `HEAD` (the last commit). `git add` copies working tree → index; `git commit` snapshots index → new commit and moves the branch; `git diff` compares working tree vs index; `git diff --staged` compares index vs HEAD; `git restore --staged` copies HEAD → index; `git restore` copies index → working tree. Every "how do I undo" question is answered by naming which of the three you want to change.

1.9.3 The baseline workflow

```
git config --global user.name "Utkarsh Kumar"
git config --global user.email "<the email on your GitHub account>"
git config --global init.defaultBranch main
git config --global pull.rebase true        # linear history by default
ssh-keygen -t ed25519 -C "<email>"          # then add ~/.ssh/id_ed25519.pub to GitHub
ssh -T git@github.com                        # verify

git init qnum && cd qnum
# ... create skeleton, .gitignore, README ...
git add -A && git status
git commit -m "Initial project skeleton"
gh repo create <user>/qnum --public --source=. --remote=origin --push
# or: git remote add origin git@github.com:<user>/qnum.git && git push -u origin main
```

1.9.4 Commit discipline

- One logical change per commit; a commit should build and pass tests on its own (project 5 depends on this for `git bisect run`).
- Message: imperative subject line ≤ 50 characters ("Add RNE rounding for fp32→bf16"), blank line, body explaining *why* and anything non-obvious, wrapped at 72. The diff already says *what*.
- Stage hunks, not files: `git add -p` lets you split one editing session into several coherent commits.
- Never commit generated files, data, or secrets. If you do, see the reflog section in project 3's lesson—but remember that pushed history is public history.

Chapter 2

Project 1: `qnum` — a low-precision numerics library

2.1 Goal

A C library implementing every number format that matters for inference—fp16, bf16, fp8 (E4M3 and E5M2), symmetric and asymmetric integer quantization (int8, int4 with per-group scales), and block floating point—with conversions to and from fp32 written purely with integer bit manipulation, every rounding mode, correct handling of subnormals, infinities, NaNs and saturation, and a measurement suite that quantifies the error each format introduces on real model weights.

Research hook

KV-cache compression, weight quantization, and the accelerator formats you will meet at Tenstorrent (block floating point) or on Hopper/Blackwell (fp8, fp4 with micro-scaling) all reduce to the bit manipulations in this project. It is also the place where compiler and architecture differences first become visible in numerical output—the same question as your WRF compiler study, at the scale of one instruction.

2.2 CS:APP reading map

Section	What you build with it
§ 2.1 bit-level ops, shifts, byte order	all encoders/decoders; the safetensors header is little-endian
§ 2.2–2.3 integer encodings, overflow	integer quantization, saturation, two's-complement int4 packing
§ 2.4 IEEE floating point, rounding	every float format in this chapter
§ 3.1–3.6 x86-64 asm, control	reading `objdump` of your converters at `-O0`/`-O2`
§ 3.11 floating-point code (SSE/AVX)	comparing against `vcvtps2ph`, `vcvtneps2bf16`

2.3 Mathematical background

2.3.1 Binary floating-point formats

A binary floating-point format with w exponent bits and $p-1$ stored fraction bits (p is the *precision*, counting the hidden leading 1) encodes a sign s, a biased exponent $e \in [0, 2^w - 1]$ and a fraction $f \in [0, 2^{p-1})$ with bias $b = 2^{w-1} - 1$:

$$x = \begin{cases} (-1)^s \, (1 + f\, 2^{-(p-1)})\, 2^{\,e-b} & 1 \le e \le 2^w - 2 \quad \text{(normal)} \\ (-1)^s \, (f\, 2^{-(p-1)})\, 2^{\,1-b} & e = 0,\ f \neq 0 \quad \text{(subnormal)} \\ (-1)^s \, 0 & e = 0,\ f = 0 \\ (-1)^s \, \infty & e = 2^w - 1,\ f = 0 \\ \text{NaN} & e = 2^w - 1,\ f \neq 0 \end{cases} \tag{2.1}$$

Subnormals have the same spacing $2^{1-b-(p-1)}$ as the smallest normals (*gradual underflow*), which is why IEEE formats are monotone in their bit patterns: for non-negative values, $x \le y \iff \text{bits}(x) \le \text{bits}(y)$ when the bits are read as unsigned integers. This fact is the basis of your ULP comparison and of several tests.

Format	s	w	$p-1$	bias	max finite	min normal / min subnormal	∞, NaN
fp32	1	8	23	127	$\approx$ 3.40e38	2^{-126} / 2^{-149}	IEEE
fp16	1	5	10	15	65504	2^{-14} / 2^{-24}	IEEE
bf16	1	8	7	127	$\approx$ 3.39e38	2^{-126} / 2^{-133}	IEEE
E5M2	1	5	2	15	$57344 = 1.75 \cdot 2^{15}$	2^{-14} / 2^{-16}	IEEE
E4M3	1	4	3	7	$448 = 1.75 \cdot 2^{8}$	2^{-6} / 2^{-9}	no ∞; NaN = s.1111.111

E4M3 (OCP FP8 specification) is *not* IEEE-shaped: the top exponent $e = 15$ is used for finite values ($448 = 1.111_2 \cdot 2^8$ with $f = 111$) except the single pattern $f = 111$, which is NaN. There are no infinities; overflow produces NaN (or saturates to ± 448 in *saturating* mode—implement both as a flag). E5M2 is exactly binary16 with 8 fewer fraction bits and behaves like IEEE.

bf16 is the upper 16 bits of fp32: same exponent field, 7 fraction bits. Conversion is therefore a pure rounding of the low 16 bits with no exponent-range handling.

Unit roundoff. With round-to-nearest the relative error of representing a real x in the normal range is bounded by $u = 2^{-p}$:

$$\left| \frac{\hat{x} - x}{x} \right| \le u = 2^{-p}. \tag{2.2}$$

Thus $u_{\text{fp32}} = 2^{-24} \approx$ 6e−8, $u_{\text{fp16}} = 2^{-11} \approx$ 4.9e−4, $u_{\text{bf16}} = 2^{-8} \approx$ 3.9e−3, $u_{\text{E4M3}} = 2^{-4} = 6.25\%$, $u_{\text{E5M2}} = 2^{-3} = 12.5\%$. The two fp8 formats trade range (E5M2) against precision (E4M3); your measurements on real weights should show E4M3 winning for weights and E5M2 for gradients/activations with outliers.

2.3.2 Rounding by integer arithmetic

Let X be the 32-bit unsigned integer whose bits are the fp32 value x, and suppose we keep the top k fraction bits and drop $d = 23 - k$. Write $X = Q \cdot 2^d + R$ with $R = X \bmod 2^d$ the discarded bits and $H = 2^{d-1}$ the halfway point.

Round toward zero (truncation)

Q.

Round to nearest, ties to even (RNE)

$$Q_{\mathrm{RNE}} = \left\lfloor \frac{X + H - 1 + (Q \bmod 2)}{2^d} \right\rfloor. \tag{2.3}$$

Check the three cases: $R < H$ adds less than 2^d to $Q2^d$, so the floor is Q; $R > H$ pushes past the boundary, giving $Q+1$; $R = H$ adds exactly $2^d - 1 + (Q \bmod 2)$, which crosses only when Q is odd—rounding to the even neighbour. The familiar bf16 idiom `(X + 0x7FFF + ((X >> 16) & 1)) >> 16` is Equation (2.3) with $d = 16$.

Stochastic rounding

draw U uniform on $\{0, \ldots, 2^d - 1\}$ and return $\lfloor (X + U)/2^d \rfloor$. Then Pr[round up] $= R/2^d$ and $\mathbb{E}[\hat{x}] = x$ exactly—the estimator is unbiased, which is why it matters for training and for accumulating many small updates, and why it is useless for a single weight tensor (it adds variance without benefit). Use a 64-bit PCG or xoshiro256 generator; never `rand()`.

Carry propagation is a feature. When Q's fraction field is all ones and rounding adds one, the carry propagates into the exponent field: $1.111\ldots \to 10.000\ldots$ renormalizes to $1.000 \times 2^{e+1}$, which is exactly right. The same carry turns the largest finite value into infinity for IEEE formats—also right. For E4M3 the carry would produce the NaN pattern, so you must detect overflow (compare against the max-finite bit pattern) *before* packing.

2.3.3 Narrowing the exponent range (fp32 → fp16/fp8)

bf16 shares fp32's exponent, so Equation (2.3) is the whole conversion. For fp16 and fp8 the target has a smaller exponent range and the algorithm is:

1. Extract s, e, f from X. Handle $e = 255$ separately: $\pm\infty \to \pm\infty$ (E4M3: NaN or saturate), NaN → a quiet NaN of the target (keep the sign, set the top fraction bit, keep at least one fraction bit set).
2. Compute the unbiased exponent $E = e - 127$ (for fp32 subnormals, $E = -126$ and the hidden bit is 0; for the formats here every fp32 subnormal underflows to ± 0 in the target, so you may short-circuit).
3. Form the 24-bit significand $M = 2^{23} + f$.
4. Let $E_{\min} = 1 - b_t$ be the target's minimum normal exponent.
   - If $E > E_{\max}$ (after rounding; see step 6): overflow $\to \pm\infty$ or saturate.
   - If $E \geq E_{\min}$: *normal path.* Drop $d = 23 - (p_t - 1)$ bits of M with the chosen rounding; the target biased exponent is $E + b_t$; if rounding carried out of the significand, the carry lands in the exponent (correct).
   - If $E < E_{\min}$: *subnormal path.* The target can only represent multiples of $2^{E_{\min} - (p_t - 1)}$. Shift M right by $d + (E_{\min} - E)$ bits with rounding—the extra shift is exactly how many binary places x is below the normal range. The target exponent field is 0. If the shift exceeds 24 bits the result is ± 0 (or the smallest subnormal with probability $R/2^d$ under stochastic rounding). If rounding carries into bit $p_t - 1$, the value becomes the smallest normal: the exponent field becomes 1, again by the natural carry if you build the packed value as `(exp << (p_t-1)) + rounded_sig`.
5. Pack s, exponent, fraction.
6. Overflow check: compare the packed magnitude against the format's maximum finite pattern (fp16: `0x7BFF`; E5M2: `0x7B`; E4M3: `0x7E`). Anything greater is overflow.

The widening direction (fp16 → fp32) is simpler: normals re-bias the exponent ($e_{32} = e_{16} - 15 +$

127) and shift the fraction left by 13; subnormals need normalization—find the leading one with `__builtin_clz`, shift it into the hidden position, and set $e_{32} = 127 - 14 - \text{shift}$; infinities and NaNs map to $e_{32} = 255$ with the fraction shifted left.

2.3.4 Integer quantization

Given a group $x_1, \ldots, x_g$ of fp32 values and B bits:

Symmetric. $q_{\max} = 2^{B-1} - 1$ (7 for int4, 127 for int8); the full negative range -2^{B-1} is deliberately unused so that the grid is symmetric.

$$\Delta = \frac{\max_i |x_i|}{q_{\max}}, \qquad q_i = \operatorname{clamp}\Big(\operatorname{round}\Big(\frac{x_i}{\Delta}\Big), -q_{\max}, q_{\max}\Big), \qquad \hat{x}_i = \Delta\, q_i. \tag{2.4}$$

Asymmetric (with zero point). Uses the full range $[0, 2^B - 1]$:

$$\Delta = \frac{\max_i x_i - \min_i x_i}{2^B - 1}, \qquad z = \operatorname{round}\Big(-\frac{\min_i x_i}{\Delta}\Big), \tag{2.5}$$

$$q_i = \operatorname{clamp}\Big(\operatorname{round}\Big(\frac{x_i}{\Delta}\Big) + z,\ 0,\ 2^B - 1\Big), \qquad \hat{x}_i = \Delta\,(q_i - z). \tag{2.6}$$

Store Δ as bf16 or fp16 and z as a B-bit unsigned. Storage cost per weight with group size g and a 16-bit scale is $B + 16/g$ bits (int4, $g = 32$: 4.5 bits; with an 8-bit zero point: 4.75).

Error. Within the clipping range the error $x_i - \hat{x}_i$ is approximately uniform on $[-\Delta/2, \Delta/2]$, so

$$\mathbb{E}\big[(x - \hat{x})^2\big] = \frac{\Delta^2}{12}, \qquad \text{SQNR} = 10 \log_{10} \frac{\sum_i x_i^2}{\sum_i (x_i - \hat{x}_i)^2} \text{ dB}. \tag{2.7}$$

Because Δ is set by the group's largest magnitude, one outlier inflates Δ for its whole group and crushes the small values; smaller groups contain the damage at the cost of more scale bits. Your measurement suite should plot SQNR against g for real weight matrices and show exactly this trade-off.

Packing int4. Two's-complement nibbles, element $2j$ in the low nibble of byte j and $2j + 1$ in the high nibble. Unpacking requires sign extension: `(int8_t)(n << 4) >> 4` for the low nibble is the standard idiom (arithmetic right shift of a signed value—§ 2.3; note that right-shifting negative signed integers is implementation-defined in C17 but arithmetic on every compiler you will use; say so in a comment).

2.3.5 Block floating point

A block floating-point (BFP) format shares one exponent among a block of n elements and stores a short signed mantissa per element. Tenstorrent's BFP8/BFP4/BFP2 formats are of this kind (blocks of 16 with an 8-bit shared exponent and 8/4/2-bit elements). Implement the general family BFP(m, n): n elements, each a sign bit plus an m-bit mantissa *with an explicit leading one* (no hidden bit, because small elements will have leading zeros after alignment), and an 8-bit shared biased exponent.

Encoding a block $x_1, \dots, x_n$ with fp32 exponents e_i (biased) and 24-bit significands M_i:

$$e_s = \max_i e_i \quad \text{(shared exponent)}, \tag{2.8}$$

$$\tilde{M}_i = \operatorname{round}\Big(M_i \cdot 2^{-(24-m)} \cdot 2^{-(e_s - e_i)}\Big) \quad \text{(align to shared exponent, keep } m \text{ bits)}, \tag{2.9}$$

$$\hat{x}_i = (-1)^{s_i}\, \tilde{M}_i\, 2^{\,e_s - 127 - (m-1)}. \tag{2.10}$$

Elements with $e_s - e_i \geq m$ become zero. Storage is $m + 1 + 8/n$ bits per element: BFP$(7, 16)$ costs 8.5 bits.

Error. The absolute error of every element in the block is at most half a unit of the block's grid, $\frac{1}{2}2^{e_s - 127 - (m-1)}$, so the *relative* error of element i is roughly

$$\left|\frac{\hat{x}_i - x_i}{x_i}\right| \lesssim 2^{\,(e_s - e_i) - m}, \tag{2.11}$$

i.e. it doubles for every octave an element sits below the block maximum. This is the same outlier sensitivity as group integer quantization (BFP is, in fact, integer-group quantization with a power-of-two scale), and the comparison between BFP$(7, 16)$ and int8-with-fp16-scale-per-16 is a natural figure for your blog.

2.3.6 Comparing floats: ULPs

For two finite floats of the same sign, the number of representable values between them is $|\text{bits}(a) - \text{bits}(b)|$ as integers. For mixed signs, map each to a signed ordering key $k(x) = \text{bits}(x)$ if $x \geq 0$ else $-(\text{bits}(x) \wedge \text{0x7FFFFFFF})$, then take $|k(a) - k(b)|$. Implement this as `qn_ulp_distance` and use it in `CK_ASSERT_ULP`.

2.3.7 The safetensors container

A safetensors file is: an 8-byte little-endian unsigned header length N; N bytes of JSON mapping tensor names to {`dtype, shape, data_offsets:[begin,end]`}; then the raw little-endian tensor bytes, offsets relative to the end of the header. Reading it needs `mmap` (or `fread`) and a minimal JSON parser; writing a 200-line recursive-descent JSON parser is worthwhile, or vendor `jsmn`. Weights for testing: export a small Hugging Face model with `safetensors.torch.save_file`, or use the `stories15M` checkpoint from `llama2.c` converted by a script you write in `scripts/`.

2.4 Architecture

2.4.1 Directory layout

```
qnum/
  include/qnum/
    fp.h          fp16, bf16, e4m3, e5m2 <-> fp32; rounding modes; classification
    intq.h        int8/int4 symmetric & asymmetric group quantization, packing
    bfp.h         block floating point encode/decode
    stats.h       error metrics: max abs, rmse, sqnr, ulp histograms
    rng.h         PCG32/xoshiro256** for stochastic rounding
    safetensors.h minimal reader
    ulp.h         ordering key and ULP distance
  src/            fp.c intq.c bfp.c stats.c rng.c safetensors.c json.c
```

```
  tests/          test_fp16.c test_bf16.c test_fp8.c test_intq.c test_bfp.c test_stats.
    c test_safetensors.c
  bench/          bench_convert.c   (throughput of each converter, scalar vs intrinsics
    )
  scripts/        gen_golden.py (numpy/torch reference tables), plot_sqnr.py,
    export_weights.py
  docs/           machine.md design.md blog.md
```

2.4.2 Public API

```c
/* fp.h */
typedef uint16_t qn_f16;  typedef uint16_t qn_bf16;
typedef uint8_t  qn_e4m3; typedef uint8_t  qn_e5m2;
typedef enum { QN_RNE = 0, QN_RTZ, QN_STOCHASTIC } qn_round;
typedef struct { bool saturate; } qn_fp8_opts;   /* E4M3 overflow: NaN (default) or
    +-448 */
typedef struct qn_rng qn_rng;                     /* rng.h: PCG32 state */

qn_f16  qn_f32_to_f16 (float x, qn_round r, qn_rng *rng);
qn_bf16 qn_f32_to_bf16(float x, qn_round r, qn_rng *rng);
qn_e4m3 qn_f32_to_e4m3(float x, qn_round r, qn_rng *rng, qn_fp8_opts o);
qn_e5m2 qn_f32_to_e5m2(float x, qn_round r, qn_rng *rng);
float   qn_f16_to_f32 (qn_f16 h);
float   qn_bf16_to_f32(qn_bf16 h);
float   qn_e4m3_to_f32(qn_e4m3 h);
float   qn_e5m2_to_f32(qn_e5m2 h);

/* bulk versions: n elements, may use F16C/AVX-512 when compiled with -march=native */
void qn_f32_to_f16_n(const float *x, qn_f16 *y, size_t n, qn_round r, qn_rng *rng);
void qn_f16_to_f32_n(const qn_f16 *x, float *y, size_t n);
/* ... same for bf16, e4m3, e5m2 ... */

/* classification, format-generic */
typedef struct { uint8_t exp_bits, frac_bits; int bias; bool has_inf; } qn_fmt;
extern const qn_fmt QN_FMT_F16, QN_FMT_BF16, QN_FMT_E4M3, QN_FMT_E5M2;
bool qn_is_nan(uint32_t bits, qn_fmt f);  bool qn_is_inf(uint32_t bits, qn_fmt f);
bool qn_is_subnormal(uint32_t bits, qn_fmt f);
```

```c
/* intq.h */
typedef struct {
  uint8_t  bits;        /* 4 or 8 */
  bool     symmetric;
  size_t   group;       /* elements per scale, e.g. 32, 64, 128; 0 = per-tensor */
} qn_intq_cfg;

typedef struct {
  qn_intq_cfg cfg;
  size_t   n;
  uint8_t *q;           /* packed: int8 -> n bytes; int4 -> ceil(n/2) bytes */
  qn_bf16 *scale;       /* ceil(n/group) scales */
  uint8_t *zero;        /* asymmetric only, ceil(n/group) entries */
} qn_intq_tensor;
```

```
size_t qn_intq_bytes(qn_intq_cfg c, size_t n);           /* total storage incl.
    metadata */
void   qn_intq_quantize(const float *x, size_t n, qn_intq_cfg c, qn_intq_tensor *out);
void   qn_intq_dequantize(const qn_intq_tensor *t, float *x);
int8_t qn_int4_unpack_lo(uint8_t b);  int8_t qn_int4_unpack_hi(uint8_t b);
uint8_t qn_int4_pack(int8_t lo, int8_t hi);

/* bfp.h */
typedef struct { uint8_t mant_bits; uint16_t block; } qn_bfp_cfg;   /* e.g. {7,16} */
void qn_bfp_encode(const float *x, size_t n, qn_bfp_cfg c, qn_round r,
                   uint8_t *shared_exp /* ceil(n/block) */, uint8_t *mant /* n */);
void qn_bfp_decode(const uint8_t *shared_exp, const uint8_t *mant, size_t n,
                   qn_bfp_cfg c, float *x);

/* stats.h */
typedef struct { double max_abs, rmse, mean_rel, sqnr_db; uint64_t ulp_hist[33]; }
    qn_err;
qn_err qn_compare(const float *ref, const float *approx, size_t n);

/* ulp.h */
int32_t  qn_order_key(float x);
uint32_t qn_ulp_distance(float a, float b);
```

2.4.3 Implementation notes

- Type punning float↔uint32 goes through `memcpy` (the compiler turns it into a register move); never through a pointer cast, which violates strict aliasing.
- Write the scalar converters first in a single generic routine parameterized by `qn_fmt`, then specialize once it is tested; the specializations must be bit-identical to the generic one (a test).
- The bulk functions dispatch at runtime on `__builtin_cpu_supports("f16c")` / `"avx512bf16"` to intrinsics (`_mm256_cvtps_ph` with `_MM_FROUND_TO_NEAREST_INT`, `_mm512_cvtneps_pbh`) and fall back to the scalar path. The intrinsics are the oracle for your scalar code, not the other way round.
- Make the library allocation-free: callers own buffers; `qn_intq_bytes` tells them how much to allocate.

2.5 Assignments

A1.1 Repository, harness, build *6 h*

Goal. A repository that builds with both compilers under all four `BUILD` variants, runs an (empty) test suite, and is published.

Steps. (1) Create the layout from Section 1; write `.gitignore` first. (2) Write `ck.h`: the macros, constructor-based registration, `--filter`, `--list`, a summary line, non-zero exit on failure. (3) Makefile with the variant logic and `-MMD` dependencies. (4) `tests/test_ck.c` that tests the harness itself: a passing assertion, a deliberately failing one guarded by an environment variable, and `CK_ASSERT_ULP` on adjacent floats. (5) README with the five questions answered, headline number marked "TBD". (6) `docs/machine.md` with `lscpu`/cache sizes. (7) First commit and push.

Acceptance. `make test` and `make BUILD=asan test` pass with `CC=gcc` and `CC=clang`; `-Wconversion` produces no warnings; `objdump -s -j .init_array` on the test binary shows one

entry per test; tag `v0.0.1` is on GitHub.
Feeds. Nothing directly; the harness serves all six projects.

A1.2 fp16 and bf16, both directions, three rounding modes *12 h*

Goal. Bit-exact conversions between fp32 and the two 16-bit formats, verified exhaustively.
Steps. (1) `ulp.h`: ordering key and ULP distance; test on hand-picked pairs including ± 0 and sign changes. (2) `rng.h`: PCG32 with seed and a `next_bits(d)` helper. (3) The generic narrowing routine parameterized by `qn_fmt`, implementing the six-step algorithm of Section 2 literally, RTZ first, then RNE (Equation (2.3)), then stochastic. (4) Widening for both formats, including subnormal normalization with `__builtin_clz`. (5) `scripts/gen_golden.py` producing `tests/golden/f16.csv` and `bf16.csv` (random plus the edge-case list). (6) The exhaustive, golden, monotonicity, ties and stochastic tests. (7) Specialize the hot paths once the generic routine is green; add the bit-identity test between generic and specialized.
Acceptance. All 2^{16} round trips for both formats; goldens match; monotonicity over 10^6 pairs; tie cases round to even in both carry directions; stochastic means within 4σ; the specialized paths equal the generic one on 2^{24} random inputs; ASan clean. Commit per step; tag `v0.1.0`.
Feeds. Post 1 sections 5–6 (mechanism), the RNE proof, the worked subnormal example; Post 1b's method.

A1.3 fp8: E4M3 and E5M2 *8 h*

Goal. The two OCP fp8 formats, with E4M3's non-IEEE top binade and both overflow behaviours.
Steps. (1) Add the two `qn_fmt` entries and the `has_inf` flag. (2) Implement the E4M3 overflow check against `0x7E` before packing; implement the saturating option. (3) NaN canonicalization for a format with a single NaN pattern. (4) Goldens from `torch.float8_e4m3fn` and `float8_e5m2` (note that `e4m3fn` is the no-infinity variant—say so in a comment). (5) Exhaustive 2^8 round trips, overflow-by-one-ULP cases, subnormal boundary cases (2^{-9}, 2^{-16} and their halves).
Acceptance. Goldens match in both overflow modes; every E4M3 pattern except `0x7F`/`0xFF` round-trips; a value $448 + \frac{1}{2}$ ulp becomes NaN (non-saturating) or 448 (saturating); fp32 values in $[2^{-10}, 2^{-9})$ round to 0 or 2^{-9} exactly as RNE dictates.
Feeds. Post 1 section 7 ("why E4M3 has no infinity") and the range-vs-precision figure.

A1.4 Integer group quantization and block floating point *10 h*

Goal. int8/int4 symmetric and asymmetric group quantizers with packing, and the BFP(m, n) family.
Steps. (1) `qn_intq_bytes` and the storage layout (document it in the header). (2) Symmetric quantizer, then asymmetric; scales stored as bf16 via A1.2. (3) int4 pack/unpack with the sign-extension idiom and a comment on implementation-defined right shifts. (4) BFP encode: per-block max exponent, alignment shifts, rounding, explicit leading one; decode. (5) Property tests from the test plan; a test that n not divisible by the group size is handled. (6) `qn_compare` in `stats.h` with the ULP histogram.
Acceptance. All quantization tests pass; `qn_intq_bytes` matches the $B + 16/g$ formula; BFP encode/decode idempotent; `qn_compare` on known inputs gives the expected SQNR to 0.01 dB.
Feeds. Post 1 sections 8–9 (BFP as integer quantization with power-of-two scales), the $e_s - e_i$ histogram.

A1.5 Measurement suite on a real checkpoint *10 h*

Goal. A table and three figures quantifying every format on real weights.
Steps. (1) Minimal JSON parser (or vendor `jsmn`) and the safetensors reader; `scripts/export_weights.py` converting `stories110M` (and optionally TinyLlama) to safetensors. (2) `bench/quant_report.c`: for every 2-D tensor and every format/group configuration, compute `qn_compare` on the weights and the relative Frobenius error of $W\hat{x}$ for 16 random x. Write a CSV. (3) `scripts/plot_sqnr.py`: SQNR vs bits-per-weight scatter (one point per configuration, colour by format); SQNR by layer type; $e_s - e_i$ histograms for an attention and an MLP matrix. (4) Headline number into the README.
Acceptance. CSV with one row per (tensor, format, group); the three figures in `docs/figures/` regenerate from the CSV by one command; README headline updated; `docs/log.md` has the surprising rows noted.
Feeds. Post 1 central figure and results section.

A1.6 Assembly study and hardware oracles *8 h*

Goal. Read what the compiler does with your converter; make the hardware instruction the oracle for your software.
Steps. (1) `objdump -d -M intel` of `qn_f32_to_bf16` at `-O0`, `-O2`, `-O2 -march=native`, for gcc and clang; paste and annotate in `docs/asm-notes.md`: where the rounding add is, whether a branch survived for NaN, what `-march=native` changed. (2) Add the F16C/AVX-512-BF16 fast paths with runtime dispatch. (3) `bench/exhaustive_f16.c`: all 2^{32} fp32 inputs through scalar RNE vs `_mm256_cvtps_ph`; report mismatches (expect zero) and wall time. (4) `bench/bench_convert.c`: GB/s for scalar `-O0`, scalar `-O2`, intrinsics, for each format.
Acceptance. Zero mismatches over 2^{32} (document the runtime); throughput table in the README; the annotated assembly explains at least three differences between `-O0` and `-O2`.
Feeds. Post 1 section 10 (compiler aside) and all of Post 1b.

2.6 Test plan

Exhaustive round trips
For all 2^{16} fp16 patterns: `f32_to_f16(f16_to_f32(h)) == h` for every non-NaN h; NaNs stay NaN. Same for bf16 and for both fp8 formats (2^8 patterns).

Exhaustive against the oracle
For all 2^{16} *fp16* values widened to fp32 and then re-narrowed under RNE, every rounding mode agrees (there is nothing to round). For 2^{24} random fp32 values compare the scalar path to the F16C intrinsic (skip with `CK_SKIP` when the CPU lacks it); the full 2^{32} sweep lives in `bench/`.

Golden tables
`scripts/gen_golden.py` writes CSVs of (fp32 bits, expected fp16/bf16 bits) using `numpy.float16` and `torch.bfloat16`, plus fp8 via `torch.float8_e4m3fn`/`e5m2`, including hand-picked edge cases: ± 0, smallest subnormal, largest subnormal, smallest normal, max finite, max finite $+\frac{1}{2}$ ulp, ties at every magnitude, values that round up into the next binade, values that overflow by one ULP, NaN payloads.

Monotonicity
For 10^6 random pairs $a \le b$ (finite, non-NaN): `to_f16(a) <= to_f16(b)` under the ordering key, for every format and RNE/RTZ.

Ties to even
Construct values exactly halfway between two fp16 neighbours; check even results in both directions of the carry (including $1.111\ldots1\,|\,1 \to 10.000$ at a binade boundary).

Stochastic rounding
For 100 values, 10^6 draws each: sample mean within 4σ of x where $\sigma = \Delta\sqrt{p(1-p)/10^6}$ and $p = R/2^d$. Also: with $R = 0$ the result never changes.

Integer quantization
Dequantization error $\leq \Delta/2 + \epsilon$ for every in-range element; clipping only when $|x| > q_{\max}\Delta$ (never, for symmetric); int4 pack/unpack identity for all 256 byte values; group boundaries (n not a multiple of the group size).

BFP
Elements equal to the block max are exact to m bits; an element m octaves below the max decodes to zero; encode/decode is idempotent (encode(decode(encode(x))) = encode(x)).

Stats
`qn_compare` on identical arrays gives zero error and SQNR $= +\infty$ (represent as `INFINITY`); on x vs $x(1 + 10^{-3})$ gives SQNR ≈ 60 dB.

Safetensors
A file written by Python round-trips: names, dtypes, shapes, and a checksum of each tensor's bytes.

Sanitizers
`make BUILD=asan test` clean; `-Wconversion` clean (this is where you will discover implicit int promotions in shift expressions).

2.7 Measurements and blog briefs

Produce with `bench/` and `scripts/`: (i) SQNR per layer for bf16, fp16, E4M3, E5M2, int8 ($g = \{32, 128, \text{tensor}\}$), int4 ($g = \{32, 64, 128\}$), BFP(7, 16), BFP(3, 16); (ii) the distribution of $e_s - e_i$ within BFP blocks for attention vs MLP weights; (iii) conversion throughput in GB/s for scalar `-O0`, scalar `-O2`, intrinsics; (iv) the end-to-end effect: relative Frobenius error of $W\hat{x}$ vs Wx for a few layers.

Blog plan

Post 1. "Every inference number format, from the bits up"
Audience and thesis. For people who use `torch.bfloat16` or `--quantization awq` daily without having written a rounding routine. Thesis: all low-precision formats are one algorithm—align, round, pack—with different constants, and they all fail the same way: the largest value in a group sets the grid for the smallest. Subtitle carries the headline: e.g. "int4 with 32-element groups keeps 38 dB on `stories110M`; BFP8 keeps 41 dB at twice the bits".
Length. 3000–3500 words; this is the longest post because it carries the most derivation.
Figures.
1. **Figure 1.** (central) SQNR (dB, y) vs bits per weight (x, 3–16), one marker per (format, group size), colour by family (float/int/BFP), averaged over all 2-D tensors with min–max whiskers. Caption states which format wins at each bit budget.
2. **Figure 2.** Bit-layout diagram of the five float formats side by side (an SVG you draw), with the hidden bit and the bias annotated.
3. **Figure 3.** Number line of representable values near 1.0 for fp16, bf16, E4M3, E5M2 (log x),

showing spacing; this is what makes "unit roundoff" visible.

4. **Figure 4.** Histogram of $e_s - e_i$ (octaves below block max) for one attention and one MLP matrix, with the "bits lost" axis on top.
5. **Figure 5.** SQNR vs group size g for int4 and int8, showing the $16/g$ scale overhead on a secondary axis.
6. **Figure 6.** Conversion throughput (GB/s) bar chart: scalar `-O0`, scalar `-O2`, F16C, per format.

Outline.

1. Lede: the one algorithm, the one failure mode, the headline number.
2. Fig. 1 and its reading.
3. Setup: checkpoint, which tensors, how SQNR is computed, tag.
4. IEEE encoding (Equation (2.1)) with Fig. 2; the monotonicity-of-bit-patterns fact and why it matters for testing.
5. Rounding with integers: Equation (2.3) and the three-case proof written out; the bf16 one-liner; why carry propagation is a feature. Code excerpt: the 6-line RNE.
6. Narrowing the exponent: the subnormal path, with a worked fp32→E4M3 example on a real weight value (show the bits at each step).
7. Why E4M3 has no infinity: what the extra binade buys, what saturation costs; Fig. 3.
8. Integer quantization: the symmetric and asymmetric formulas, the $\Delta^2/12$ error, the $B+16/g$ storage; Fig. 5.
9. BFP is integer quantization with power-of-two scales: the encoding, the relative error bound $2^{(e_s-e_i)-m}$, Fig. 4, and why MLP matrices suffer more.
10. The compiler aside: 15 lines of `objdump` at `-O2`, what `-march=native` does, Fig. 6.
11. What went wrong: candidates from `docs/log.md`—the NaN that became infinity, the sign-extension of int4, the `-Wconversion` shift promotion.
12. What this predicts: KV-cache quantization (keys have outliers per channel; predict that per-channel scaling beats per-token for K and the reverse for V) and the BFP behaviour to expect on Tenstorrent hardware.
13. Reproduce box.

Code to show. The RNE expression; the subnormal shift computation; the int4 unpack idiom; one `objdump` excerpt. Nothing else.

Post 1b. "Verifying a float converter against every one of the four billion inputs"

Thesis. Some functions have a small enough domain that testing *is* proof; fp32→fp16 is one, and the oracle is a CPU instruction. **Length** 1200–1500 words. **Figure:** wall time of the sweep vs threads (or vs `-O` level) and a table of mismatch counts per rounding mode and per bug you planted on purpose (round-half-up, missing subnormal shift, off-by-one overflow check) to show the sweep catches each. **Outline:** why exhaustive is possible here (2^{32} inputs, ∼1 minute); building the oracle with `_mm256_cvtps_ph`; the planted-bug table; what exhaustive testing cannot tell you (bf16 and fp8 have the same domain, but int quantization does not—property tests return); reproduce box. **What went wrong:** the first sweep "passed" because NaN payloads compared equal under `==` on bits but not on value—state which comparison you want.

2.8 Git lesson and push checklist

Git lesson for this project

Topics: object model and the three trees (Section 1.9); `.gitignore`; SSH keys; first remote; tags; commit messages.

Practice during this project.

1. After the skeleton commit, run `git cat-file -p` on the commit, the tree, and a blob. Edit one file, commit, and observe that only the changed blob, its tree path, and the commit are new objects (`find .git/objects -type f | wc -l` before and after).
2. Commit in small units: one commit for `ck.h`, one per format, one per test file, one for each benchmark. Use `git add -p` at least once to split an over-large edit.
3. When A1.2 closes: `git tag -a v0.1.0 -m "fp16/bf16 complete"` and `git push --follow-tags`. Annotated tags are objects with their own metadata; lightweight tags are just refs—use annotated for releases.
4. Write `.gitignore` *before* the first build so that `build/` never enters history. If a generated file slips in: `git rm --cached <file>`, commit, and note that it still exists in older commits.
5. Keep `main` always building. You have no branches yet; project 2 introduces them.

Push checklist (every project): `make fmt` → `make BUILD=asan test` → `make test` → `git status` shows only intended files → commit → `git push`. Update the README's headline number when it changes.

Chapter 3

Project 2: `cforward` — an optimized single-core transformer forward pass

3.1 Goal

A plain-C decoder-only transformer forward pass (Llama architecture: RMSNorm, RoPE, grouped-query attention, SwiGLU) that loads real weights, matches the reference implementation's logits, and is then optimized step by step—loop order, cache blocking, SIMD intrinsics, multiple accumulators, quantized weights from project 1—with every step placed on a roofline you measured yourself. The project starts with CS:APP's *memory mountain* so that the roofline's two ceilings are numbers from your machine, not from a datasheet.

Research hook

Prefill is compute-bound and decode is bandwidth-bound; every serving paper you read or write assumes this. Here you derive it, measure it on a CPU, and see what quantized weights do to the decode ceiling—the same analysis you will do for GPUs, Tenstorrent cores, and bandwidth-rich but FLOPs-poor accelerators.

3.2 CS:APP reading map

Section	What you build with it
§ 5.1–5.6 optimization blockers, CPE	measuring each kernel in cycles per element
§ 5.7 modern processor operation, latency vs throughput	the multiple-accumulator argument
§ 5.8–5.11 loop unrolling, reassociation, SIMD	the matvec/GEMM ladder
§ 5.12–5.14 memory performance, limiting factors	why decode saturates DRAM, not the FMA units
§ 6.2–6.4 memory hierarchy, caches	blocking and the miss-rate analysis
§ 6.5–6.7 writing cache-friendly code, the memory mountain	assignment A2.0

3.3 Mathematical background

3.3.1 The model

Notation: model dimension d, layers L, query heads h, key/value heads h_{kv} (GQA; $h_{kv} = h$ is multi-head attention), head dimension $d_h = d/h$, MLP hidden size F, vocabulary V, sequence position t (0-based), RoPE base θ_0 (10000 for Llama 2, 500000 for Llama 3).

Per layer ℓ the parameters are $W_q \in \mathbb{R}^{hd_h \times d}$, $W_k, W_v \in \mathbb{R}^{h_{kv}d_h \times d}$, $W_o \in \mathbb{R}^{d \times hd_h}$, $W_1, W_3 \in \mathbb{R}^{F \times d}$, $W_2 \in \mathbb{R}^{d \times F}$ and two RMSNorm gains $g^{\text{attn}}, g^{\text{ffn}} \in \mathbb{R}^d$. Globally: embedding $E \in \mathbb{R}^{V \times d}$, final norm g^{out}, output projection $W_{\text{out}} \in \mathbb{R}^{V \times d}$ (often tied to E).

RMSNorm.

$$\text{RMSNorm}(x; g)_i = g_i \frac{x_i}{\sqrt{\frac{1}{d}\sum_{j=1}^{d} x_j^2 + \epsilon}}, \qquad \epsilon = 10^{-5} \text{ or } 10^{-6} \text{ (from the config).} \tag{3.1}$$

Rotary position embedding. For each head, pair up dimensions $(2i, 2i+1)$, $i = 0, \ldots, d_h/2 - 1$, with frequency $\omega_i = \theta_0^{-2i/d_h}$. At position t the pair is rotated by angle $t\omega_i$:

$$\begin{pmatrix} q'_{2i} \\ q'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos t\omega_i & -\sin t\omega_i \\ \sin t\omega_i & \cos t\omega_i \end{pmatrix} \begin{pmatrix} q_{2i} \\ q_{2i+1} \end{pmatrix}. \tag{3.2}$$

Applied to q and k (not v), this makes $q_t \cdot k_u$ depend only on $t - u$. Precompute $\cos t\omega_i$, $\sin t\omega_i$ for all $t < T_{\max}$ once (an $T_{\max} \times d_h/2$ table each). Two pairing conventions exist—adjacent pairs $(2i, 2i+1)$ as above (`llama2.c`, GGML) and half-split pairs $(i, i + d_h/2)$ (Hugging Face)—and weights exported from one convention must be permuted for the other; this is the single most common source of "logits almost match".

Attention (one new token at position t, head j, kv head $j' = \lfloor j/(h/h_{kv}) \rfloor$).

$$s_u = \frac{q_j \cdot K_u^{(j')}}{\sqrt{d_h}} \quad (u = 0..t), \qquad a = \text{softmax}(s), \qquad o_j = \sum_{u=0}^{t} a_u V_u^{(j')}. \tag{3.3}$$

During prefill the same is done for all positions at once with the causal mask $s_{tu} = -\infty$ for $u > t$.

Numerically stable softmax. With $m = \max_u s_u$: $a_u = e^{s_u - m} / \sum_v e^{s_v - m}$. Subtracting m changes nothing mathematically and prevents overflow of e^s in fp32 for $s > 88.7$.

Online softmax (one pass, no stored scores; the core of FlashAttention and of the blocked attention kernel in A2.4). Maintain (m, l, o) initialized to $(-\infty, 0, \mathbf{0})$; for each new (s_u, V_u):

$$m' = \max(m, s_u), \qquad l' = l\, e^{m - m'} + e^{s_u - m'}, \qquad o' = o\, e^{m - m'} + e^{s_u - m'} V_u, \tag{3.4}$$

and output o/l at the end. Processing a block of B keys at a time uses the same update with the block's local max and partial sums. Prove to yourself (and test) that it equals the two-pass version exactly in real arithmetic and to within a few ULPs in fp32.

MLP (SwiGLU).

$$y = W_2\big(\text{SiLU}(W_1\tilde{x}) \odot W_3\tilde{x}\big), \qquad \text{SiLU}(z) = \frac{z}{1 + e^{-z}}. \tag{3.5}$$

One layer. $\tilde{x} = \mathrm{RMSNorm}(x; g^{\mathrm{attn}})$; $x \leftarrow x + W_o\,\mathrm{Attn}(\tilde{x})$; $\tilde{x} = \mathrm{RMSNorm}(x; g^{\mathrm{ffn}})$; $x \leftarrow x + \mathrm{MLP}(\tilde{x})$. After L layers: $\mathrm{logits} = W_{\mathrm{out}}\,\mathrm{RMSNorm}(x; g^{\mathrm{out}})$.

Sampling. Greedy: $\arg\max$. Temperature τ: $\mathrm{softmax}(\mathrm{logits}/\tau)$. Top-$p$: sort descending, keep the smallest prefix with cumulative probability $\geq p$, renormalize, sample. Greedy decoding is what the tests use (deterministic).

3.3.2 Cost model

Parameter count (ignoring norms): $P \approx L\big[d \cdot hd_h + 2\,d \cdot h_{kv}d_h + hd_h \cdot d + 3dF\big] + 2Vd$. One decode step performs, per layer, a matrix-vector product with each weight matrix (2 FLOPs per weight) plus attention over t cached positions ($\approx 4t\,hd_h$ FLOPs):

$$\mathrm{FLOPs}_{\mathrm{decode}}(t) \approx 2P + 4Lthd_h, \qquad \mathrm{Bytes}_{\mathrm{decode}}(t) \approx P\,b_w + 2L\,t\,h_{kv}d_h\,b_{kv}, \tag{3.6}$$

with b_w, b_{kv} bytes per weight and per KV element. Prefill of T tokens: FLOPs $\approx 2PT + 2LT^2hd_h$, bytes $\approx Pb_w + \mathrm{KV}(T)$.

Arithmetic intensity is FLOPs per byte moved from DRAM. For a matrix–vector product $y = Wx$ with $W \in \mathbb{R}^{m\times n}$: $2mn$ FLOPs over $mn\,b_w$ bytes $\Rightarrow$ $\mathrm{AI} = 2/b_w$ FLOP/byte: 0.5 for fp32, 1 for bf16, 2 for int8, 4 for int4. For prefill with T tokens the weights are reused T times: $\mathrm{AI} \approx 2T/b_w$.

Roofline. With measured peak compute F_{peak} (FLOP/s) and DRAM bandwidth β (B/s), attainable performance is

$$F_{\mathrm{att}}(\mathrm{AI}) = \min\big(F_{\mathrm{peak}},\ \beta \cdot \mathrm{AI}\big), \qquad \text{ridge point } \ \mathrm{AI}^* = F_{\mathrm{peak}}/\beta. \tag{3.7}$$

A CPU with $F_{\mathrm{peak}} = 1$ TFLOP/s and $\beta = 50$ GB/s has $\mathrm{AI}^* = 20$; decode at $\mathrm{AI} \leq 4$ is therefore bandwidth-bound by a factor of five or more even with int4 weights, and the token rate is bounded by

$$\mathrm{tokens/s} \leq \frac{\beta}{P\,b_w + \mathrm{KV}(t)}. \tag{3.8}$$

Prefill crosses the ridge when $T > \mathrm{AI}^*\,b_w/2$—about 40 tokens for fp32 on that machine. Plot the ridge, then plot every kernel version as a point.

Peak compute. $F_{\mathrm{peak}} = \mathrm{cores} \times f_{\mathrm{clock}} \times \mathrm{FMA\ units} \times 2 \times \mathrm{lanes}$, with 8 fp32 lanes for AVX2 and 16 for AVX-512; a typical desktop core with two 256-bit FMA units at 4 GHz gives 128 GFLOP/s per core. Measure it instead (A2.0): the sustained all-core AVX frequency is lower than the sticker.

3.3.3 Why one accumulator is slow (§ 5.7)

A reduction $acc \mathrel{+}= a_i b_i$ forms a dependency chain through acc: each FMA must wait for the previous one's result, so throughput is one FMA per latency λ (4 cycles on recent Intel and AMD cores), giving $\mathrm{CPE} = \lambda/\mathrm{lanes}$ no matter how many FMA units exist. With k independent accumulators the chains interleave:

$$\mathrm{CPE} = \max\left(\frac{\lambda}{k \cdot \mathrm{lanes}},\ \frac{1}{\mathrm{FMA\ units} \cdot \mathrm{lanes}}\right), \tag{3.9}$$

so saturating two FMA units with latency 4 needs $k \geq 8$ accumulators (64 partial sums in fp32 AVX2). Reassociating a floating-point sum this way changes the result in the last bits; the compiler is not allowed to do it without `-ffast-math`, which is why you do it by hand—and why bit-identical results across builds are not to be expected. Keep an `-O2`-without-fast-math reference build for the tests.

3.3.4 Cache blocking for GEMM (§ 6.6)

For $C \mathrel{+}= AB$ with $n \times n$ fp32 matrices, line size 64 bytes ($w = 16$ floats), the ijk loop reads a column of B with stride n—one miss per element—so misses $\approx n^3 + n^3/w$. The ikj order makes the inner loop contiguous in both B and C: misses $\approx 2n^3/w$. Blocking with $b \times b$ tiles keeps three tiles resident: $3b^2 \cdot 4 \le C_{L1} \Rightarrow b \le \sqrt{C_{L1}/12}$, about 52 for a 32 KB L1d, so $b \in \{32, 48\}$. Each $(n/b)^3$ tile multiplication then costs $\approx 2b^2/w$ misses, total $\approx \frac{2n^3}{w\,b}$: blocking buys a factor of b over ikj. Add a second level of blocking for L2 ($b_2 \approx \sqrt{C_{L2}/12}$) and you have the structure of every BLAS. For decode you only need matrix–*vector* products, where blocking cannot help (every weight is touched once) and the only levers are bytes per weight and bandwidth utilization; that contrast is the lesson.

3.3.5 The memory mountain (§ 6.6.1)

Read throughput as a function of working-set size S and stride σ (in 8-byte words): for each (S, σ) sum an array of S bytes with the given stride, enough repetitions to take ≥ 100 ms, report bytes/s. The surface exposes L1, L2, L3 and DRAM plateaus along the size axis and the hardware prefetcher's reach along the stride axis. Add a pointer-chasing latency test (a random cyclic permutation; dependent loads) to get cycles per access at each level. These numbers go into `docs/machine.md` and the roofline.

3.4 Architecture

3.4.1 Directory layout

```
cforward/
  include/cforward/
    model.h       config, weights (memory-mapped), tokenizer handle
    kernels.h     rmsnorm, rope, matvec/gemm (several variants), attention, silu-gate,
    softmax
    forward.h     cf_forward_token / cf_forward_prefill / sampling
    tokenizer.h   llama2.c-style sentencepiece BPE (encode/decode)
    roofline.h    cycle counters (rdtsc / clock_gettime), perf-event wrappers
  src/            one .c per header; kernels/ subdir with matvec_naive.c matvec_blocked
    .c matvec_avx2.c ...
  tests/          test_rmsnorm.c test_rope.c test_softmax.c test_matvec.c
    test_attention.c test_forward.c
  bench/          mountain.c peak_flops.c peak_bw.c bench_matvec.c bench_decode.c
  scripts/        export_safetensors.py (HF -> safetensors + config.json),
    golden_logits.py, plot_roofline.py, plot_mountain.py
  data/           (ignored) stories15M.safetensors tokenizer.bin golden/*.bin
```

3.4.2 Public API

```c
/* model.h */
typedef struct {
  int dim, n_layers, n_heads, n_kv_heads, hidden_dim, vocab_size, max_seq_len;
  float norm_eps, rope_theta;
  enum { CF_W_F32, CF_W_BF16, CF_W_INT8, CF_W_INT4 } wtype;    /* via qnum */
  int group;                                                    /* int quant group */
} cf_config;
```

```
typedef struct cf_model cf_model;              /* opaque: mmap'd weights + per-layer
    pointers */
int  cf_model_open(const char *safetensors_path, const char *config_json, cf_model **
    out);
void cf_model_close(cf_model *m);
const cf_config *cf_model_config(const cf_model *m);

/* kernels.h -- every kernel has a reference and >=1 optimized variant with identical
    signature */
typedef void (*cf_matvec_fn)(float *y, const void *W, const float *x, int m, int n, int
     wtype, int group);
extern const struct { const char *name; cf_matvec_fn fn; } cf_matvec_variants[];
void cf_rmsnorm(float *y, const float *x, const float *g, int d, float eps);
void cf_rope(float *q, float *k, int pos, int n_heads, int n_kv_heads, int head_dim,
             const float *cos_tab, const float *sin_tab);
void cf_softmax(float *x, int n);
void cf_attention(float *out, const float *q, const float *K, const float *V,
                  int pos, int n_heads, int n_kv_heads, int head_dim, int kv_stride);
void cf_silu_gate(float *h1, const float *h3, int n);          /* h1 = silu(h1) * h3 */

/* forward.h */
typedef struct cf_state cf_state;              /* activations + KV cache [L][T][h_kv*
    d_h] (contiguous in P2) */
cf_state *cf_state_new(const cf_model *m);
void      cf_state_free(cf_state *s);
float    *cf_forward(cf_model *m, cf_state *s, int token, int pos);     /* returns
    logits[V] */
void      cf_prefill(cf_model *m, cf_state *s, const int *tokens, int n, float *
    last_logits);
int       cf_sample_greedy(const float *logits, int V);
int       cf_sample_topp(const float *logits, int V, float temp, float top_p, uint64_t
    *rng);

/* roofline.h */
uint64_t cf_cycles(void);                       /* rdtsc; calibrate against
    clock_gettime */
typedef struct { long long cycles, instructions, l1d_miss, llc_miss, dtlb_miss; }
    cf_counters;
int  cf_perf_open(cf_counters *c);  void cf_perf_read(cf_counters *c);   /*
    perf_event_open */
```

3.4.3 Weight layout

Store every matrix row-major as $W[m][n]$ so that $y_i = \sum_j W_{ij} x_j$ reads row i contiguously (one dot product per output; this is what SIMD and the multiple-accumulator trick want). Quantized weights (project 1) are stored row-major in groups along n so that a row's scales are contiguous too. Memory-map the safetensors file read-only with `MAP_PRIVATE`: no copy, instant startup, and the OS page cache does the rest (chapter 9 preview; project 4 makes it explicit).

3.4.4 The optimization ladder for `matvec`

Each rung is a separate source file and a separate entry in `cf_matvec_variants`, so that they can be benchmarked and tested against each other:

R1. `naive`: the textbook double loop, `-O2`. Record CPE.
R2. `unroll4`: inner loop unrolled 4× with one accumulator; shows the dependency-chain limit.
R3. `acc8`: 8 scalar accumulators; CPE should drop by nearly λ.
R4. `avx2`: `_mm256_fmadd_ps` with 1 vector accumulator; then `avx2_acc4`/`acc8`.
R5. `avx2_bf16`: load bf16 weights, widen with a shift (`_mm256_slli_epi32` after `_mm256_cvtepu16_epi32`), FMA. Halves bytes, roughly doubles tokens/s if bandwidth-bound.
R6. `avx2_int8`, `avx2_int4`: dequantize on the fly (`cvtepi8_epi32` → `cvtepi32_ps` → multiply by group scale), or better, accumulate integer dot products per group with `_mm256_madd_epi16` and apply the scale once per group.
R7. `omp`: rows split across threads with OpenMP; confirms you hit β with all cores.

For prefill, `gemm` variants: `ijk`, `ikj`, `blocked`, `blocked_avx2` (micro-kernel computing a 6×16 tile of C in registers: 12 ymm accumulators, broadcast A, load B).

3.5 Assignments

A2.0 Memory mountain, peak FLOP/s, peak bandwidth, roofline *8 h*

Goal. Your machine's two ceilings and its cache plateaus, measured, in `docs/machine.md`.
Steps. (1) `roofline.h`: `rdtsc` calibrated against `clock_gettime`; a `perf_event_open` wrapper for cycles and instructions. (2) `bench/mountain.c`: sizes 16 KB to 256 MB (powers of two with midpoints), strides 1–32 words, reading with 4 independent accumulators (so the measurement is not latency-bound), repetitions scaled to $\geq$ 100 ms; CSV output. (3) `bench/latency.c`: pointer chasing over a random cyclic permutation; cycles per dependent load vs size. (4) `bench/peak_flops.c`: a loop of `_mm256_fmadd_ps` with 8, 12, 16 independent accumulators, single core and all cores (OpenMP); report GFLOP/s and the implied sustained frequency. (5) `bench/peak_bw.c`: read, write, and copy bandwidth for a 1 GB buffer, single core and all cores. (6) `scripts/plot_mountain.py` and `plot_roofline.py` (empty roofline: ceilings and ridge point only).
Acceptance. Mountain plot shows distinguishable L1/L2/L3/DRAM plateaus whose sizes agree with `lscpu` within a factor of two; latency plot shows the same transitions; measured peak FLOP/s is within 30% of the datasheet formula; `machine.md` lists all numbers with the commands that produced them.
Feeds. Post 2 Fig. 1 (mountain) and the ceilings of Fig. 2.

A2.1 Reference forward pass *14 h*

Goal. A naive fp32 forward pass whose logits match the Python reference.
Steps. (1) `scripts/export_safetensors.py`: `stories15M` and `stories110M` to safetensors plus `config.json`, with the RoPE pairing convention stated in the config. (2) `model.h`: `mmap` the file, parse the header with the project-1 reader (copy or submodule), set up per-layer pointers. (3) Tokenizer (`tokenizer.bin` from `llama2.c`; encode/decode). (4) Kernels, naive versions only: `rmsnorm`, `rope` (both conventions behind a flag), `matvec_naive`, `softmax`, `attention` (two-pass), `silu_gate`. (5) `cf_forward` for one token; `cf_prefill` by looping tokens (slow, fine). (6) Golden tests: per-kernel against NumPy; 200 greedy tokens against the reference; logits at the last position

for 5 prompts.
Acceptance. Every golden test passes at the stated tolerances; a 200-token greedy generation from `stories110M` is identical to the reference; valgrind memcheck is clean on a 10-token run; ASan clean.
Feeds. Post 2 setup; Post 2b's reference implementation.

A2.2 The matvec ladder, R1–R4 *10 h*

Goal. Four variants of `matvec` with a CPE table and roofline points.
Steps. (1) `bench/bench_matvec.c`: for a given variant and $m \times n$, time 50 repetitions after warm-up, report CPE (cycles per weight element) and GFLOP/s and GB/s; sizes matching `stories110M`'s matrices and one 4096×4096. (2) Implement R1 (`naive`, measure), R2 (`unroll4`, measure—observe no gain), R3 (`acc8`, measure), R4 (`avx2`, `avx2_acc4`, `avx2_acc8`). (3) The accumulator sweep: a variant parameterized by $k = 1..16$ accumulators (macro-generated), for Fig. 3. (4) Add each variant's point to the roofline script. (5) Equivalence tests for all variants.
Acceptance. CPE table in the README with all variants; `acc8` $< 0.6\times$ naive, `avx2_acc8` $< 0.25\times$ naive; the k-sweep shows a knee at $k \approx \lambda\times$ FMA units; for the 4096^2 fp32 matrix, `avx2_acc8` sits within 20% of the bandwidth ceiling on the roofline (if it does not, you have found a problem worth a log entry).
Feeds. Post 2 Fig. 2 points, Fig. 3, the mechanism section on dependency chains.

A2.3 Quantized weights via `qnum` *10 h*

Goal. bf16, int8 and int4 weight paths and the tokens/s-vs-bytes figure.
Steps. (1) Extend the exporter to emit bf16/int8/int4 weights (using `qnum` from Python via a small `ctypes` binding, or re-quantize in C at load time with `qn_intq_quantize`—the latter is simpler). (2) R5 `avx2_bf16`; R6 `avx2_int8` with per-group integer dot product (`_mm256_maddubs_epi16`/`madd_epi16`) and scale applied per group; R6 `avx2_int4` with nibble unpacking. (3) `bench/bench_decode.c`: tokens/s for a 256-token generation per weight type, single core and all cores; also the bound $\beta/(Pb_w)$ from `machine.md`. (4) Equivalence tests against naive-on-dequantized-weights. (5) Accuracy check: greedy outputs under int4 vs fp32 for 10 prompts (expect divergence after some tokens; report where).
Acceptance. All equivalence tests pass; tokens/s roughly doubles from fp32 to bf16 and again to int8 on the large matrices (if it does not, the kernel is not bandwidth-bound yet—profile with `perf` before closing); figure of tokens/s vs bytes/weight with the bound drawn.
Feeds. Post 2 central figure (Fig. 4) and the "what this predicts" section.

A2.4 Attention and prefill *12 h*

Goal. Online-softmax blocked attention, a blocked GEMM for prefill, and the prefill-vs-decode curves.
Steps. (1) Online-softmax attention processing keys in blocks of 32 with the (m, l, o) update; ULP test against two-pass. (2) Fuse RoPE into the Q/K projection output. (3) GEMM ladder for prefill: `ijk`, `ikj`, `blocked` ($b \in \{16, 32, 48, 64\}$), `blocked_avx2` with the 6×16 micro-kernel. (4) `cf_prefill` using GEMM; causality test. (5) `bench/bench_prefill.c`: throughput (tokens/s) vs $T \in \{1, 2, 4, \ldots, 512\}$; mark the ridge point. (6) `valgrind --tool=cachegrind` on one blocked vs unblocked run to compare simulated L1 miss rates with the arithmetic in the math section.
Acceptance. Online-softmax ULP test passes; causality test passes; blocked GEMM reaches $\geq 50\%$ of peak FLOP/s on 1024^2 (document the number if lower and why); prefill throughput

crosses the decode throughput at a T within a factor of 2 of the predicted $\mathrm{AI}^* \, b_w/2$.
Feeds. Post 2 Fig. 5 and Fig. 6; all of Post 2b.

A2.5 Hardware counters *6 h*

Goal. Confirm the bandwidth-bound diagnosis with counters, not inference from timing.
Steps. (1) Wire `cf_perf_open` into `bench_matvec` and `bench_prefill`: cycles, instructions, L1D misses, LLC misses, dTLB misses. (2) IPC and bytes/cycle per kernel variant. (3) L1 miss rate vs block size b for the GEMM. (4) Write the counter table into `docs/perf.md` with the `perf stat` invocations that reproduce them.
Acceptance. For the decode matvec, measured DRAM bytes/s (from LLC misses $\times$ 64 B) is within 25% of the timing-derived GB/s; the miss-rate-vs-b curve has its minimum near the predicted $b \le \sqrt{C_{L1}/12}$.
Feeds. Post 2 results section (the counter table that closes the argument).

3.6 Test plan

Kernel unit tests vs NumPy goldens
For `rmsnorm`, `rope` (both pairing conventions), `softmax` (including inputs with $|s| > 100$ and all-equal inputs), `silu_gate`: random inputs, outputs within 10^{-5} relative.

Variant equivalence
Every `matvec`/`gemm` variant vs `naive` on random W, x with $m, n \in \{1, 7, 8, 9, 64, 65, 1024\}$; tolerance 10^{-4} relative to $\|y\|$ (different association). For quantized variants, compare against `naive` applied to the *dequantized* weights with tolerance 10^{-6} (same numbers, different order only).

Online = offline softmax
Attention with online softmax vs two-pass on random K, V for $t \in \{1, 2, 31, 32, 33, 1000\}$: max ULP distance ≤ 8.

Causality
Prefill logits at position t equal single-token decode logits after feeding tokens $0..t$ one by one.

End-to-end golden
200 greedy tokens equal the Python reference (`scripts/golden_logits.py` runs `llama2.c`'s `model.py` or HF); store logits of the last position for 5 prompts and compare with relative 10^{-4}.

Shape edge cases
$h_{kv} < h$ (GQA) with a synthetic random-weight config; d_h not a multiple of 16.

Performance regression
`bench_matvec` prints CPE per variant; a test asserts `acc8` CPE $< 0.6\times$ `naive` CPE and `avx2_acc8` $< 0.25\times$ (soft thresholds, skipped under sanitizers).

Sanitizers
ASan/UBSan on the whole test suite; `valgrind --tool=cachegrind` on `bench_matvec` once, to compare simulated vs measured miss rates.

3.7 Measurements and blog briefs

(i) Memory mountain surface; (ii) roofline with one point per matvec variant and per weight type; (iii) tokens/s vs bytes/weight with the $\beta/(Pb_w)$ bound drawn; (iv) CPE vs number of accumulators

$k = 1..16$; (v) prefill throughput vs T crossing the ridge; (vi) L1 miss rate vs block size b; (vii) the hardware-counter table.

Blog plan

Post 2. "A roofline for LLM decode, measured from scratch on one CPU core"
Audience and thesis. For people who have heard "decode is memory-bound" and want to see the ceiling move. Thesis: decode is a bandwidth problem; you can watch the token rate track bytes-per-weight and stop exactly where the roofline says, and no FLOP optimization moves it. Subtitle: "X tokens/s fp32 $\to$ Y int4 on one core, against a $\beta/(Pb_w)$ bound of Z".
Length. 2800–3300 words.
Figures.

1. **Figure 1.** The memory mountain: read throughput (GB/s, z) over working-set size ($\log x$) and stride (y), as a surface or as a family of curves; plateaus labelled with the cache they correspond to.
2. **Figure 2.** (central) Roofline: GFLOP/s ($\log y$) vs arithmetic intensity ($\log x$); ceilings from A2.0; one point per matvec variant for the 4096^2 matrix (fp32 ladder marching up the bandwidth slope) and per weight type (points moving right along the slope); the GEMM points up near the compute ceiling for contrast.
3. **Figure 3.** CPE vs number of accumulators $k = 1..16$, scalar and AVX2, with the predicted $\max(\lambda/k, 1/\text{units})$ curve dashed.
4. **Figure 4.** Decode tokens/s (y) vs bytes per weight (x: 4, 2, 1, 0.5) for `stories110M`, with the bound $\beta/(Pb_w)$ as a dashed line; single-core and all-core series.
5. **Figure 5.** Prefill throughput vs T ($\log x$), with the ridge-point T marked and decode throughput as a horizontal reference.
6. **Figure 6.** L1 miss rate vs block size b from counters and from cachegrind, with the $\sqrt{C_{L1}/12}$ line.

Outline.

1. Lede: three numbers (fp32, int4 tokens/s, the bound) and the claim.
2. Fig. 2, read aloud: what each point is and why they stop.
3. Setup: machine, model, flags, runs, tag.
4. The mountain (Fig. 1) and what each plateau costs; the latency numbers from pointer chasing.
5. The cost model: FLOPs and bytes per decode step, arithmetic intensity $2/b_w$, the ridge point with your machine's numbers; a worked example for `stories110M` giving the predicted tokens/s before any measurement.
6. Why one accumulator is slow: the dependency-chain argument, Fig. 3, and the code excerpt showing eight accumulators. Say explicitly that this changes the floating-point result and why the compiler will not do it for you.
7. Blocking for prefill: the miss-count arithmetic, Fig. 6, and the 6×16 micro-kernel's register budget.
8. Quantized matvec: the int8 integer-dot-product trick in 12 lines; Fig. 4 and the sentence "the bound was predicted in section 5 before the kernel existed".
9. Prefill vs decode: Fig. 5 and the crossing.
10. The counter table: bytes/s from LLC misses vs from timing; IPC per variant. This is what makes the argument more than a curve fit.
11. What went wrong: the RoPE pairing convention; the first "bandwidth-bound" kernel that was actually latency-bound (how the k sweep revealed it); turbo frequency drift between

runs.

12. What this predicts: on an H100 with 3.3 TB/s and 8B params at 2 bytes, the same arithmetic gives ∼200 tokens/s per sequence—and why batching (project 6) is the only way past it; on a FLOPs-poor accelerator the ridge moves left and prefill becomes the problem instead. One falsifiable prediction.
13. Reproduce box: `make bench`, the three scripts, 20 minutes.

Code to show. The 8-accumulator AVX2 dot product; the online-softmax update is saved for 2b; the int8 group dot product; the `perf_event_open` attribute setup (10 lines) because nobody shows it.

Post 2b. "Online softmax, explained with one invariant"

Thesis. The whole of FlashAttention's numerics is one invariant: after processing keys $0..u$, (m, l, o) equals (max, sum of e^{s-m}, weighted sum of V with weights e^{s-m}). **Length** 1200–1600 words. **Figures:** a diagram of the (m, l, o) state being rescaled when a new maximum arrives; ULP-distance histogram between online and two-pass results for t up to 4096. **Outline:** the stability problem ($e^{88.7}$ overflows); the two-pass version; the invariant and its proof by induction (write the rescaling step out); blocked processing as the same update with block-local statistics; the C implementation in 20 lines; the ULP histogram and why the differences are a few ULPs and not zero; what changes on a GPU (the block is a tile and the rescale is the expensive part). **What went wrong:** initializing m to 0 instead of $-\infty$ gives correct results on most inputs and wrong ones when all scores are negative—state how the test caught it.

3.8 Git lesson and push checklist

Git lesson for this project

Topics: branches as pointers; fast-forward vs three-way merge; rebase and interactive rebase; `--force-with-lease`; stash; conflict resolution; manual `git bisect`.

Practice.

1. Each rung of the ladder is a branch: `git switch -c kern/avx2-acc8`. Commit freely, then `git rebase -i main` to `squash`/`fixup`/`reword` into 1–3 clean commits before merging. Inspect `.git/refs/heads/` to see the branch pointer move.
2. Merge with `git switch main && git merge --ff-only kern/avx2-acc8`. When `--ff-only` refuses (main moved), rebase the branch first: linear history keeps `bisect` simple.
3. Deliberately create a conflict: change the same line of `kernels.h` on two branches; resolve with `git diff`, the conflict markers, `git add`, `git rebase --continue`. Learn `git merge --abort` / `rebase --abort`.
4. Pushing a rebased branch requires `git push --force-with-lease origin kern/avx2-acc8`—never plain `--force`, never on `main`. `--force-with-lease` refuses if the remote moved since your last fetch.
5. `git stash` (`push -m "wip rope"`, `list`, `pop`) to switch branches with a dirty tree.
6. Plant a performance regression on purpose (e.g. reintroduce one accumulator in a middle commit), then find it with `git bisect start; git bisect bad; git bisect good v0.1.0`, running `bench_matvec` at each step and marking `good`/`bad`. Project 5 automates this.
7. Tag `v0.2.0` when A2.4 closes.

Chapter 4

Project 3: `ltrace-ml` — an interposition tracer and sampling profiler

4.1 Goal

A shared library, loaded with `LD_PRELOAD`, that intercepts calls into other libraries—first `malloc`/`free`/`realloc`, then the CUDA runtime (`cudaMalloc`, `cudaMemcpyAsync`, `cudaLaunchKernel`, ...)—logging sizes, timings and call stacks with overhead low enough to run under a real PyTorch or vLLM workload; plus a `SIGPROF`-driven sampling profiler that emits flame-graph input. A small launcher (`fork`/`exec`) runs any program under the tracer. Symbolization is done offline by a Python post-processor from raw addresses and the process's memory map.

Research hook

This becomes the measurement tool for your serving and compiler work: what does PyTorch actually ask the CUDA runtime for, how often, and from where? Which host-side code path burns the time between kernel launches? CUPTI and Nsight answer parts of this; building the poor man's version teaches the linker and signal machinery they stand on, and gives you a tool you can point at a WRF binary just as easily.

4.2 CS:APP reading map

Section	What you build with it
§ 7.4–7.7 ELF, symbols, resolution, relocation	reading `readelf -r`, understanding which calls can be intercepted
§ 7.10–7.12 dynamic linking, PIC, PLT/GOT, lazy binding	how `LD_PRELOAD` wins symbol lookup; `dlsym(RTLD_NEXT)`
§ 7.13 library interpositioning	the three interposition levels; `--wrap` variant
§ 8.2–8.4 exceptions, processes, `fork/execve/waitpid`	the launcher; forked test runner for `ck.h`
§ 8.5 signals, blocking, async-signal safety	the sampling profiler
§ 8.6 nonlocal jumps	(read, not used; know why you avoid `longjmp` here)

4.3 Background

4.3.1 Symbol resolution and why preloading works

At load time the dynamic linker builds a *lookup scope*: the executable, then `LD_PRELOAD` libraries in order, then `DT_NEEDED` libraries breadth-first. An undefined symbol reference in any object is bound to the *first* definition found in that order. A definition of `malloc` in your preloaded library therefore shadows glibc's for every caller that goes through the PLT—including calls from *other* shared libraries (libtorch, libcudart). Calls that cannot be intercepted: (i) calls resolved at static link time (a statically linked `cudart`, which is `nvcc`'s default; check with `ldd` and `/proc/<pid>/maps`); (ii) internal calls within a library that bypass the PLT (glibc's internal `malloc` uses); (iii) symbols with protected visibility.

`dlsym(RTLD_NEXT, "malloc")` continues the search *after* the calling library, returning glibc's definition—your "real" function. The bootstrapping hazard: `dlsym` may itself call `malloc` (it does, in some glibc versions, on first use), re-entering your hook before the real pointer is known. Standard solution: a thread-local reentrancy flag, and a small static bump allocator used only while the flag is set during initialization.

Link-time alternative. `-Wl,--wrap=malloc` makes the linker resolve references to `malloc` to `__wrap_malloc` and references to `__real_malloc` to the original. It only affects objects linked with that flag—so it catches the program's own calls but not libtorch's. Build both variants and compare counts; the difference *is* the lesson about linkage units.

Relocations you will see. `readelf -r` on your tracer shows `R_X86_64_JUMP_SLOT` entries for each PLT target and `R_X86_64_GLOB_DAT` for data references through the GOT; the relocation formulas (§ 7.7) are $S + A$ for these and $S + A - P$ for PC-relative `R_X86_64_PC32` references—the same arithmetic project 5 performs when it emits a RIP-relative load.

4.3.2 Capturing the caller

The cheapest caller identity is the return address: `__builtin_return_address(0)`. Full stacks are walked through frame pointers: with `-fno-omit-frame-pointer`, every frame has `[rbp] = saved caller rbp` and `[rbp+8] = return address`, so the walk is

```
fp = current rbp
while fp != 0 and depth < MAX and fp is readable (within [rsp, stack_top)):
    ra = *(fp + 8); emit ra; fp = *fp
```

Frames from code compiled without frame pointers (most distro libraries) break the chain; accept truncated stacks and document it, or fall back to `libunwind` (`unw_backtrace`) as an optional backend. `backtrace()` from `<execinfo.h>` is *not* usable inside a malloc hook or a signal handler: it may allocate on first use and is not async-signal-safe.

4.3.3 Offline symbolization

Log raw addresses. At exit, dump `/proc/self/maps` alongside the log. With address-space randomization, a code address a inside a mapping with start m and file offset o corresponds to file virtual address $a - m + o$ (plus the segment's `p_vaddr`−`p_offset` adjustment, which `addr2line -e lib.so` and `nm` already expect—verify this on one symbol by hand). The Python post-processor resolves names with `addr2line`/`nm -C` and aggregates. Keeping symbolization off the hot path is what makes per-call tracing cheap.

4.3.4 Sampling profiler

Mechanism. `timer_create(CLOCK_PROCESS_CPUTIME_ID)` with `SIGEV_SIGNAL` and signal `SIGPROF`, or `setitimer(ITIMER_PROF)`, at frequency f. For per-thread attribution use one timer per thread with `SIGEV_THREAD_ID` (`sigev_notify_thread_id`), created from a `pthread_create` interposer. Install the handler with `sigaction`, flags `SA_SIGINFO | SA_RESTART`. The third handler argument is a `ucontext_t*`: `uc_mcontext.gregs[REG_RIP]`, `[REG_RBP]`, `[REG_RSP]` give the interrupted program counter and frame, from which you walk the stack as above.

Async-signal safety. Inside the handler: no `malloc`, no `printf`, no locks, no `errno`-clobbering without save/restore. Write samples to a per-thread preallocated ring buffer with a single-producer index; a flusher thread (or `atexit`) drains it with `write(2)`. The producer–consumer argument (§ 12.5) is simple here because each buffer has exactly one writer (the handler, which cannot interrupt itself if `SIGPROF` is blocked during delivery, which `sigaction` does by default).

Statistics. Each sample is a Bernoulli trial for "in function g" with probability p_g (fraction of CPU time). With $N = fT$ samples, $\hat{p}_g = n_g/N$ has standard error $\sqrt{p_g(1-p_g)/N}$. To estimate a function using 1% of time to ±20% relative at 95% confidence:

$$N \geq \left(\frac{1.96}{0.2}\right)^2 \frac{1 - p_g}{p_g} \approx 9500 \text{ samples} \approx 10 \text{ s at } 1 \text{ kHz}. \tag{4.1}$$

Print confidence intervals in the report; most profilers don't, and the absence is why people chase noise. **Overhead** is $f \cdot c$ for per-sample cost c; at $f = 1$ kHz and $c \approx 2\,\mu$s it is 0.2%. **Aliasing:** a workload with a periodic phase at frequency f_p sampled at a commensurate f sees the same phase every time; use a prime frequency (997 Hz) or jitter each interval by a random ±10%.

Output format. Folded stacks, one line per distinct stack: `main;cf_forward;cf_matvec_avx2 1234` (frames root→leaf separated by `;`, then the count). `flamegraph.pl` or `inferno-flamegraph` renders it.

4.3.5 Process control for the launcher (§ 8.4)

`ltrace-ml run [opts] -- prog args`: `fork()`; in the child, set `LD_PRELOAD` (prepend to any existing value), optionally `LTRACE_OUT`, `LTRACE_HZ`, then `execvp`; in the parent, forward `SIGINT`/`SIGTERM` to the child, `waitpid`, and exit with the child's status (`WEXITSTATUS`, or 128+

signal number if `WIFSIGNALED`). The tracer library flushes in an `atexit` handler and in a `SIGTERM` handler (only async-signal-safe calls).

4.3.6 What CUDA looks like from here

PyTorch's CUDA caching allocator requests large segments from `cudaMalloc` rarely and carves them itself, so a trace of a training step shows a burst of `cudaMalloc` at warm-up and then almost none—your tool makes this visible in one run. `cudaMemcpyAsync` sizes and directions (`cudaMemcpyKind`) expose the input pipeline; `cudaLaunchKernel` gives the launch rate and host-side gaps between launches (the symptom of CPU-bound serving). Kernel names are not available from the runtime API without CUPTI, but the function pointer argument can be symbolized offline against the fatbinary's host stubs—often enough to identify the operator. Interpose the *driver* API (`cuLaunchKernel`) as well if the runtime is statically linked but `libcuda.so` is not (it never is).

4.4 Architecture

4.4.1 Directory layout

```
ltrace-ml/
  include/ltrace/
    hooks.h        interposer registration macros; LT_HOOK(name, ret, args...)
    log.h          per-thread ring buffer, event records, flush
    stack.h        frame-pointer walker, optional libunwind backend
    prof.h         sampling profiler: start/stop, per-thread timers
    maps.h         /proc/self/maps snapshot
  src/
    hooks_malloc.c hooks_pthread.c hooks_cuda.c (dlopen-free: plain symbol definitions)
    log.c stack.c prof.c maps.c init.c (constructor: read env, set up buffers)
    launcher/main.c (ltrace-ml run ... ; ltrace-ml report ...)
  tools/
    symbolize.py   raw log + maps -> symbolized events / folded stacks / summary tables
  tests/
    targets/       small programs with known behaviour (alloc_counter.c, cpu_shares.c,
    threads.c)
    test_hooks.c test_stack.c test_prof.c test_launcher.c
  bench/           overhead.c (traced vs untraced malloc-heavy loop; profiler overhead
    vs Hz)
```

4.4.2 Public API and formats

```c
/* log.h -- fixed-size binary records, little-endian, written with write(2) */
typedef struct {
  uint64_t ts_ns;        /* CLOCK_MONOTONIC */
  uint32_t tid;
  uint16_t kind;         /* LT_MALLOC, LT_FREE, LT_REALLOC, LT_CUDA_MALLOC,
    LT_CUDA_MEMCPY, LT_LAUNCH, LT_SAMPLE ... */
  uint16_t nframes;      /* number of return addresses following the record */
  uint64_t a0, a1, a2;   /* kind-specific: size, ptr, direction, device ptr, ... */
  /* followed by nframes * uint64_t */
} lt_event;
```

```c
typedef struct lt_ring lt_ring;                 /* SPSC ring, power-of-two size, per
    thread */
lt_ring *lt_ring_get(void);                     /* thread-local, lazily created with the
     bump allocator */
int  lt_ring_push(lt_ring *r, const lt_event *e, const uint64_t *frames);   /* async-
    signal-safe */
void lt_flush_all(void);                        /* called at exit / SIGTERM */

/* stack.h */
int lt_walk_fp(void *rbp, void *rsp, uint64_t *out, int max);          /* returns
    frames written */

/* prof.h */
int  lt_prof_start(unsigned hz, bool per_thread);
void lt_prof_stop(void);

/* hooks.h -- example of the pattern each hook follows */
#define LT_REAL(name) (lt_real_##name ? lt_real_##name : lt_resolve_##name())
void *malloc(size_t n);  /* implemented as: guard; t0; p = LT_REAL(malloc)(n); log;
    return p */
```

Environment variables:

```
LTRACE_OUT=<dir>         default ./ltrace.<pid>/
LTRACE_HOOKS=malloc,cuda,pthread
LTRACE_HZ=997            profiler frequency; 0 disables
LTRACE_STACKS=0|fp|unwind
LTRACE_MAXDEPTH=32
LTRACE_RING_MB=16        per-thread ring size
```

Output directory: `events.bin` (records), `maps.txt` (`/proc/self/maps` at exit), `meta.json` (hz, start time, argv). `tools/symbolize.py` produces `events.tsv`, `alloc_summary.md` (counts/bytes by call site, live-at-exit), `cuda_summary.md`, and `profile.folded`.

4.4.3 Hook pattern (pseudocode)

```
thread_local int in_hook = 0

malloc(n):
    if in_hook or !initialized:  return bump_alloc(n)      # bootstrap / reentrancy
    in_hook = 1
    real = LT_REAL(malloc)                                   # dlsym(RTLD_NEXT) once,
    cached
    t0 = now(); p = real(n); dt = now() - t0
    frames = walk_fp(__builtin_frame_address(0), ...) if stacks enabled
    ring_push({ts=t0, kind=MALLOC, a0=n, a1=p, a2=dt}, frames)
    in_hook = 0
    return p

free(p):
    if p is inside the bump arena: return                    # never hand bump memory
    to glibc
    ... same shape ...
```

4.5 Assignments

A3.1 Allocation hooks and the launcher *10 h*

Goal. `ltrace-ml run -- prog` produces a by-call-site allocation table for any dynamically linked program.
Steps. (1) `init.c`: a constructor that reads the environment, allocates per-thread rings from a static bump arena, opens the output directory, registers `atexit`. (2) `hooks_malloc.c` following the hook pattern: `malloc`, `free`, `calloc`, `realloc`, `posix_memalign`, `aligned_alloc`; return address only. (3) `log.c`: the SPSC ring and `lt_flush_all` using only `write(2)`. (4) `maps.c`: copy `/proc/self/maps` at exit. (5) The launcher: `fork/execvp`, `LD_PRELOAD` prepending, status propagation, signal forwarding. (6) `tools/symbolize.py`: parse records, resolve return addresses via `addr2line` per mapping, aggregate count/bytes/live by site. (7) Trace `cforward` decoding 100 tokens and read the table.
Acceptance. `alloc_counter` test exact; reentrancy test passes; launcher status tests pass; the `cforward` table identifies the tokenizer's per-token allocations (and you fix them in `cforward`—note the commit).
Feeds. Post 3 sections 4–6 (lookup scope, bootstrapping, offline symbolization).

A3.2 Stack walking and the two interposition levels *8 h*

Goal. Call stacks on every event; a measured comparison of `LD_PRELOAD` vs `--wrap`.
Steps. (1) `stack.c`: frame-pointer walk bounded by the thread's stack limits (`pthread_getattr_np`); depth limit. (2) Folded-stack output from `symbolize.py`; render one flame graph of `cforward`'s allocations. (3) `--wrap` build variant of the same hooks (`__wrap_malloc`/`__real_malloc`); the four-target interposition matrix test. (4) `readelf -r` walk-through in `docs/linking-notes.md`: find the `JUMP_SLOT` for `malloc` in `cforward`, in `libc`, in your tracer.
Acceptance. Stack test passes (known chain resolved in order); truncated chains do not crash; the matrix test encodes which of the four cases are intercepted and passes; the notes file explains each row by the lookup-scope rule.
Feeds. Post 3 the `readelf -r` figure and the "what cannot be intercepted" section.

A3.3 Sampling profiler, single-threaded *10 h*

Goal. A `SIGPROF` profiler with confidence intervals and flame-graph output.
Steps. (1) `prof.c`: `timer_create` at `LTRACE_HZ`; `sigaction` handler with `SA_SIGINFO|SA_RESTART`; read `REG_RIP`/`REG_RBP` from the `ucontext`; walk; push a sample record. (2) Audit the handler for async-signal safety (list every call in a comment). (3) `cpu_shares.c` target with calibrated 70/20/10 work; the accuracy test with intervals. (4) `symbolize.py --profile`: folded stacks with counts and a summary table with $\hat{p}$ and 95% intervals per function. (5) Jittered intervals; a test target with a periodic 100 Hz phase to show aliasing at 100 Hz sampling and its absence at 997 Hz jittered.
Acceptance. Accuracy test passes; aliasing demonstration recorded in `docs/log.md` with numbers; a flame graph of `cforward` decode shows `matvec` dominating with the expected share.
Feeds. Post 3 section 7; all of Post 3b.

A3.4 Threads *8 h*

Goal. Per-thread attribution and a TSan-clean tracer under a multithreaded target.
Steps. (1) `hooks_pthread.c`: interpose `pthread_create` to wrap the start routine (create the thread's ring and timer on entry, flush on exit via a `pthread_key` destructor). (2) Per-thread `SIGEV_THREAD_ID` timers. (3) Ring overflow counters reported at exit. (4) Stress test (8 threads, 10 kHz, 30 s) under TSan; the OpenMP build of `cforward` traced and profiled per thread.
Acceptance. Stress test passes under TSan with zero reports; per-thread profile of OpenMP `cforward` shows the expected symmetric shares and the serial sections on thread 0.
Feeds. Post 3 results (the per-thread profile) and Post 6's profiling of the server.

A3.5 CUDA runtime interposition *10 h*

Goal. A trace of what a PyTorch step asks of `libcudart`.
Steps. (1) Declare the needed prototypes yourself in `hooks_cuda.c` (no CUDA headers at build time): `cudaMalloc`, `cudaFree`, `cudaMemcpy`, `cudaMemcpyAsync`, `cudaLaunchKernel`, `cudaStreamSynchronize`, `cudaDeviceSynchronize`; resolve with `RTLD_NEXT`. (2) Record sizes, kinds, stream ids, function pointers, timestamps. (3) Verify with `ldd//proc/<pid>/maps` that the target's `cudart` is dynamic. (4) Run: `python -c` doing a forward/backward of a small model; then a HF `generate` loop; then (optional) vLLM for 50 requests. (5) `symbolize.py --cuda`: timeline of `cudaMalloc` bytes, memcpy bytes by direction over time, launch rate over time with inter-launch gaps histogram.
Acceptance. The timeline shows the caching allocator's warm-up burst and near-zero `cudaMalloc` afterwards; launch-rate plot exists; `docs/cuda-notes.md` records what could not be intercepted and why.
Feeds. Post 3 central figure and the PyTorch section.

A3.6 Overhead study *5 h*

Goal. Numbers for the cost of the tool.
Steps. (1) `bench/overhead.c`: malloc-heavy loop (10^7 allocations of mixed sizes) untraced, traced without stacks, traced with stacks; profiler at 100, 1000, 10000 Hz on `cforward`. (2) Report slowdown factors and, for the profiler, measured overhead vs the $f \cdot c$ model.
Acceptance. Table in README; profiler overhead at 997 Hz below 1%; stacks cost documented.
Feeds. Post 3 results; Post 3b overhead paragraph.

A3.7 Stretch: a WRF binary *4 h*

Goal. Point the profiler at a WRF-Fire run from the compiler study and compare host-side profiles across two compiler builds.
Acceptance. Two flame graphs and a one-paragraph comparison in `docs/log.md`.
Feeds. A paragraph in Post 3's "what this predicts elsewhere".

4.6 Test plan

Exact counts

`tests/targets/alloc_counter.c` performs exactly N mallocs of known sizes after printing a marker; the test runs it under the tracer and checks the count and byte total of events

after the marker (libc's own startup allocations happen before). Also: every `free` matches a logged `malloc`; live-at-exit equals the planted leak.

Reentrancy
A hook configured to call `printf` (deliberately, in a test build) must neither recurse infinitely nor deadlock; bump-arena pointers passed to `free` are ignored.

Stack walker
A target with a known call chain `a->b->c->malloc` compiled with frame pointers yields those return addresses in order (resolve offline and compare names); a frame-pointer-less frame truncates cleanly rather than crashing (guard reads against the stack bounds from `pthread_getattr_np`).

Profiler accuracy
`cpu_shares.c` spends 70/20/10% of time in three functions, calibrated by work units; at 997 Hz for 10 s the estimates fall within the computed 95% intervals in 19 of 20 runs (run 20 times in `bench/`, 1 time in tests with a wide tolerance).

Signal safety
Run the profiler at 10 kHz under a target that calls `malloc` in a tight loop for 30 s with 8 threads: no deadlock, no crash, ring overflow counted and reported, never silently dropped. Run under TSan.

Launcher
Exit status propagation (0, 3, killed by `SIGSEGV` $\Rightarrow$ 139); `SIGINT` forwarded; existing `LD_PRELOAD` preserved.

Symbolizer
Round trip: known function addresses in a PIE binary at two different ASLR layouts resolve to the same names.

Interposition matrix
Table of (static, dynamic) $\times$ (PRELOAD, –wrap): which calls are seen. Encode it as a test with four tiny targets.

4.7 Measurements and blog briefs

(i) Allocation histogram by size and by call site for `cforward` and for a PyTorch forward pass; (ii) `cudaMalloc` timeline showing the caching allocator's warm-up; (iii) `cudaMemcpy` bytes by direction; (iv) launch rate vs time with host gaps; (v) profiler overhead vs frequency; (vi) estimated fractions with confidence intervals vs ground truth; (vii) the interposition matrix.

Blog plan

Post 3. "What does PyTorch actually ask the CUDA runtime for? 400 lines of `LD_PRELOAD` later"
Audience and thesis. For people who profile with Nsight or `torch.profiler` and have never looked underneath. Thesis: dynamic linking is an API you can program against; a preloaded library plus offline symbolization gives you most of a tracer at almost no overhead, and what it shows about a real framework is not what the documentation suggests. Subtitle: "N `cudaMalloc` calls in the first second, M in the next minute".
Length. 2500–3000 words.
Figures.
1. **Figure 1.** (central) Timeline of a PyTorch training step loop: `cudaMalloc` bytes (bars) and

kernel-launch rate (line) over time, warm-up burst visible, with the steady state annotated.
2. **Figure 2.** Lookup-scope diagram: executable → `LD_PRELOAD` libs → `DT_NEEDED` breadth-first, with the arrow showing where `malloc` resolves with and without the preload.
3. **Figure 3.** `readelf -r` excerpt (as a code figure) of the `JUMP_SLOT` relocation for `malloc` in `cforward`, next to the `objdump` of the PLT stub.
4. **Figure 4.** `cudaMemcpy` bytes by direction per step; inter-launch gap histogram (log x).
5. **Figure 5.** Interposition matrix as a 2×2 table with check marks.
6. **Figure 6.** Overhead: slowdown vs mode (no stacks, stacks) and profiler overhead vs frequency with the $f \cdot c$ line.

Outline.

1. Lede: the question, the tool, the headline count.
2. Fig. 1 and the caching-allocator explanation in two sentences.
3. Setup: framework version, model, GPU, tag; how `cudart` was confirmed dynamic.
4. Symbol lookup scope (Fig. 2) and the CS:APP rule; `RTLD_NEXT`; Fig. 3.
5. The bootstrapping problem: why `dlsym` may call `malloc`, the thread-local guard and the bump arena, in 15 lines of code.
6. Why symbolize offline: ASLR arithmetic ($a - m + o$), the `maps` snapshot, and the cost you avoid on the hot path.
7. The sampling profiler in one section: the `ucontext` registers, the frame-pointer walk (code excerpt), and the one rule of async-signal safety; the accuracy test with intervals (a small table).
8. What the traces show about PyTorch: Fig. 1 again in detail, Fig. 4, the H2D pattern of the input pipeline, launch gaps as the symptom of CPU-bound serving.
9. What cannot be intercepted (Fig. 5): static `cudart`, internal PLT bypass, protected visibility; the `--wrap` comparison numbers.
10. Overhead (Fig. 6).
11. What went wrong: the infinite recursion on first `dlsym`; `backtrace()` allocating inside a hook; the `free` of a bump-arena pointer handed to glibc.
12. What this predicts: the same technique on `libtt_metal` or any accelerator runtime; what a CUPTI-free profiler can and cannot see; one prediction about launch-gap distribution under vLLM.
13. Reproduce box.

Code to show. The hook pattern (with guard); the frame-pointer walk; the `sigaction` setup; one `readelf` excerpt.

Post 3b. "A sampling profiler is a statistics problem"

Thesis. A profile is an estimate with an error bar, and the three decisions—sample count, sampling frequency, jitter—are statistical decisions most tools make silently. **Length** 1200–1500 words. **Figures:** estimated vs true fraction for the 70/20/10 target with 95% intervals at $N = 10^2, 10^3, 10^4$; the aliasing demonstration (a 100 Hz periodic phase sampled at 100 Hz vs 997 Hz jittered). **Outline:** the Bernoulli model and the interval formula with the "1% to ±20% needs 9500 samples" example; overhead as $f \cdot c$ and the measured c; aliasing and the prime/jitter fix; what the profiler cannot attribute (frames without frame pointers, time in the kernel); reproduce box. **What went wrong:** the first accuracy test failed because the work loop was optimized away at `-O2`—how to calibrate work units so the compiler cannot remove them.

4.8 Git lesson and push checklist

Git lesson for this project

Topics: history archaeology (`log -S`, `log -G`, `blame`); the reflog as a safety net; worktrees; versioned hooks; `.gitattributes`.

Practice.

1. `git log -S'RTLD_NEXT' --oneline` finds the commit that introduced a string; `-G<regex>` matches diff lines; `git blame -w -C -C src/hooks_malloc.c` follows moved code. Use them to write the "history" section of `docs/design.md`.
2. Break something on purpose: `git reset --hard HEAD~3`, then recover with `git reflog` and `git reset --hard HEAD@{1}`. Understand that the reflog is local and expires (90 days by default); unreachable objects are deleted by `gc`.
3. `git worktree add ../ltrace-wrap wrap-variant` to build the `--wrap` variant side by side with the PRELOAD one without stashing or switching. `git worktree list`, `remove`.
4. Version your hooks: put `pre-commit` (runs `make fmt` check and `make BUILD=asan test`) in `.githooks/` and set `git config core.hooksPath .githooks`; document it in the README. A hook that is too slow gets bypassed with `--no-verify`, so keep it under 30 s.
5. `.gitattributes`: `*.bin binary`, `tools/* linguist-language=Python`, `* text=auto eol=lf`.
6. Tag `v0.3.0` at A3.5.

Chapter 5

Project 4: `pagedkv` — a memory allocator and a paged KV cache with prefix caching

5.1 Goal

Two layers. First, a general-purpose dynamic memory allocator (CS:APP's Malloc Lab done to completion: segregated explicit free lists, boundary-tag coalescing, in-place `realloc`, measured throughput and utilization against glibc). Second, built on `mmap`'d memory and the same design ideas, a vLLM-style paged KV-cache manager: fixed-size blocks, per-sequence block tables, reference counting with copy-on-write for forked sequences, a radix tree over token prefixes for prefix caching, pluggable eviction policies, and a cold tier on disk. The attention kernel from project 2 is adapted to read K/V through block tables, and TLB behaviour with 4 KB vs 2 MB pages is measured.

Research hook

This is the data structure behind PagedAttention and SGLang's RadixAttention. Building it with a replacement-policy analysis attached lets you ask the operating-systems question—how far is a practical eviction policy from the offline optimum?—for KV blocks instead of pages, and the capacity-limited block pool is the same object as the memory bank in your memory-slot experiments.

5.2 CS:APP reading map

Section	What you build with it
§ 9.1–9.6 address spaces, page tables, TLBs	the huge-page experiment; TLB reach arithmetic
§ 9.7 case study: Core i7 address translation	interpreting `dTLB-load-misses`
§ 9.8 memory mapping, `mmap`, copy-on-write	the block pool; cold tier via file mapping; COW analogy
§ 9.9 dynamic memory allocation	the allocator (all of it)
§ 9.10–9.11 garbage collection, memory bugs	reference counting; the shadow checker in tests

5.3 Mathematical background

5.3.1 Allocator design

Block format. Alignment 16. Header word: `size | prev_alloc << 1 | alloc` (the low 4 bits of a 16-aligned size are free). Allocated blocks: header + payload (no footer, since the next block's `prev_alloc` bit replaces it—this saves 8 bytes per block). Free blocks: header, `pred`/`succ` pointers, padding, footer. Minimum block: $8 + 8 + 8 + 8 = 32$ bytes. Block size for a payload request r:

$$\mathrm{bsize}(r) = \max\big(32,\ \lceil (r+8)/16 \rceil \cdot 16\big). \tag{5.1}$$

Segregated free lists. Size classes $\{32, 48, 64, 96, 128, 192, 256, \dots\}$—powers of two with a midpoint in each binade—up to 4 KB, then one list per power of two to 1 MB, then one oversize list. Class index from `__builtin_clzl`: for size s, let $k = 63 - \mathrm{clz}(s)$; the midpoint test is bit $k - 1$. Placement: first fit in the request's class, then the next classes; split when the remainder ≥ 32; the remainder goes to its class (LIFO insertion).

Coalescing on free with boundary tags: four cases (neither neighbour free; next free; prev free; both), using the `prev_alloc` bit to avoid reading the previous block's footer unless it is free. Immediate coalescing keeps fragmentation low; deferred coalescing is faster—measure both.

realloc. Shrink in place; grow in place if the next block is free and large enough (or is the epilogue and the heap can extend); otherwise allocate-copy-free.

Metrics. Peak utilization $U = \max_t \big(\sum \mathrm{payload}_t\big)/\mathrm{heap\ size\ at\ end}$; throughput in operations per second over a trace. Internal fragmentation is rounding waste; external is free memory unusable because it is non-contiguous. The CS:APP trace format—`a id size`, `r id size`, `f id`—is what `bench/traces/` uses; generate traces mimicking inference workloads (many equal-sized medium blocks with bursts of large ones) in addition to the classic ones.

Buddy allocator (optional variant, used later for the block pool's size classes if you support mixed block sizes): orders $k = 0..K$, block size $2^k m_0$; the buddy of a block at offset o at order k is at $o \oplus 2^k m_0$; splitting and merging are $O(\log)$ and internal fragmentation is up to 50%.

5.3.2 KV-cache geometry

Per token, per layer, with h_{kv} KV heads of dimension d_h and b_{kv} bytes per element:

$$\mathrm{kv}_{\mathrm{layer}} = 2\, h_{kv}\, d_h\, b_{kv}, \qquad \mathrm{kv}_{\mathrm{token}} = L \cdot \mathrm{kv}_{\mathrm{layer}}. \tag{5.2}$$

Llama-3-8B ($L = 32$, $h_{kv} = 8$, $d_h = 128$, fp16): 4 KB per layer per token, 128 KB per token; a 128K-token context is 16 GB. `stories15M` is tiny ($L = 6$, $d = 288$); use it for correctness and a synthetic large config for the memory experiments.

Blocks. A block holds B tokens of one layer's K (or V, or both—choose K and V in separate pools so a kernel's reads are contiguous per tensor): block bytes $= B\, h_{kv}\, d_h\, b_{kv}$; $B = 16$ for Llama-3-8B gives 32 KB per K block per layer. Layout inside a block: `[B][h_kv][d_h]` (token-major) so appending a token writes one contiguous slab and attention for head j' reads with stride $h_{kv}d_h$—or `[h_kv][B][d_h]` (head-major) so a head's keys are contiguous. Implement token-major first, head-major as a measured alternative.

Fragmentation. A sequence of n tokens wastes at most $B - 1$ token slots (internal), versus a contiguous preallocation to $T_{\max}$ that wastes $T_{\max} - n$. With a pool of $N = M/\text{block bytes}$ blocks and S concurrent sequences, the expected waste is $S(B - 1)/2$ tokens—the argument for B in the 16–32 range.

5.3.3 Sharing: reference counts and copy-on-write

Each physical block has `refcnt`. `fork(seq)` copies the block table and increments every block's count: $O(\text{len}/B)$ and no data copied. A write to position t of a sequence whose block has `refcnt > 1` first allocates a fresh block, copies the B tokens, decrements the old count, and updates the table—copy-on-write, exactly the kernel's `fork` strategy for pages (§ 9.8.2). Only the *last* block of a forked sequence is ever written, so at most one copy per fork.

Invariants (tested after every operation in the property test): $\sum_{\text{blocks}} \text{refcnt} = \sum_{\text{seqs}} |\text{table}| + |\text{radix-tree-held blocks}|$; a block is on the free list iff `refcnt == 0` and it is not in the tree; no two sequences write the same block.

5.3.4 Prefix cache: radix tree over blocks

Because blocks are the unit of sharing, prefix matching has granularity B: node = one full block of B token ids, keyed by those ids, child lookup by the next block's ids (hash of B ints, or a sorted child array). A finished (or in-flight) sequence inserts its *full* blocks; partial last blocks are not cacheable. `match(tokens)` walks from the root while the next B tokens equal a child's key, returning the list of physical blocks and the matched length $m = kB$. A new request then reuses those blocks (incrementing `refcnt`) and only prefills tokens $m..n$.

Eviction. Candidates are tree leaves with `refcnt == 1` (held only by the tree). Policies, each a score function; evict the minimum:

- LRU: score $= t_{\text{last access}}$.
- LFU: hit count, with aging.
- Depth-aware LRU: score $= t_{\text{last}} - \gamma \cdot \text{depth}$, where depth is the node's distance from the root; shallow nodes (shared system prompts) are protected because evicting one orphans every descendant.
- OPT (Belady's MIN): evict the leaf whose next access is furthest in the future. Not implementable online—it needs the future—but computable offline from a recorded trace (below), and it is the yardstick every other policy is measured against.

Store leaves in an intrusive doubly linked list ordered by score for LRU ($O(1)$), or a binary heap keyed by score for the others ($O(\log n)$); evicting a leaf may expose its parent as a new leaf.

5.3.5 Reuse distance and optimal replacement

Treat the stream of block accesses (every `match` hit, every append) as a reference trace $r_1, r_2, \ldots, r_n$ over block ids, exactly as the virtual-memory chapter treats page references. Two classical tools then give you the whole hit-rate picture without running a simulation per cache size.

Reuse (stack) distance. For access r_i, let d_i be the number of *distinct* blocks referenced since the previous reference to the same block ($d_i = \infty$ for a first reference). Under LRU with capacity C blocks, access i is a hit iff $d_i < C$. Hence a single pass computing the histogram $H(d)$ yields the LRU miss-ratio curve for *every* capacity at once (Mattson's stack algorithm):

$$\mathrm{miss}_{\mathrm{LRU}}(C) = \frac{1}{n}\Big(n_\infty + \sum_{d \geq C} H(d) \Big). \tag{5.3}$$

Computing d_i naively is $O(n^2)$; with a Fenwick (binary indexed) tree over access timestamps it is $O(n \log n)$: keep, for each block, the timestamp of its last access; mark timestamps in the tree; d_i is the count of marked timestamps greater than the block's previous timestamp, after which you unmark the old timestamp and mark the new one. This is the algorithm behind every modern cache-sizing tool, and it is twenty lines.

Stack property. LRU and OPT are *stack algorithms*: the set of blocks resident with capacity C is a subset of the set resident with capacity $C + 1$, so misses can only decrease as capacity grows. FIFO is not, which is Belady's anomaly (more memory, more misses); construct the classic trace $1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5$ with FIFO at $C = 3$ (9 misses) and $C = 4$ (10 misses) as a test case, and note that your radix tree's LIFO free-list reuse order can introduce the same effect if you are careless about what "access" means.

Belady's MIN (OPT). Offline, with the whole trace known: on a miss with a full cache, evict the resident block whose next reference is furthest in the future (or never). Compute next-use times in one reverse pass over the trace (next_i = index of the next reference to r_i, or ∞), then simulate forward with a max-heap keyed by next use. OPT is a lower bound on the misses of any online policy for a given capacity; the ratio $\mathrm{miss}_{\mathrm{LRU}}(C)/\mathrm{miss}_{\mathrm{OPT}}(C)$ is the competitive gap you measure. Two caveats specific to KV blocks: (i) blocks are not independent—a child block is useful only while its ancestors are resident—so the tree-structured OPT evicts leaves first and treats a subtree's next use as the min over its descendants; implement the plain block-level OPT first (a valid lower bound), then the leaf-constrained variant (achievable); (ii) blocks referenced by running sequences are pinned (`refcnt > 1`) and excluded from eviction in every policy, so the effective capacity is C minus the pinned set.

What the curves tell you. For a multi-agent workload with shared system prompts, $H(d)$ is bimodal: a spike at small d (the prompts, re-referenced constantly) and a long tail (per-conversation blocks). LRU already captures the spike; the interesting region is where C sits in the tail, and there LRU, LFU and depth-aware LRU separate. Plot (5.3) together with the simulated LFU, depth-aware and OPT curves against C; identify the smallest C at which LRU is within 10% of OPT—that number, not peak hit rate, is the design output (it is how large the pool must be before policy stops mattering).

Working set. Denning's working set $W(t, \tau)$ is the set of distinct blocks referenced in $(t - \tau, t]$; its size over time, for a few τ, is a one-figure description of the workload and tells you whether a fixed pool can ever hold the hot set. Compute it from the same timestamps in $O(n)$ with a sliding window.

5.3.6 Pages and the TLB

A pool of M bytes with 4 KB pages has $M/4096$ pages; an L1 dTLB of 64 entries covers 256 KB and a 2048-entry STLB covers 8 MB, so random block accesses across a multi-GB pool miss the TLB almost every time and pay a page walk (tens of ns, partially hidden by the page-walk caches). With 2 MB pages the STLB covers 4 GB. Reserve the pool with `mmap(NULL, M, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)`, 2 MB-aligned, then `madvise(MADV_HUGEPAGE)` (transparent huge pages; check `/sys/kernel/mm/transparent_hugepage/enabled`) or `MAP_HUGETLB` (preallocated via `/proc/sys/vm/nr_hugepages`). First-touch page faults cost $\sim$ 0.1–0.25 μs per 4 KB page (zeroing included); `MAP_POPULATE` prefaults. Measure all of it with `perf stat` (events `dTLB-load-misses`, `dTLB-loads`, `page-faults`).

Cold tier. A file in `/tmp` (or a real NVMe path) accessed with `pwrite`/`pread` or mapped with `mmap(MAP_SHARED)`; `madvise(MADV_DONTNEED)` and `posix_fadvise(POSIX_FADV_DONTNEED)` let you defeat the page cache so you measure the device, not DRAM. The block manager's `swap_out(block)`/`swap_in(block)` move blocks between tiers; eviction from the hot tier may swap instead of drop.

5.4 Architecture

5.4.1 Directory layout

```
pagedkv/
  include/pagedkv/
    malloc.h       pk_malloc/pk_free/pk_realloc/pk_calloc + pk_heap_stats
    pool.h         fixed-size block pool over mmap: alloc/free block ids, refcounts,
    huge pages
    blocktable.h   per-sequence logical->physical mapping, append, fork (COW)
    radix.h        prefix tree over blocks; match/insert/evict; policies
    tier.h         cold tier (file-backed); swap in/out; throttle for experiments
    kvcache.h      facade used by cforward: write_kv(seq, layer, pos, k, v), gather for
     attention
    reuse.h        reference-trace recorder; reuse-distance histogram (Fenwick); LRU
    MRC; OPT simulator; working set
  src/             malloc.c pool.c blocktable.c radix.c tier.c kvcache.c reuse.c
  tests/           test_malloc.c test_pool.c test_blocktable.c test_radix.c test_tier.c
     test_kvcache.c test_props.c
  bench/           mdriver.c (trace replay, util/throughput), tlb.c (4K vs 2M), tier_bw
    .c, mrc.c (policies vs OPT)
  scripts/         gen_traces.py, model_check.py (Python reference model for property
    tests), plots
```

5.4.2 Public API

```
/* malloc.h */
int   pk_heap_init(size_t max_bytes);      /* mmap'd arena with emulated sbrk */
void *pk_malloc(size_t n);  void pk_free(void *p);  void *pk_realloc(void *p, size_t n)
    ;
void *pk_calloc(size_t a, size_t b);
typedef struct { size_t heap_bytes, payload_bytes, peak_payload, n_free_blocks; }
    pk_heap_stats;
```

```
pk_heap_stats pk_stats(void);  int pk_check_heap(void);     /* consistency checker,
    used by tests */

/* pool.h */
typedef uint32_t pk_block;  #define PK_NOBLOCK UINT32_MAX
typedef struct { size_t block_bytes; uint32_t n_blocks; bool huge_pages; bool populate;
     } pk_pool_cfg;
typedef struct pk_pool pk_pool;
pk_pool *pk_pool_create(pk_pool_cfg c);   void pk_pool_destroy(pk_pool *);
pk_block pk_pool_alloc(pk_pool *);        /* refcnt = 1; PK_NOBLOCK if empty */
void     pk_pool_incref(pk_pool *, pk_block);  void pk_pool_decref(pk_pool *, pk_block)
    ; /* frees at 0 */
uint32_t pk_pool_refcnt(const pk_pool *, pk_block);
void    *pk_pool_ptr(const pk_pool *, pk_block);    uint32_t pk_pool_free_count(const
    pk_pool *);

/* blocktable.h */
typedef struct { pk_block *phys; uint32_t n_blocks, n_tokens, cap; } pk_table;
int  pk_table_append_token(pk_table *, pk_pool *, uint32_t B);   /* allocs a block when
     needed */
int  pk_table_fork(const pk_table *src, pk_table *dst, pk_pool *);
int  pk_table_make_writable(pk_table *, pk_pool *, uint32_t logical_block, size_t
    block_bytes); /* COW */
void pk_table_release(pk_table *, pk_pool *);

/* radix.h */
typedef enum { PK_EVICT_LRU, PK_EVICT_LFU, PK_EVICT_DEPTH_LRU } pk_policy;   /* OPT
    lives in reuse.h (offline) */
typedef struct pk_radix pk_radix;
pk_radix *pk_radix_create(pk_pool *, uint32_t B, pk_policy);
uint32_t  pk_radix_match(pk_radix *, const int32_t *tokens, uint32_t n, pk_block *out,
    uint32_t max); /* returns matched blocks */
int       pk_radix_insert(pk_radix *, const int32_t *tokens, uint32_t n, const pk_block
     *phys);
uint32_t  pk_radix_evict(pk_radix *, uint32_t want_blocks);   /* returns freed count */
typedef struct { uint64_t lookups, hit_blocks, inserted, evicted; } pk_radix_stats;

/* kvcache.h -- what cforward's attention sees */
typedef struct pk_kv pk_kv;      /* owns K and V pools (one pair per layer) and all
    tables */
pk_kv *pk_kv_create(const cf_config *cfg, size_t pool_bytes, uint32_t B, bool
    huge_pages);
typedef struct { const pk_block *blocks; uint32_t n_tokens; uint32_t B; const pk_pool *
    kpool, *vpool; } pk_kv_view;
void pk_kv_write(pk_kv *kv, uint32_t seq, int layer, uint32_t pos, const float *k,
    const float *v);
pk_kv_view pk_kv_view_for(const pk_kv *kv, uint32_t seq, int layer);

/* reuse.h -- offline analysis of a recorded block-reference trace */
typedef struct { uint32_t *ref; size_t n; } pk_trace;             /* block ids in
    access order */
int    pk_trace_record_begin(pk_radix *, pk_trace *);            /* radix tree appends
    every access */
```

```
void   pk_reuse_histogram(const pk_trace *, uint64_t *H, size_t maxd, uint64_t *n_inf);
      /* O(n log n) */
void   pk_mrc_lru(const uint64_t *H, size_t maxd, uint64_t n_inf, size_t n, double *
    miss /* [maxd] */);
size_t pk_sim_opt(const pk_trace *, size_t C, const uint8_t *pinned);
        /* misses */
size_t pk_sim_opt_tree(const pk_trace *, size_t C, const uint32_t *parent);
       /* leaf-constrained */
size_t pk_sim_policy(const pk_trace *, size_t C, pk_policy);
       /* same harness */
void   pk_working_set(const pk_trace *, size_t tau, uint32_t *wss /* [n] */);
```

5.4.3 Attention over paged blocks

`cf_attention` gains a variant `cf_attention_paged(out, q, view_k, view_v, ...)`: for key position u, the address is `pk_pool_ptr(kpool, blocks[u / B]) + (u % B) * h_kv * d_h * sizeof(float)`. Iterate block by block so the inner loop over the B tokens of a block is contiguous and the online-softmax update (project 2) is applied per block. Measure the cost of indirection against the contiguous kernel; it should be within a few percent for $B \geq 16$.

5.5 Assignments

A4.1 The allocator *14 h*

Goal. A segregated-fit allocator with measured utilization and throughput against glibc.
Steps. (1) `bench/mdriver.c`: trace parser (`a`/`r`/`f`), payload-pattern verification, utilization and throughput reporting, a `--glibc` switch. (2) `scripts/gen_traces.py`: the classic trace shapes (coalescing, realloc, binary) and two inference-like traces (steady medium blocks with large bursts; many short-lived small blocks). (3) Implicit free list with immediate coalescing and `pk_check_heap`; pass correctness on all traces. (4) Explicit free list (LIFO). (5) Segregated classes with the `clz` index; measure; tune class boundaries. (6) In-place `realloc`; footer elimination for allocated blocks; deferred coalescing as a measured variant. (7) The shadow checker in the test harness.
Acceptance. All traces correct under the shadow checker; $U \geq 0.85$ on the classic traces; throughput within $3\times$ of glibc; `pk_check_heap` passes after every operation in debug builds; a table in `docs/alloc.md` with (variant, trace, U, ops/s).
Feeds. Post 4 section 4 (the allocator in two pages) and Fig. 2.

A4.2 Block pool, pages and the TLB *10 h*

Goal. An `mmap`'d block pool with huge-page support and the TLB experiment.
Steps. (1) `pool.c`: reservation, alignment, `madvise(MADV_HUGEPAGE)` / `MAP_HUGETLB`, `MAP_POPULATE`, free-list stack, refcounts with debug underflow checks. (2) `bench/tlb.c`: touch one cache line in each of 10^7 random blocks over pools of 256 MB, 1, 4, 16 GB, with 4 KB and 2 MB pages; ns per access and `dTLB-load-misses` via `perf stat`; prefetch-defeating access order (dependent, as in pointer chasing). (3) Page-fault cost: time to first-touch a 4 GB pool with and without `MAP_POPULATE`; faults counted. (4) Record THP status and `nr_hugepages` procedure in `docs/machine.md`.
Acceptance. Pool tests pass; the TLB figure shows the 4 KB curve rising where the pool exceeds STLB reach and the 2 MB curve flat to several GB; page-fault cost per GB reported; miss counts

consistent with the latency jump.
Feeds. Post 4 central figure (Fig. 1) and section 7.

A4.3 Block tables, fork, copy-on-write *8 h*

Goal. Sequence-level bookkeeping with sharing semantics proven by a model-based test.
Steps. (1) `blocktable.c`: append (allocating on block boundary), fork (refcount increments), `make_writable` (COW), release. (2) `kvcache.c`: per-layer K and V pools; `pk_kv_write`; `pk_kv_view_for`. (3) `scripts/model_check.py`: a Python model of the same operations over dictionaries that emits a random operation trace and expected observations; `tests/test_props.c` replays it and checks the three invariants after every step. (4) Benchmark: fork 64 sequences from a 2K-token prefix; measure blocks allocated and bytes copied (expect ≤ 1 block per fork).
Acceptance. COW tests and 10^5-step property test pass; invariants hold; the fork benchmark reports at most one block copied per fork.
Feeds. Post 4 section 6 (COW with the kernel analogy).

A4.4 Radix tree and online eviction policies *10 h*

Goal. Prefix caching with LRU, LFU and depth-aware LRU, and a hit-rate benchmark on agent-style workloads.
Steps. (1) `radix.c`: node = block, child lookup by hash of B ids, `match`, `insert`, refcount interaction with the pool. (2) Eviction: intrusive LRU list; heap-based LFU and depth-aware variants; leaf exposure on eviction. (3) `scripts/gen_workload.py`: A agents with 1–2K-token system prompts, multi-turn conversations with branching, interleaved arrivals; emits token sequences. (4) `bench/prefix_hitrate.c`: replay a workload through the tree with a given pool size and policy; report block hit rate, tokens prefilled, evictions. (5) Random-sequence tests against the brute-force prefix finder; eviction-safety tests.
Acceptance. Radix tests pass; hit-rate benchmark produces a CSV over pool sizes and policies; eviction never touches a block with `refcnt > 1` (asserted in debug).
Feeds. Post 4 section 5 (eviction scores) and the simulated policy curves of Fig. 3.

A4.5 Reuse distance, OPT, working set *12 h*

Goal. The offline analysis of Section 5.3.5 and the policies-vs-OPT figure.
Steps. (1) `reuse.c`: trace recorder hooked into the radix tree (every access appends a block id). (2) Brute-force $O(n^2)$ reuse distance (test oracle), then the Fenwick-tree version; the histogram and Equation (5.3). (3) LRU simulator for cross-checking the histogram-derived curve at every C. (4) FIFO simulator and the Belady-anomaly test trace. (5) OPT: reverse pass for next-use, forward simulation with a max-heap; then the leaf-constrained variant using the parent array from the radix tree. (6) Working-set size over time with a sliding window. (7) `bench/mrc.c`: on the A4.4 workloads, miss ratio vs C for LRU (both ways), LFU, depth-aware, OPT, tree-OPT; `scripts/plot_mrc.py`; the "smallest C within 10% of OPT" number.
Acceptance. All reuse-analysis tests in the test plan pass; histogram-derived and simulated LRU curves agree exactly; OPT $\leq$ every policy at every C and monotone; the MRC figure and the 10%-of-OPT capacity are in the README.
Feeds. Post 4 Fig. 3–4 and section 8; all of Post 4b.

A4.6 Cold tier and paged attention *10 h*

Goal. A second storage tier and `cforward` reading K/V through block tables.
Steps. (1) `tier.c`: file-backed store, `swap_out`/`swap_in`, page-cache defeat, throttle option; `bench/tier_bw.c` vs `dd`. (2) Eviction-to-tier instead of drop, with a hit from the tier counted separately. (3) `cf_attention_paged` in `cforward` (branch there; submodule or copy here) iterating block by block with the online-softmax update per block; bit-identity test. (4) Benchmark paged vs contiguous attention vs $B \in \{8, 16, 32, 64\}$. (5) End to end: 64 forked sequences sharing a 2K prefix, decoding 128 tokens each, through the paged path.
Acceptance. Tier tests pass with throughput within 20% of `dd`; paged attention bit-identical to contiguous; overhead $\leq 5\%$ at $B \geq 16$; the end-to-end run reports total blocks and bytes and matches the fork arithmetic.
Feeds. Post 4 Fig. 5 and results; Post 6's engine.

5.6 Test plan

Allocator correctness
Trace replay with a payload pattern (fill each block with its id; verify before free); alignment of every returned pointer; `pk_check_heap` after every op in debug builds (free list consistency, no adjacent free blocks after coalescing, header/footer agreement, every free block in exactly one list). A *shadow checker* in the test harness—a byte bitmap marking allocated ranges—catches overlap, since ASan cannot see inside your allocator.

Coalescing
Four hand-built cases; realloc in place (pointer unchanged) when the next block is free; realloc preserves contents.

Utilization/throughput
Assert $U \geq 0.85$ on the classic traces and throughput within $3\times$ of glibc (soft).

Pool
Alloc until `PK_NOBLOCK`; free all; alloc again returns every id exactly once; refcounts never underflow (abort in debug).

COW semantics
Fork, write to child's last block, parent's data unchanged, child's block table differs in exactly one entry, old block's refcount decremented.

Radix
Random token sequences with shared prefixes: `match` agrees with a brute-force longest-common-prefix over all inserted sequences (rounded down to B); insert is idempotent; eviction never frees a block with refcount > 1; after evicting everything the pool is full again.

Property/model-based test
10^5 random operations (append, fork, finish+insert, match+reuse, evict) against a Python dictionary model exported as a trace; the three invariants from the sharing section hold after every step.

Reuse analysis
Reuse distances on hand-made traces match a brute-force $O(n^2)$ computation; the LRU miss count from (5.3) equals the simulated LRU miss count for every C on 10^3 random traces; FIFO on the Belady-anomaly trace gives 9 misses at $C = 3$ and 10 at $C = 4$; OPT misses $\leq$ every other policy's misses for every C; OPT is monotone in C; the leaf-constrained OPT never evicts a node with a resident child.

Tier
: Swap out, corrupt the hot copy, swap in, compare; throughput of the tier within 20% of `dd`-measured device bandwidth.

Paged attention
: Bit-identical to contiguous attention for the same K/V (same operation order within blocks—design the kernel so this is true, and the test enforces it).

Sanitizers
: ASan on everything except the allocator's own arena (use a `PK_ASAN_POISON` hook with `__asan_poison_memory_region` for free blocks—optional, instructive).

5.7 Measurements and blog briefs

(i) Utilization and throughput vs glibc per trace; (ii) random-access latency over the pool for 4 KB vs 2 MB pages, with dTLB miss counts; (iii) page-fault cost per GB; (iv) miss ratio vs pool size C for LRU (from the histogram and from simulation—they must agree), LFU, depth-aware LRU, OPT and tree-OPT on synthetic agent workloads; (v) reuse-distance histogram and working-set size over time; (vi) paged vs contiguous attention throughput vs B.

Blog plan

Post 4. "Paging a KV cache on a CPU: what the TLB thinks of PagedAttention"
Audience and thesis. For people who know vLLM's block tables from the paper and have not thought about the hardware page tables underneath them. Thesis: the KV cache is a virtual-memory problem twice over—blocks are pages at the application level, and the pool is pages at the hardware level—and the same arithmetic (reach, fragmentation, replacement) governs both. Subtitle: "random block access costs X ns with 4 KB pages and Y ns with 2 MB pages once the pool passes Z MB".
Length. 3000–3500 words.
Figures.
1. **Figure 1.** (central) Access latency (ns, y) vs pool size (log x, 256 MB to 16 GB) for 4 KB and 2 MB pages, with L1 dTLB and STLB reach marked as vertical lines; dTLB miss rate as a secondary panel.
2. **Figure 2.** Allocator utilization and throughput, your variants vs glibc, per trace (grouped bars).
3. **Figure 3.** Miss ratio vs pool size C (blocks, log x) for LRU, LFU, depth-aware LRU, OPT, tree-OPT on the multi-agent workload; the "within 10% of OPT" capacity marked.
4. **Figure 4.** Reuse-distance histogram (log–log) for the same workload, bimodal, with the system-prompt spike annotated; inset: working-set size over time for two τ.
5. **Figure 5.** Paged vs contiguous attention throughput vs block size B.
6. **Figure 6.** Block-geometry diagram: logical sequence → block table → physical blocks, with a forked sequence sharing all but the last block (refcounts shown).

Outline.
1. Lede: the two-level claim and the headline latency numbers.
2. Fig. 1 and the TLB-reach arithmetic that predicts where the curves bend (worked: 2048 entries × 4 KB = 8 MB; × 2 MB = 4 GB).
3. Setup.
4. The allocator in two pages: boundary tags, segregated lists, the $\lceil (r+8)/16 \rceil \cdot 16$ rounding,

Fig. 2, and what utilization means. Code excerpt: coalescing case 4.
5. Block geometry: kv per token, block bytes, the $S(B-1)/2$ expected waste, Fig. 6.
6. Sharing: refcounts and copy-on-write, the `fork(2)` analogy, the three invariants, and how the model-based test enforces them.
7. Eviction scores: LRU, LFU, depth-aware; why shallow nodes deserve protection.
8. Reuse distance in twenty lines, Equation (5.3), and Fig. 3–4: the capacity at which policy stops mattering is the design output.
9. The TLB experiment in detail: huge pages, `MAP_POPULATE`, page-fault cost; Fig. 1 again with the miss counts.
10. Paged attention: the indirection cost, Fig. 5, and why $B = 16$ is where everyone lands.
11. What went wrong: THP silently disabled (`madvise` vs `always`); the first latency benchmark measured the prefetcher, not the TLB; refcount underflow on double release caught by the debug assert.
12. What this predicts: GPU TLBs and why vLLM's 16-token blocks and 2 MB CUDA allocations interact; what a memory bank with a few hundred slots (your memory-slot experiments) looks like through the same lens—a reuse-distance histogram is the first thing to compute.
13. Reproduce box (needs root for `nr_hugepages`; say so).

Code to show. Coalescing case; the COW path; the Fenwick reuse-distance core; the `mmap`+`madvise` setup.

Post 4b. "Belady's MIN for KV blocks: how far is LRU from optimal?"

Thesis. Every KV-eviction heuristic should be reported against the offline optimum, and the optimum is cheap to compute from a trace. **Length** 1400–1800 words. **Figures:** Fig. 3 from the main post, reproduced alone; a small diagram of the reverse next-use pass; a bar chart of LRU/OPT miss ratio at three capacities for three workload shapes (single agent, 8 agents, 64 agents). **Outline:** the reference-trace view of a KV cache; stack distance and the one-pass miss-ratio curve (the Fenwick trick, with code); the stack property and Belady's anomaly (the 12-reference trace worked by hand, FIFO at $C = 3$ and 4); OPT and its proof sketch of optimality (exchange argument, three sentences); the tree constraint and why the plain OPT is still a valid lower bound; the gap on the three workloads; what an online policy could exploit (depth, agent id) and how much is left; reproduce box. **What went wrong:** counting an append as an access of the new block (which is always a miss) inflated every curve equally and hid the policy differences—define the trace precisely.

5.8 Git lesson and push checklist

Git lesson for this project

Topics: pull requests against yourself; branch protection; conventional commits; `revert` vs `reset` on public history; `cherry-pick`; `rebase --onto`; GitHub Releases.

Practice.

1. Protect `main` on GitHub (require PR, require status checks once CI exists in project 6). From now on nothing lands on `main` except through a PR.
2. For each assignment: branch, commits, `gh pr create --fill`, then review your own diff on GitHub as if it were someone else's—leave comments, fix them in new commits, then `gh pr merge --squash` (or `--rebase` if the commits are already clean). Squash merging is why the branch's commit hygiene can be looser than in project 2.

3. Adopt conventional commits: `feat(reuse): add leaf-constrained OPT`, `fix(pool): refcount underflow on double decref`, `perf(attn): block-major K layout`, `test`, `docs`, `chore`. These feed the CHANGELOG in project 6.
4. Undo on public history with `git revert <sha>` (a new commit), never `reset`; keep `reset` for local branches.
5. `git cherry-pick <sha>` to carry a fix from a feature branch to `main` (then rebase the branch to drop the duplicate); `git rebase --onto main A B` to transplant a sub-range of commits.
6. `gh release create v0.4.0 --generate-notes` after A4.6.

Chapter 6

Project 5: `tinyjit` — an AVX2 JIT compiler for fused tensor operations

6.1 Goal

A small compiler, in C, from a tensor-expression IR (elementwise arithmetic, transcendental approximations, reductions over fp32 arrays, and the quantize/dequantize operations of project 1) to x86-64 AVX2 machine code generated at run time into executable memory and called through a function pointer. It performs operator fusion (one loop, no intermediate arrays), linear-scan register allocation over the 16 `ymm` registers, vectorized main loop with a masked tail, and a constant pool addressed RIP-relatively—which means you compute a relocation by hand. As a stretch, the same code generator writes a relocatable ELF object file that `gcc` can link, closing the loop with CS:APP chapter 7.

Research hook

This is the last mile of every DL compiler (XLA, TVM, Triton's CPU backend, Tenstorrent's TT-MLIR lowering to kernels): an expression DAG becomes a loop nest becomes instruction bytes, and register pressure and memory traffic decide performance. Doing it for AVX2 removes every abstraction between you and the numbers, and the fused-vs-unfused measurement is the elementary case of the fusion decisions those compilers make.

6.2 CS:APP reading map

Section	What you build with it
§ 3.2–3.6 instruction formats, operands, control	the encoder; loop structure; condition codes for `jb`
§ 3.7 procedures, the x86-64 calling convention	your kernel's ABI (`rdi`, `rsi`, ...; callee-saved set)
§ 3.11 floating-point code, SIMD registers	`ymm` usage, `vzeroupper`
§ 7.4–7.7 ELF sections, symbols, relocations	the constant pool (`R_X86_64_PC32` arithmetic); the `.o` writer
§ 8.5 signals	catching `SIGSEGV`/`SIGILL` from a bad encoding in a forked test child
§ 9.8 `mmap`, protections	W⊕X: write, then `mprotect` to execute

6.3 Background

6.3.1 The IR

A kernel is a DAG of nodes over a single implicit loop index $i \in [0, n)$:

- `Arg(k)`: the k-th pointer argument, read as a vector of 8 floats at i.
- `Const(c)`: a broadcast fp32 constant.
- Binary `Add, Sub, Mul, Div, Max, Min`; unary `Neg, Abs, Sqrt, Rsqrt, Exp, Recip, Floor`; `FMA(a,b,c)` $= ab + c$.
- `Select(mask, a, b)` and comparisons `CmpLT, CmpGE` producing lane masks.
- `Store(k, v)`: write v to argument k at i.
- `ReduceSum(v)`, `ReduceMax(v)`: accumulate across the loop; written to a scalar output after the loop.
- Integer ops needed for the quantizers: `CvtF2I` (`vcvtps2dq`, with rounding mode), `CvtI2F`, `ShlI`, `AndI`, `OrI`, `AddI`.

The generated function has the fixed signature

```
typedef void (*tj_kernel)(float *const *args, size_t n, float *reduce_out);
```

so that every kernel is called the same way; `args[k]` is loaded into a register once in the prologue.

Two-pass operations (RMSNorm: a reduction followed by an elementwise pass that uses it) are expressed as two kernels, or one kernel with a `Pass2` barrier node that splits the loop in two; start with two kernels and measure whether the fused two-pass version (second loop re-reads x while it is still in cache for $n \leq$ L2) is worth it.

6.3.2 Lowering to a loop

```
prologue: load args[k] into GPRs (rdi points at args; r8..r11, rbx.. as needed)
          rcx = 0 (index); rdx = n & ~7 (vector bound)   # or lea rdx, [rsi-7] and use
    jb
          zero reduction accumulators
loop:     if rcx >= rdx goto tail
          for node in topo_order: emit(node)             # vector body, 8 lanes
          rcx += 8; goto loop
tail:     if rcx == n goto done
```

```
            build lane mask for (n - rcx) lanes; emit body with vmaskmovps loads/stores
done:       horizontal-reduce accumulators -> reduce_out[j]
            vzeroupper; ret
```

Unroll the vector body $U = 2$–4 times (independent iterations) when register pressure allows: the reduction accumulators then split into U chains (project 2's lesson).

6.3.3 Register allocation

The body is straight-line code per iteration, so live ranges are intervals [def, last use] in topological order and linear scan is optimal enough: walk nodes in order; at each definition, free registers whose interval ended; if none is free, spill the live value with the *furthest* next use to a stack slot (`vmovups [rsp+off], ymm`) and reload on use. Reserve `ymm15` for the tail mask and `ymm14` as a scratch for two-operand idioms. Constants are loaded with `vbroadcastss` from the pool at each use unless there is a free register (hoist loop-invariant constants into registers in the prologue when pressure is low). Track and print the maximum pressure per kernel; it is a useful column in the benchmark table.

6.3.4 The System V x86-64 calling convention (§ 3.7)

Integer/pointer arguments arrive in `rdi, rsi, rdx, rcx, r8, r9`; floating-point in `xmm0--7`; return in `rax`/`xmm0`. Callee-saved: `rbx, rbp, r12--r15` (and `rsp`); everything else, including *all* vector registers, is caller-saved, so a leaf kernel may use `ymm0--15` freely. The stack must be 16-byte aligned at a `call`; on entry `rsp` is therefore $\equiv 8 \pmod{16}$. Spill slots must be 32-byte aligned for `vmovaps` (or use `vmovups`, which tolerates misalignment at a small cost); allocate with `sub rsp, S` and restore before `ret`. The 128-byte red zone below `rsp` may be used without adjusting `rsp` in a leaf function. Emit `vzeroupper` before returning: mixing dirty upper `ymm` halves with legacy SSE code in the caller costs a transition penalty on Intel cores.

6.3.5 Instruction encoding

An x86-64 instruction is: legacy prefixes, then either a REX prefix (`0100WRXB`) or a VEX prefix, the opcode (1–3 bytes), ModRM, optional SIB, optional displacement, optional immediate.

ModRM = `mod(2) reg(3) rm(3)`: `mod=11` means `rm` is a register; `mod=00` with `rm=100` means a SIB byte follows; `mod=00`, `rm=101` means RIP-relative with a 32-bit displacement; `mod=01/10` add an 8/32-bit displacement. **SIB** = `scale(2) index(3) base(3)` with scale 2^{scale}. REX bits `R`, `X`, `B` extend `reg`, `index`, `base` to 4 bits (registers 8–15); `W` selects 64-bit operand size.

VEX (used by every AVX instruction). Two-byte form `C5` $[\bar{R}\,\bar{v}_3\bar{v}_2\bar{v}_1\bar{v}_0\,L\,p_1p_0]$ for the `0F` opcode map with `X=B=0`, `W=0`; three-byte form `C4` $[\bar{R}\bar{X}\bar{B}\,m_4..m_0]$ $[W\,\bar{v}_3..\bar{v}_0\,L\,p_1p_0]$ where `mmmmm` selects the opcode map (`00001`: `0F`, `00010`: `0F38`, `00011`: `0F3A`). $\bar{v}$ is the *inverted* second source register; $L = 1$ selects 256-bit; `pp` encodes an implied prefix (`00`: none, `01`: `66`, `10`: `F3`, `11`: `F2`). The bars mean the bits are stored inverted.

Worked example. `vaddps ymm0, ymm1, ymm2` is `VEX.256.0F 58 /r`: two-byte VEX; $\bar{R} = 1$ (no extension), $\bar{v} = \overline{0001} = 1110$ (ymm1), $L = 1$, $pp = 00 \Rightarrow$ `1 1110 1 00 = F4`; opcode `58`; ModRM `11 000 010 = C2` (reg=ymm0 destination, rm=ymm2). Bytes: `C5 F4 58 C2`. For `vfmadd231ps ymm0, ymm1, ymm2` (`VEX.256.66.0F38 B8 /r`): `C4`; $\bar{R}\bar{X}\bar{B} = 111$, map `00010` $\Rightarrow$ `E2`; $W = 0$,

$\bar{v} = 1110$, $L = 1$, $pp = 01 \Rightarrow$ `0 1110 1 01 = 75`; opcode `B8`; ModRM `C2`. Bytes: `C4 E2 75 B8 C2`.

Reference encodings (all verified with GNU `as`; your encoder tests reproduce these and the assembler is the oracle for everything else):

Instruction	Bytes
`vmovups ymm0, [rdi+rcx*4]`	`C5 FC 10 04 8F`
`vmovups [rdx+rcx*4], ymm0`	`C5 FC 11 04 8A`
`vaddps/vmulps/vsubps/vdivps/vmaxps ymm0,ymm1,ymm2`	`C5 F4 {58|59|5C|5E|5F} C2`
`vfmadd231ps ymm0, ymm1, ymm2`	`C4 E2 75 B8 C2`
`vbroadcastss ymm3, [rip+disp32]`	`C4 E2 7D 18 1D` *disp32*
`vxorps ymm0, ymm0, ymm0`	`C5 FC 57 C0`
`vrsqrtps ymm0, ymm1` / `vsqrtps`	`C5 FC 52 C1` / `C5 FC 51 C1`
`vcvtps2dq ymm0, ymm1` / `vcvtdq2ps`	`C5 FD 5B C1` / `C5 FC 5B C1`
`vpaddd ymm0, ymm1, ymm2`	`C5 F5 FE C2`
`vpslld ymm0, ymm1, 23`	`C5 FD 72 F1 17`
`vroundps ymm0, ymm1, imm8`	`C4 E3 7D 08 C1` *imm8*
`vmaskmovps ymm0, ymm1, [rdi+rcx*4]`	`C4 E2 75 2C 04 8F`
`vmaskmovps [rdx+rcx*4], ymm1, ymm0`	`C4 E2 75 2E 04 8A`
`vextractf128 xmm1, ymm0, 1`	`C4 E3 7D 19 C1 01`
`vhaddps xmm0, xmm0, xmm0`	`C5 FB 7C C0`
`vmovss [rdx], xmm0`	`C5 FA 11 02`
`xor ecx, ecx` / `add rcx, 8` / `cmp rcx, rdx`	`31 C9` / `48 83 C1 08` / `48 39 D1`
`jb rel32` / `jmp rel32`	`0F 82` *rel32* / `E9` *rel32*
`mov rax, [rdi+8]` / `lea rax, [rdx-7]`	`48 8B 47 08` / `48 8D 42 F9`
`vzeroupper` / `ret`	`C5 F8 77` / `C3`

Changing a register changes the obvious 3-bit field (and a REX/VEX extension bit for registers 8–15); derive the general encoder from these and test it against the assembler for every register combination (Section 6.6).

Branch displacements and the constant pool are relocations. A `jb rel32` at address p with target t has displacement $t - (p + 6)$: target minus the address of the *next* instruction. A RIP-relative `vbroadcastss` of a constant at pool address S from an instruction occupying $[p, p+9)$ has disp32 $= S - (p + 9)$. In CS:APP's notation this is $S + A - P$ with $A = -4$ (the displacement field's own size) and P the address of the displacement field: $S - 4 - (p + 5) = S - (p + 9)$. Emit the loop with forward branches patched after the target is known (a tiny two-pass or back-patch assembler), and place the pool after `ret`, 32-byte aligned, patching each use once the pool address is fixed.

6.3.6 Executable memory

`mmap(NULL, size, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)`; write code and pool; `mprotect(PROT_READ|PROT_EXEC)`; cast to `tj_kernel`. Never map `PROT_WRITE|PROT_EXEC` (W⊕X; some hardened kernels refuse). x86 keeps instruction and data caches coherent for self-modifying code at the page level once protections change, so no cache flush is required (ARM would need `__builtin___clear_cache`; say so in a comment—your GB10 machine is ARM, so the *structure* of the JIT should admit a second backend even if you never write it).

6.3.7 Transcendentals in SIMD

exp. Range-reduce $x = k \ln 2 + r$ with $k = \text{round}(x/\ln 2)$ (`vroundps` mode 0) and $|r| \le \frac{\ln 2}{2} \approx 0.3466$. Approximate e^r by a degree-5 or 6 polynomial; the Taylor remainder bound is $|r|^6/720 \approx 2.4 \times 10^{-6}$ relative for degree 5, while a minimax fit on $[-\ln 2/2, \ln 2/2]$ (fit the coefficients with a least-squares or Remez script in `scripts/`) reaches $\sim 2 \times 10^{-7}$. Evaluate with Horner's scheme ($p = c_5$; $p = pr + c_4$; ...; five dependent FMAs). Reconstruct 2^k by integer arithmetic: bits $= (k+127) \ll 23$ (`vcvtps2dq`, `vpaddd`, `vpslld`), then multiply. Clamp x to $[-87.3, 88.7]$ first to keep $k+127 \in [0, 255]$. Accuracy test target: ≤ 2 ULP vs `expf`.

rsqrt. `vrsqrtps` gives ≈ 12 bits; one Newton step $y' = y\,(1.5 - 0.5\,x\,y^2)$ gives ≈ 23 bits (three FMAs). Division: `vdivps` has latency $\sim$ 11–13 cycles and throughput of one per 5 cycles on 256-bit; `vrcpps` + one Newton step $y' = y(2 - xy)$ is faster if many divisions are in flight. Offer both and measure.

SiLU. SiLU$(x) = x/(1 + e^{-x})$ composed from the above; for RMSNorm, rsqrt(mean$(x^2) + \epsilon$) is a scalar broadcast between the two passes.

Horizontal reductions. After the loop, each accumulator `ymm` holds 8 partial sums: `vextractf128` the high half, `vaddps` the two `xmm`s, then two `vhaddps` (or a shuffle sequence) to get the scalar in lane 0; `vmovss` it out. The order of this reduction fixes the floating-point result; document it and keep it stable across versions so that tests can be bit-exact.

6.3.8 Why fusion pays

An unfused chain of k elementwise operations over n floats moves roughly $12n$ bytes per operation (two reads, one write) $\Rightarrow 12kn$ bytes. The fused kernel moves $4n(n_{\text{in}} + n_{\text{out}})$. For n larger than L2 both are bandwidth-bound and the speedup bound is $3k/(n_{\text{in}} + n_{\text{out}})$; below L2 the unfused version's intermediates hit in cache and the gap narrows to instruction overhead. Plot speedup vs n across the cache boundaries; it is the fusion decision rule every compiler implements, measured.

6.3.9 Writing a relocatable ELF object (stretch)

Sections: `.text` (the code), `.rodata` (the pool), `.symtab`, `.strtab`, `.shstrtab`, `.rela.text`. Symbols: one `STT_FUNC`, `STB_GLOBAL` per kernel in `.text`; one `STT_SECTION` for `.rodata`. Each pool reference becomes a relocation `R_X86_64_PC32` with `r_offset` at the disp32 field, symbol `.rodata`, addend = pool offset − 4. Then `gcc main.c kernel.o` links it and `readelf -r kernel.o` shows exactly the relocation you computed by hand in the JIT. The ELF64 header, section header table, and `Elf64_Sym`/`Elf64_Rela` layouts are in `<elf.h>`; write them with plain structs.

6.4 Architecture

```
tinyjit/
  include/tinyjit/
    ir.h        node arena, builder API (tj_arg, tj_const, tj_add, ..., tj_store,
    tj_reduce_sum)
    asm.h       byte buffer, label/patch list, encoder functions (one per instruction
    form)
    regalloc.h  linear scan over the IR, spill slots, pressure stats
    codegen.h   lowering: IR -> asm buffer (prologue, loop, tail, epilogue, pool)
    jit.h       tj_compile(graph) -> tj_kernel + metadata; tj_release
```

```
    elfout.h    (stretch) write .o
    interp.h    reference interpreter for the IR (scalar, double precision internally)
  src/          ir.c asm.c regalloc.c codegen.c jit.c elfout.c interp.c mathlib.c (poly
     coefficients)
  tests/        test_asm.c (vs assembler goldens) test_regalloc.c test_codegen.c
    test_math.c test_fuzz.c test_elf.c
  bench/        fused_vs_unfused.c (rmsnorm, silu-gate, quantize) vs -O3 C vs numpy
  scripts/      asm_golden.py (emits .s, assembles, dumps bytes -> tests/golden/*.bin),
     fit_exp_poly.py
```

```c
/* ir.h */
typedef struct tj_graph tj_graph;  typedef uint32_t tj_val;
tj_graph *tj_graph_new(int n_args);
tj_val tj_arg(tj_graph*, int k);           tj_val tj_const(tj_graph*, float c);
tj_val tj_add(tj_graph*, tj_val, tj_val);  /* sub mul div max min fma ... */
tj_val tj_exp(tj_graph*, tj_val);          tj_val tj_rsqrt(tj_graph*, tj_val);
tj_val tj_cvt_f2i(tj_graph*, tj_val, int rounding);  tj_val tj_shl_i(tj_graph*, tj_val,
     int);
void   tj_store(tj_graph*, int k, tj_val);
int    tj_reduce_sum(tj_graph*, tj_val);   /* returns index into reduce_out */

/* jit.h */
typedef struct { tj_kernel fn; void *mem; size_t code_bytes; int max_pressure, spills;
    } tj_compiled;
int  tj_compile(const tj_graph*, tj_compiled *out, int unroll);
void tj_release(tj_compiled*);
void tj_dump(const tj_compiled*, FILE*);   /* hex bytes; pipe into objdump -D -b binary
     -m i386:x86-64 */

/* interp.h */
void tj_interpret(const tj_graph*, float *const *args, size_t n, float *reduce_out);

/* asm.h (excerpt) */
typedef struct { uint8_t *buf; size_t len, cap; /* patches */ } tj_asm;
typedef int tj_label;
void tj_vaddps(tj_asm*, int dst, int src1, int src2);
void tj_vmovups_load(tj_asm*, int dst, int base, int index, int scale, int32_t disp);
void tj_vbroadcastss_rip(tj_asm*, int dst, tj_label pool_const);
tj_label tj_label_new(tj_asm*);  void tj_label_bind(tj_asm*, tj_label);
void tj_jb(tj_asm*, tj_label);   void tj_jmp(tj_asm*, tj_label);
int  tj_asm_finalize(tj_asm*);   /* resolve all patches; error if any unbound */
```

6.5 Assignments

A5.1 The encoder, golden-tested against the assembler *10 h*

Goal. Byte-exact encoders for every instruction form in the reference table, for all register combinations.

Steps. (1) `asm.h`: byte buffer, `emit8/32`, label and patch list, `finalize`. (2) Write the VEX helpers: a function that produces the 2-byte or 3-byte prefix from (map, pp, W, L, vvvv, R, X, B), choosing the 2-byte form whenever legal. (3) ModRM/SIB/displacement helper handling the

rsp/rbp special cases and disp8 vs disp32 selection. (4) One encoder per instruction form in the table. (5) scripts/asm_golden.py: for each form, enumerate operands, emit a .s file, assemble with as, extract bytes with objcopy -O binary or objdump, write tests/golden/<form>.bin plus an index. (6) tests/test_asm.c replays the index. (7) tj_dump and the objdump -D -b binary -m i386:x86-64 round trip documented in the README.
Acceptance. Every golden case byte-equal (several thousand cases); label tests pass; tj_dump output disassembles to what you emitted.
Feeds. Post 5 section 5 (one instruction decoded bit by bit); all of Post 5b.

A5.2 IR, interpreter, first kernel *8 h*

Goal. y = a*b + c compiled and running from JIT memory.
Steps. (1) ir.h: node arena, builder functions, topological order. (2) interp.c: scalar interpreter computing in double, rounding to float at stores. (3) codegen.c: prologue (load arg pointers), vector loop with fixed register assignment (no allocator yet), scalar tail loop using xmm forms (add the scalar encoders to A5.1's golden set), epilogue. (4) jit.c: mmap, copy, mprotect, cast; tj_release. (5) Forked-child test execution in ck.h. (6) Tests for the three ops at all tail lengths.
Acceptance. a*b+c matches the interpreter for all listed n; a deliberately corrupted byte in the test produces "SIGILL in test X" rather than a runner crash; W⊕X respected (never PROT_WRITE|PROT_EXEC).
Feeds. Post 5 sections 3–4 (IR, loop shape).

A5.3 Register allocation, masked tail, unrolling, fuzzing *12 h*

Goal. Arbitrary DAGs compile correctly under register pressure.
Steps. (1) regalloc.c: live intervals from the topological order; linear scan; spill slot assignment in the stack frame (32-byte aligned via and rsp, -32 after saving rbp); pressure and spill statistics. (2) Constant pool after ret, RIP-relative vbroadcastss, patched at finalize—this is the relocation computation; write it as $S + A - P$ in the code comment. (3) Masked tail with vmaskmovps and a mask table in the pool (8 masks of 8 lanes). (4) Unrolling by U with accumulator splitting for reductions; horizontal reduction epilogue. (5) tests/test_fuzz.c: random DAG generator with a seed, including deep chains that force spills; page-boundary placement for tail safety. (6) Determinism test.
Acceptance. 10^4 fuzz cases pass under ASan; spilling cases pass; page-boundary tail test passes; bytes deterministic.
Feeds. Post 5 sections 6–7 (linear scan, constant pool as relocation), Fig. 3.

A5.4 Transcendentals and the fusion benchmark *12 h*

Goal. exp, rsqrt, recip to spec; RMSNorm and SiLU-gate kernels replacing cforward's; the fused-vs-unfused figure.
Steps. (1) scripts/fit_exp_poly.py: least-squares (or Remez) coefficients for e^r on $[-\ln 2/2, \ln 2/2]$, degrees 4–6, with max relative error reported. (2) Lower Exp: clamp, $k = \mathrm{round}(x/\ln 2)$, Horner, exponent reconstruction via integer ops. (3) Rsqrt with one Newton step; Recip via vrcpps+Newton and via vdivps as a switch. (4) ULP tests against expf, $1/\sqrt{x}$, $1/x$ over 10^7 random inputs across the full range; histogram output. (5) RMSNorm as two kernels (reduce; scale) and as one two-pass kernel; SiLU-gate. (6) Plug into cforward behind a build flag; end-to-end golden test still passes. (7) bench/fused_vs_unfused.c: SiLU-gate chain of 4 ops, n from 2^{10} to 2^{26}, fused JIT vs unfused JIT vs -O3 C vs -O3 -ffast-math C; GB/s and speedup;

mark cache boundaries.
Acceptance. `exp` ≤ 2 ULP, `rsqrt`/`recip` ≤ 1 ULP after Newton (document any exception near the clamp limits); `cforward` goldens pass with JIT kernels; fusion figure shows the bandwidth-bound regime beyond L2 with speedup near the $3k/(n_{\text{in}} + n_{\text{out}})$ bound.
Feeds. Post 5 central figure (Fig. 1), Fig. 4, sections 8–9.

A5.5 Integer ops and quantizer kernels *8 h*

Goal. JIT-generated conversions that are bit-identical to `qnum`.
Steps. (1) Integer node types and encoders (`vpaddd`, `vpslld`, `vpsrld`, `vpand`, `vpor`, `vcvtps2dq` with rounding via `vroundps`). (2) fp32→bf16 RNE as a graph (the Equation (2.3) expression in vector integer ops) storing packed 16-bit results (requires `vpackusdw` + `vpermq`, add to the golden set). (3) int8 symmetric group quantize: per-group max via a reduction kernel, then scale/round/-clamp/pack. (4) Tests against `qnum` over 2^{24} random inputs; stress on NaN/Inf inputs.
Acceptance. Bit-identical to `qnum` for bf16 (all finite inputs; NaNs map to a NaN) and int8; throughput vs `qnum`'s intrinsics path reported.
Feeds. Post 5 results (one paragraph: "the quantizer from post 1, now emitted by the compiler from post 5").

A5.6 Stretch: relocatable ELF object *8 h*

Goal. `tinyjit` writes a `.o` that `gcc` links; `readelf -r` shows your relocations.
Steps. (1) `elfout.c`: ELF header, section headers, `.text`, `.rodata`, `.symtab`, `.strtab`, `.shstrtab`, `.rela.text`; one global function symbol per kernel; `R_X86_64_PC32` for each pool reference with addend -4 plus pool offset. (2) A test that writes the object, links it with a C driver, runs it, compares with the JIT version. (3) `docs/elf-notes.md`: `readelf -S/-s/-r` output annotated against the fields you wrote.
Acceptance. `readelf` parses without warnings; linked kernel equals JIT kernel; the notes map each relocation to the JIT's patch.
Feeds. Post 5 section 10.

6.6 Test plan

Encoder goldens
`scripts/asm_golden.py` generates, for every encoder function, every (dst, src1, src2) register triple in 0..15 and every (base, index) GPR pair with displacements $\{0, 8, -7, 0x100, -0x100\}$, assembles them with `as`, and stores the bytes. The test asserts byte equality. This is a few thousand cases and runs in under a second.

Labels and patches
Forward and backward branches at distances < 128 and > 128 bytes (you emit rel32 always, so the test asserts the displacement arithmetic); unbound label is a compile error, not a crash.

Codegen vs interpreter
For each IR op, random inputs, $n \in \{0, 1, 7, 8, 9, 15, 16, 17, 1000, 1023, 1024, 1025\}$: fp32 results equal the interpreter's (which computes in double and rounds once) within 1 ULP for arithmetic, 2 ULP for `exp`, 1 ULP for `rsqrt` after Newton.

Fuzz
Generate 10^4 random DAGs (depth ≤ 12, up to 6 args, random stores and reductions); JIT vs interpreter; include graphs that force spilling (more than 16 live values). Run with ASan:

out-of-bounds writes past `out[n]` are detected because the test allocates exactly n floats with a red zone.

Crash isolation
Tests that execute JIT code run in a forked child; `SIGSEGV/SIGILL` is reported as a failure of that case (this is why `ck.h` forks from project 3 on).

Masked tail
The tail never reads or writes beyond n: verified with page-boundary placement (`mmap` two pages, make the second `PROT_NONE`, put the array flush against the boundary).

Determinism
Compiling the same graph twice yields identical bytes; the reduction order is fixed, so `ReduceSum` is bit-identical run to run.

Quantizer kernels
JIT fp32→bf16 RNE equals `qn_f32_to_bf16` for 2^{24} random inputs; int8 group quantize equals `qnum` exactly.

ELF
`readelf -r` parses the output; `gcc` links it; the linked kernel equals the JIT kernel on random inputs.

6.7 Measurements and blog briefs

(i) Fused vs unfused vs `-O3` C (`-ffast-math` and not) vs NumPy, speedup vs n with L1/L2/L3 boundaries marked; (ii) register pressure and spills per kernel; (iii) effect of unroll factor on reductions; (iv) `exp` accuracy histogram in ULPs and throughput vs `expf`; (v) compile time per kernel; (vi) JIT quantizer throughput vs `qnum`.

Blog plan

Post 5. "An AVX2 JIT for fused tensor ops in 2000 lines of C"
Audience and thesis. For people who use `torch.compile` or Triton and think of code generation as something that happens in a 100k-line project. Thesis: the last mile of a DL compiler is short—an expression DAG, a loop, a register allocator, an encoder—and the piece that feels most mysterious (placing a constant) is the relocation formula from the linking chapter. Subtitle: "fused SiLU-gate at X GB/s vs Y GB/s unfused beyond L2; compile time $Z\,\mu$s".
Length. 3000–3500 words.
Figures.

1. **Figure 1.** (central) Speedup of fused over unfused (y) vs array size n (log x) for the SiLU-gate chain, with L1/L2/L3 sizes as vertical lines and the bandwidth-bound speedup limit dashed; second panel with absolute GB/s for fused, unfused, `-O3` C, `-O3 -ffast-math` C.
2. **Figure 2.** The byte-level decode of `vaddps ymm0, ymm1, ymm2`: a boxed diagram of `C5 F4 58 C2` with every bit field labelled.
3. **Figure 3.** Live-interval diagram for a 10-node DAG with the linear-scan assignment to `ymm` registers; a second DAG with a spill.
4. **Figure 4.** ULP-error histogram of the JIT `exp` vs `expf` over the full range; throughput bar next to it.
5. **Figure 5.** The constant pool as a relocation: a diagram of the instruction at p, the displacement field at P, the constant at S, and $S + A - P$.
6. **Figure 6.** `readelf -r` output of the emitted `.o` next to the JIT's patch log for the same kernel.

Outline.

1. Lede: what gets compiled, how fast, how small the compiler is.
2. Fig. 1 and the one-sentence reason (bytes moved).
3. Setup: machine, compilers compared, tag.
4. The IR and the loop shape; why one implicit loop index is enough for this class of kernels and what it cannot express (gathers, matmul).
5. One instruction, decoded (Fig. 2): REX/VEX, ModRM, SIB, the inverted `vvvv`, the opcode maps. Then the general encoder in 30 lines and the sentence "the assembler is the oracle".
6. The calling convention as a contract: which registers you may clobber, why the kernel is a leaf, `vzeroupper`.
7. Linear scan in 60 lines (Fig. 3): intervals, furthest-next-use spilling, pressure numbers for the real kernels.
8. The constant pool as a relocation (Fig. 5): $S + A - P$ derived, the back-patch, and the same arithmetic for branches.
9. `exp` with range reduction: the polynomial fit, exponent reconstruction by integer ops (code), Fig. 4; `rsqrt` and Newton.
10. The fusion measurement (Fig. 1 again): the $12kn$ vs $4n(n_{\text{in}} + n_{\text{out}})$ arithmetic, cache-resident regime vs bandwidth regime, and what `-ffast-math` buys the C compiler and why.
11. The ELF writer and Fig. 6: what a linker sees is what you computed.
12. What went wrong: the `rbp`-as-base encoding trap; a spill slot misaligned by 16 (`vmovaps` fault only on some inputs); forgetting `vzeroupper` and the 10× slowdown in the caller's SSE code.
13. What this predicts: what changes for AVX-512 (masks, 32 registers) and for NEON on the GB10 (no lane masks; scalar tails); why Triton's CPU backend and TVM make the same fusion decision; one prediction about where `torch.compile`'s fused elementwise kernels will and will not beat eager on this machine.
14. Reproduce box.

Code to show. The VEX prefix builder; the furthest-next-use spill choice; the `exp` lowering; the relocation patch; 10 lines of the ELF `Elf64_Rela` setup.

Post 5b. "How I test a code generator"

Thesis. A code generator is testable to a very high standard with four oracles—the system assembler, an interpreter, a forked crash sandbox, and a page boundary—and none of them needs a framework. **Length** 1400–1800 words. **Figures:** a table of oracles (what each catches, cost, example bug); the fuzz-case count vs bugs-found curve from your log (bugs plateau early—say when). **Outline:** assembler goldens (the enumeration script, how many cases, runtime); the interpreter as a numerics oracle and why it computes in double; forked execution and signal reporting (code from `ck.h`); the page-boundary trick for tail safety; determinism as a test; what remains untested (encodings you never emit, ABI assumptions in callers); reproduce box. **What went wrong:** a golden test that passed because the script assembled the wrong operand order—how a bit-field diagram caught what the test did not.

6.8 Git lesson and push checklist

Git lesson for this project

Topics: automated bisection; `range-diff`; `rerere`; `describe`; signed commits.
Practice.

1. Write `scripts/check.sh`: builds and runs the encoder goldens and the fuzz test with a fixed seed; exits 0/1 (exit 125 if the build fails, which tells bisect to skip). Then `git bisect start HEAD v0.4.0 && git bisect run scripts/check.sh`. Plant a one-byte encoding bug in a middle commit to try it. This only works because every commit builds and passes—the discipline from project 1.
2. After rewriting a branch with `rebase -i`, compare old and new series with `git range-diff main old-tip new-tip` before force-pushing; it shows which commits changed and how.
3. `git config rerere.enabled true`: recorded conflict resolutions replay automatically when you rebase the same branch repeatedly.
4. `git describe --tags --dirty` gives `v0.5.0-12-gabc1234-dirty`; bake it into `tj_dump`'s header via a `-DTJ_VERSION` define generated by the Makefile, so every benchmark output names the exact commit.
5. Sign commits with your SSH key: `git config gpg.format ssh`, `git config user.signingkey ~/.ssh/id_ed25519.pub`, `git config commit.gpgsign true`; add the key as a signing key on GitHub and verify the "Verified" badge.
6. Tag `v0.5.0` at A5.5.

Chapter 7

Project 6: `servlet` — a continuous-batching inference server

7.1 Goal

A C++20 server that exposes `cforward` + `pagedkv` over HTTP/1.1 with token streaming (server-sent events), built on a non-blocking `epoll` reactor, a thread pool, and an iteration-level ("continuous batching") scheduler with chunked prefill, KV-block-aware admission, preemption, and prefix-sharing-aware batching. A load generator with Poisson arrivals and trace replay measures TTFT, TPOT, throughput and goodput under SLOs. The whole thing is race-free under ThreadSanitizer, its parser survives a fuzzer, and it ships with CI, versioned releases, and the other five projects as Git submodules.

Research hook

This is a testbed for the scheduling and caching ideas in your inference research: when you later implement something in vLLM or SGLang, you will already have measured the elementary version here, where every queue and lock is yours. The multi-agent prefix-sharing experiment is the small-scale version of your cross-agent KV-reuse design.

7.2 CS:APP reading map

Section	What you build with it
§ 10.1–10.5 Unix I/O, short counts, RIO	buffered reads on sockets; partial writes; backpressure
§ 10.8–10.11 sharing files, redirection, standard I/O pitfalls	why not `FILE*` on sockets; `dup2` for log redirection
§ 11.1–11.4 networking, sockets interface	listener, accept, address handling (`getaddrinfo`)
§ 11.5–11.6 web servers, HTTP	the protocol state machine
§ 12.1–12.2 process- and I/O-multiplexing-based concurrency	the reactor (`epoll` is `select` grown up)
§ 12.3–12.5 threads, shared variables, semaphores	thread pool, queues, the `sbuf` pattern
§ 12.6 prethreading	the pool design
§ 12.7 thread safety, races, deadlocks	the TSan campaign; lock ordering

7.3 Background

7.3.1 Non-blocking I/O and the reactor

Sockets are set `O_NONBLOCK`; `epoll_create1`, `epoll_ctl(ADD|MOD|DEL)`, `epoll_wait`. With edge-triggered mode (`EPOLLET`) a readiness event is delivered once per transition, so the handler must drain until `EAGAIN`; level-triggered re-reports while data remains and is simpler but wakes more. Use edge-triggered for sockets and `EPOLLONESHOT` if several threads ever share an epoll set (they do not in this design). Watch `EPOLLRDHUP` to see half-closes.

Short counts are normal (§ 10.5): one `read` returns whatever is in the kernel buffer; one request may arrive in ten reads, ten requests in one. The parser must be incremental (resumable state machine over a byte stream), and output must be queued: `write` may accept fewer bytes than offered, after which the connection registers `EPOLLOUT` and resumes when writable. **Backpressure**: cap the per-connection output queue (e.g. 1 MB); when exceeded, stop generating tokens for that request (mark it stalled) rather than growing memory. **Slow clients and idle connections**: a timer wheel or min-heap of deadlines serviced on each `epoll_wait` timeout; close after 30 s idle. The classic denial-of-service against servers that lack this is slowloris; your load generator should include a slow-client mode to prove the defence.

Waking the reactor from other threads. The engine thread finishes a step and has tokens to deliver, but the reactor is asleep in `epoll_wait`. Register an `eventfd` in the epoll set; the engine pushes completions to a lock-free queue and writes 1 to the eventfd; the reactor drains the queue. This is the only cross-thread wake-up mechanism in the server.

7.3.2 HTTP/1.1 and SSE

Request: request line (`POST /v1/completions HTTP/1.1`), headers terminated by an empty line, body of `Content-Length` bytes (support `Transfer-Encoding: chunked` on input as a stretch). Parse as a state machine: `METHOD, PATH, VERSION, HEADER_NAME, HEADER_VALUE, CRLF, BODY, DONE`, with limits on line length (8 KB), header count (64), body size (1 MB); exceeding a limit yields `413`/`431` and closes. Response for streaming:

```
HTTP/1.1 200 OK\r\nContent-Type: text/event-stream\r\nCache-Control: no-cache\r\n
Connection: close\r\n\r\n
data: {"token":" the","index":12}\n\n           ... one event per token ...
data: {"done":true,"usage":{...}}\n\n
```

Non-streaming responses use `Content-Length`. Request bodies are JSON ({`"prompt": "...", "max_tokens": 64, "temperature": 0, "stream": true, "session": "agent-7"`}); vendor a header-only JSON library or reuse your project-1 parser ported to C++.

7.3.3 Threads and queues (§ 12.3–12.7)

Threads: 1 reactor; 1 engine (runs forward passes; the only thread that touches model and KV state); k tokenizer workers (BPE encode/decode is CPU work you do not want on the reactor); 1 metrics flusher. Queues between them: reactor→tokenizers (MPMC or one SPSC per worker), tokenizers→engine (MPSC), engine→reactor (SPSC + eventfd).

Bounded SPSC ring buffer, the workhorse: capacity 2^k, `head` written only by the consumer, `tail` only by the producer; producer writes the slot then publishes `tail.store(t+1, release)`;

consumer does `tail.load(acquire)` before reading the slot. The release/acquire pair makes the slot write visible before the index is observed—this is the memory-ordering argument you should be able to state in one sentence and TSan should be unable to refute. Mutex + condition variable is fine for MPMC queues at these rates; write the lock-free version only where profiling (project 3's tool) shows contention. Know the ABA problem and why bounded ring buffers with monotonically increasing indices avoid it.

Deadlock avoidance: at most two locks ever held together, always in the same order (document the order in `docs/design.md`); never call out to user-supplied callbacks while holding a lock. **TSan** is run on the full server under load as part of `make BUILD=tsan test`.

7.3.4 Continuous batching

The engine loop runs *iterations*; each iteration decides its batch from scratch (Orca-style iteration-level scheduling):

```
loop:
  budget = C tokens                                  # per-iteration token budget
  # 1. decode first: every RUNNING sequence gets one token slot
  batch = running; budget -= |running|
  # 2. admit prefill work while budget and KV blocks allow
  for req in waiting (FIFO, or prefix-affinity order):
      m = radix.match(req.tokens)                    # cached prefix (P4)
      need_tokens = min(budget, len(req) - m - req.prefilled)   # chunked prefill
      need_blocks = blocks for (req.prefilled + need_tokens + 1) - blocks held
      if need_tokens == 0 or pool.free < need_blocks + reserve: continue
      allocate; batch += (req, need_tokens); budget -= need_tokens
  # 3. if pool.free < |running| (cannot guarantee next decode step): preempt
  while pool.free < |running| and running nonempty:
      victim = youngest running (LIFO); free its blocks; (optionally swap to tier);
    move to waiting
  # 4. run forward: prefill chunks + decode step in one pass over layers
  step(batch)
  # 5. finish: EOS or max_tokens -> release blocks, insert full blocks into radix tree
  # 6. deliver tokens -> completion queue; eventfd signal
```

Why the budget: a 4K-token prefill would stall every running decode for the time of a 4K-token forward pass; chunking at C (e.g. 512) bounds decode inter-token latency to the step time at C tokens. **Why LIFO preemption**: the youngest sequence has the fewest tokens to recompute and has consumed the least service. **Prefix-affinity ordering**: among waiting requests, prefer those whose prefix match is longest (they need the least prefill), then group by shared prefix so a shared prefix enters the batch once (`servlet` with `pagedkv`'s refcounts makes this free).

7.3.5 Performance model

Decode step time with batch size B on a bandwidth-bound machine is approximately

$$t(B) \approx t_w + B\,t_{\text{tok}}, \qquad t_w = \frac{Pb_w}{\beta}, \quad t_{\text{tok}} \approx \frac{\text{KV bytes per sequence}}{\beta} + \text{per-token compute}, \tag{7.1}$$

so decode throughput $B/t(B)$ rises almost linearly while $Bt_{\text{tok}} \ll t_w$ and saturates at $1/t_{\text{tok}}$: batching amortizes the weight read. Measure $t(B)$ for $B = 1..64$; the fitted t_w should match project 2's bandwidth bound.

Metrics. TTFT (request arrival to first token), TPOT or ITL (mean inter-token latency after the first), end-to-end latency, throughput (tokens/s, requests/s), and *goodput*: requests/s whose TTFT $\leq a$ and TPOT $\leq b$. Report p50/p90/p99.

Queueing. Little's law $N = \lambda W$ relates mean concurrency, arrival rate and mean latency, and is a consistency check on your measurements. For an M/M/1 reference, $W = 1/(\mu - \lambda)$ with utilization $\rho = \lambda/\mu$: latency diverges as $\rho \to 1$. Your server has state-dependent service (bigger batches serve faster per request), so measure the latency–load curve empirically and locate the knee; compare with and without chunked prefill and with and without prefix caching.

Load generation. Poisson arrivals: inter-arrival $\Delta = -\ln U/\lambda$, $U \sim \mathrm{Unif}(0, 1]$. Prompt and output lengths from a trace (ShareGPT-style length distributions, or a synthetic lognormal); multi-agent workloads: A agents each with a 1–2K-token system prompt shared by all their requests, interleaved. The generator is a separate C++ program using the same reactor code (a good test of its reuse), with an open-loop mode (arrivals independent of completions—the correct way to find the knee) and a closed-loop mode (fixed concurrency).

7.3.6 Tokenizer

Implement the `llama2.c` tokenizer: a vocabulary of byte-level pieces with scores; encode by splitting into UTF-8 bytes and repeatedly merging the adjacent pair with the highest score until none exists (greedy BPE, $O(n^2)$ naive, $O(n \log n)$ with a heap); decode by concatenating pieces and handling `<0xHH>` byte tokens. It is 150 lines and removes a Python dependency from the serving path.

7.4 Architecture

```
servlet/
  third_party/   qnum/ cforward/ pagedkv/   (git submodules, pinned)
  include/servlet/
    net/reactor.hpp    epoll loop, Connection, timers, eventfd wakeup
    net/buffer.hpp     growable byte buffer with consume/append (RIO in C++)
    http/parser.hpp    incremental request parser; http/response.hpp; http/sse.hpp
    sched/scheduler.hpp  Request, states (WAITING, RUNNING, PREEMPTED, DONE), the
    iteration algorithm
    sched/policy.hpp   admission order, preemption victim selection, budget
    engine/engine.hpp  wraps cforward+pagedkv; step(batch); owns the engine thread
    engine/tokenizer.hpp
    util/spsc.hpp util/mpmc.hpp util/metrics.hpp util/json.hpp
  src/               main.cpp (config, wiring), one .cpp per header
  bench/loadgen/     loadgen.cpp (open/closed loop, traces, multi-agent), report.py
  tests/             test_parser.cpp test_reactor.cpp test_queues.cpp test_scheduler.
    cpp (fake engine)
                     test_tokenizer.cpp test_e2e.py (black-box with python requests)
  fuzz/              fuzz_parser.cpp (libFuzzer), corpus/
  .github/workflows/ ci.yml  (gcc+clang, release+asan+tsan, tests, fuzz smoke,
    submodule checkout)
```

```
// sched/scheduler.hpp (excerpt)
struct Request {
  uint64_t id; std::vector<int32_t> tokens; uint32_t prefilled = 0, generated = 0,
    max_tokens;
```

```
  float temperature; bool stream; std::string session;      // prefix-affinity key
  enum class State { Waiting, Running, Preempted, Done } state;
  pk_table table; uint32_t cached_prefix = 0;
  std::chrono::steady_clock::time_point t_arrival, t_first_token;
};
struct BatchItem { Request* req; uint32_t n_prefill_tokens; };  // 0 => one decode
    token
class Scheduler {
 public:
  Scheduler(Policy p, pk_kv* kv, uint32_t token_budget, uint32_t reserve_blocks);
  void submit(std::unique_ptr<Request>);                        // from tokenizer
    workers (MPSC)
  std::vector<BatchItem> plan();                                // steps 1-3 of the loop
  void commit(const std::vector<BatchItem>&, const StepResult&);   // steps 5-6
  Stats stats() const;
};

// engine/engine.hpp
struct StepResult { std::vector<std::pair<uint64_t,int32_t>> tokens; double step_ms; };
class Engine { public: StepResult step(const std::vector<BatchItem>&); /* calls
    cforward over paged KV */ };

// net/reactor.hpp
class Reactor { public: void add(int fd, uint32_t events, Handler*); void run(); void
    wake(); /* eventfd */ };
```

Configuration via a TOML/JSON file and flags: model paths, `B`, pool size, huge pages, token budget C, reserve blocks, policy, port, threads.

7.5 Assignments

A6.1 Reactor, buffers, parser, a static endpoint *14 h*

Goal. An `epoll` server that survives 10K idle connections, slow clients and byte-at-a-time requests, serving `GET /healthz`.

Steps. (1) `net/buffer.hpp`: growable byte buffer with `append`, `consume`, `peek`; the `read_some`/`write_some` helpers that treat `EAGAIN` and short counts correctly (unit tests with a mock fd). (2) `net/reactor.hpp`: listener with `SO_REUSEADDR`, non-blocking accept loop, edge-triggered registration, per-connection handler object, `EPOLLOUT` on demand, deadline heap with `timerfd` or `epoll_wait` timeout, `eventfd` wake. (3) `http/parser.hpp`: the incremental state machine with limits and status codes; `http/response.hpp`. (4) `tests/test_parser.cpp` with the split-anywhere property; `fuzz/fuzz_parser.cpp` with a seed corpus of 20 requests. (5) Python harness for the reactor tests (connections, slow client, half-close, fd-leak check). (6) `GET /healthz` returning JSON with uptime and connection count.

Acceptance. Parser unit and property tests pass; 60-second fuzz run clean under ASan; reactor harness passes (10K idle connections, slowloris closed at timeout, no fd leak); ASan/UBSan clean.

Feeds. Post 6 section 4 (short counts and the reactor), Post 6b's parser-fuzzing paragraph.

A6.2 Tokenizer and a single-sequence completion *8 h*

Goal. `POST /v1/completions` non-streaming, one request at a time, through the real model.
Steps. (1) `engine/tokenizer.hpp`: load `tokenizer.bin`, greedy BPE encode with a heap, decode with byte tokens; round-trip tests and the Python-reference comparison. (2) JSON request/response (vendor or port). (3) `Engine` wrapping `cforward` + `pagedkv` (submodules) with a single-sequence `generate`. (4) The endpoint, synchronous on the reactor thread for now (this is deliberately wrong; the next assignment fixes it and you will measure the difference).
Acceptance. Tokenizer tests pass; a `curl` request returns the same text as `cforward`'s CLI for temperature 0; `healthz` is unresponsive during generation (observe and log it: that is the motivation for A6.3).
Feeds. Post 6 section 3 (setup) and the "why a separate engine thread" sentence.

A6.3 Engine thread, queues, streaming *12 h*

Goal. Token streaming over SSE with the model on its own thread, race-free under TSan.
Steps. (1) `util/spsc.hpp`: bounded ring with release/acquire publication; `util/mpmc.hpp`: mutex+condvar queue; stress tests. (2) Tokenizer worker pool (k threads) fed by the reactor. (3) Engine thread loop: pull requests, generate token by token, push completions, signal `eventfd`. (4) Reactor drains completions and writes SSE events; backpressure stalls generation for a slow client; `Connection: close` handling. (5) Lock-order documentation in `docs/design.md`. (6) TSan build under a 60-second load.
Acceptance. Streaming works with `curl -N`; queue stress tests pass under TSan and `-O3`; the whole server under load has zero TSan reports; a slow client cannot grow memory beyond the per-connection cap (test).
Feeds. Post 6 section 5 (threads and queues); all of Post 6b.

A6.4 Continuous batching with chunked prefill *14 h*

Goal. The iteration-level scheduler with block-aware admission, and the $t(B)$ measurement.
Steps. (1) `sched/scheduler.hpp`: `Request` states, `plan()` implementing steps 1–3 of the loop, `commit()` implementing 5–6; token budget C; reserve blocks. (2) `Engine::step` running prefill chunks and decode tokens in one pass over layers (batched matvec per sequence; prefill chunks via GEMM). (3) Fake engine for scheduler tests; the invariants and the chunked-vs-unchunked equality test. (4) `bench/batch_step.cpp`: $t(B)$ for $B = 1..64$; fit t_w and t_{tok}; compare t_w to project 2's Pb_w/β.
Acceptance. Scheduler tests pass; end-to-end determinism test passes; the $t(B)$ fit's t_w is within 25% of the project-2 bound; a plot of throughput vs B saturating as predicted.
Feeds. Post 6 Fig. 2 and section 6.

A6.5 Preemption and prefix-affinity scheduling *10 h*

Goal. Correct behaviour when the pool fills; prefix-aware admission; the multi-agent benchmark.
Steps. (1) Preemption by recompute (LIFO victim); then by swap to `pagedkv`'s tier; a policy switch. (2) Prefix-affinity ordering in `plan()` using `pk_radix_match` lengths and the session key. (3) Multi-agent workload in the load generator (A6.6 can be developed in parallel; the generator's trace mode is needed here). (4) Tests: preempted-then-resumed output identical; preemption count and recompute tokens reported in metrics.
Acceptance. Preemption tests pass; under a deliberately undersized pool the server completes

every request (slowly) rather than failing; prefix-affinity reduces prefilled tokens on the multi-agent workload by a measured amount reported in the README.
Feeds. Post 6 section 7 (preemption) and Fig. 4.

A6.6 Load generator and the latency–load study *12 h*

Goal. The curves that are the point of the project.
Steps. (1) `bench/loadgen/loadgen.cpp` reusing the reactor: open-loop Poisson arrivals at rate λ, closed-loop at concurrency N, trace replay of (prompt length, output length, session) tuples; per-request TTFT/TPOT/E2E recorded to CSV. (2) `report.py`: percentiles, throughput, goodput under SLO pairs, Little's-law consistency check. (3) The sweep: λ from low to past the knee, for four configurations (batching on/off, chunked prefill on/off, prefix cache on/off), 90 s each, release build, pinned cores. (4) Multi-agent sweep over number of agents. (5) `ltrace-ml` profile of the server under load at the knee.
Acceptance. CSVs and plots in `docs/figures/`; the Little's-law check holds within 10%; the knee is identified for each configuration; the profile attributes non-model time (JSON, tokenization, syscalls) with percentages.
Feeds. Post 6 central figure (Fig. 1), Fig. 3–5, results.

A6.7 CI, releases, submodules, back-porting *10 h*

Goal. Six repositories at tagged releases with green CI.
Steps. (1) `ci.yml` per the Git lesson; nightly fuzz and load jobs. (2) Submodule pins updated to the other projects' release tags. (3) `CHANGELOG.md` from conventional commits. (4) Back-port `ci.yml` to projects 1–5; protect their `main` branches with the status check. (5) `gh release create v1.0.0` with the server binary and the load-generator binary. (6) The top-level `systems-from-csapp` README linking all six repos and all twelve posts.
Acceptance. CI green on all six repos; `git clone --recurse-submodules` + `make test` works from scratch on a clean machine (test it in a container); releases published.
Feeds. The series index page.

7.6 Test plan

Parser

Unit tests for every state; the *split-anywhere* property: for a corpus of valid requests, feeding the bytes in every possible 2-way split (and random k-way splits) yields the identical parsed result; limits produce the right status codes; malformed inputs never crash. `fuzz/fuzz_parser.cpp` with `clang -fsanitize=fuzzer,address` runs for 10 minutes in CI's nightly job and 30 seconds on every push.

Buffer/RIO

Partial writes simulated with a mock fd that accepts k bytes at a time; the queued output is delivered in order and completely.

Reactor

Python test harness opens N connections, sends requests byte-by-byte, half-closes, disconnects mid-stream; the server stays up, leaks no fds (`ls /proc/<pid>/fd | wc -l` before and after), and closes idle connections at the timeout.

Queues

SPSC/MPMC stress tests: 10^7 items, multiple producers/consumers, sequence numbers veri-

fied; run under TSan and with `-O3`.

Scheduler with a fake engine
A deterministic fake engine (step time $= t_w + Bt_{\text{tok}}$, tokens from a fixed sequence) makes the scheduler testable without a model: never exceeds the pool; token budget respected; FIFO among equals; preemption chooses the youngest; a preempted-then-resumed request produces exactly the same tokens as an unpreempted run; chunked prefill produces the same tokens as unchunked.

End-to-end determinism
With temperature 0, a request's output is independent of what else is in the batch (true for matvec-per-sequence decode; document any deviation).

Tokenizer
Round trip on a UTF-8 corpus (ASCII, accented, CJK, emoji, invalid bytes); encode equals the Python reference for 1000 strings.

Load test as a test
A 60-second open-loop run at a fixed λ below the knee must meet p99 TTFT and TPOT thresholds (soft, release build only, skipped in CI).

Sanitizers
ASan+UBSan on all unit tests; TSan on the queue tests and on a 60-second load run.

7.7 Measurements and blog briefs

(i) $t(B)$ with fitted t_w vs project 2's bound; (ii) latency vs offered load, p50/p99 TTFT and TPOT, four configurations; (iii) goodput vs load under two SLO settings; (iv) prefix-cache hit rate and TTFT for the multi-agent workload vs number of agents; (v) preemption count and recompute overhead vs pool size; (vi) CPU profile of the server under load with `ltrace-ml`.

Blog plan

Post 6. "Continuous batching from first principles: a CPU inference server you can read in an afternoon"

Audience and thesis. For people who run vLLM and tune `max-num-seqs` and `max-num-batched-tokens` without a model of what they do. Thesis: batching amortizes the weight read, chunking bounds latency, paging turns admission into a counting problem, and everything else is plumbing—and the plumbing is where the tail latency hides. Subtitle: "X req/s at p99 TTFT < 500 ms, $Y\times$ the single-sequence rate, on one CPU".

Length. 3000–3500 words.

Figures.

1. **Figure 1.** (central) p50 and p99 TTFT and TPOT (y, log) vs offered load λ (x) for the four configurations; knees marked; the SLO lines drawn.
2. **Figure 2.** Decode step time $t(B)$ vs batch size with the linear fit and the project-2 t_w bound; throughput $B/t(B)$ on a second axis saturating at $1/t_{\text{tok}}$.
3. **Figure 3.** TPOT histogram with and without chunked prefill at the same load (the long tail disappears).
4. **Figure 4.** Multi-agent: prefilled tokens per request and TTFT vs number of agents, prefix cache on/off.
5. **Figure 5.** Goodput vs load for two SLO pairs; the load at which goodput peaks vs the load at which throughput peaks.

6. **Figure 6.** Flame graph of the server at the knee (from `ltrace-ml`), model time collapsed, non-model time expanded.
7. **Figure 7.** Architecture diagram: reactor, tokenizer pool, engine, queues, `eventfd`; lock order annotated.

Outline.

1. Lede: the four claims and the headline numbers.
2. Fig. 1 read configuration by configuration.
3. Setup: model, pool size, B, C, machine, how load was generated (open loop—say why), runs, tag.
4. Short counts shape everything: partial reads, partial writes, backpressure, the slowloris test; the incremental parser; 20 lines of the output queue.
5. Threads and queues (Fig. 7): why the model has one thread, the SPSC publish/consume pair with the memory-ordering sentence, the `eventfd` handshake.
6. The iteration loop in 25 lines; the $t(B)$ model and Fig. 2; "batching amortizes the weight read" with the fitted numbers.
7. Chunked prefill: the budget argument and Fig. 3; the cost (prefill takes more iterations).
8. Admission as counting: blocks needed vs free, the reserve, LIFO preemption and why.
9. Prefix-affinity scheduling and Fig. 4; the connection to the radix tree of post 4.
10. Goodput (Fig. 5) and Little's law as a sanity check on the whole measurement.
11. The profile (Fig. 6): where the non-model time goes, and the two fixes it motivated.
12. What went wrong: edge-triggered `epoll` without draining to `EAGAIN`; a deadlock from acquiring the metrics lock inside the scheduler lock (found by the lock-order rule, not by TSan); a load test that measured the load generator.
13. What this predicts: the same curves on a GPU with the numbers scaled; why vLLM's defaults sit where they do; what cross-agent prefix sharing is worth at scale; one prediction about p99 TTFT under chunked prefill for a given C on an H100.
14. Reproduce box: two machines or two pinned core sets; 40 minutes for the full sweep.

Code to show. The output-queue/`EPOLLOUT` logic; the SPSC pair; `plan()`; the Poisson inter-arrival line.

Post 6b. "Finding data races in a server with ThreadSanitizer, and the one that TSan could not see"

Thesis. TSan finds races in what runs; lock-order discipline finds deadlocks in what could run; both are necessary and neither is sufficient. **Length** 1400–1800 words. **Figures:** the lock-order graph for the server (nodes: locks; edges: acquisition order; must be acyclic); an annotated TSan report for a real race from your log; a diagram of the SPSC release/acquire pair with the happens-before edge drawn. **Outline:** what TSan instruments and what it costs (slowdown measured); three races it found in this server, each with the report excerpt and the fix; the release/acquire argument for the ring in one paragraph; the deadlock that TSan cannot see because the two acquisitions never overlapped in the test run, and the lock-order rule that catches it statically; what `helgrind` and `-fsanitize=thread` disagree on; reproduce box. **What went wrong:** suppressing a "benign" race on a statistics counter and later reading garbage percentiles—no race is benign in a report you will publish.

7.8 Git lesson and push checklist

Git lesson for this project

Topics: submodules; CI with GitHub Actions; semantic versioning and CHANGELOG; releases with binaries; packfiles and `gc`.

Practice.

1. `git submodule add git@github.com:<user>/cforward third_party/cforward` (and the others). The superproject records a *commit hash* per submodule in a gitlink tree entry (`git ls-tree HEAD third_party`) plus `.gitmodules`. Updating: `cd third_party/cforward && git fetch && git checkout v0.2.1`, then commit the new pointer in the superproject. Fresh clones need `git clone --recurse-submodules` or `git submodule update --init --recursive`; CI must do this (`actions/checkout` with `submodules: recursive`). Know the alternative (`git subtree`) and why you chose submodules (independent release cadence, six separate blog-able repos).
2. CI (`.github/workflows/ci.yml`): matrix over {`gcc, clang`} × {`release, asan, tsan`}; steps: checkout with submodules, install deps, `make BUILD=${{ matrix.build }} test`, 30-second fuzz smoke, upload the test log as an artifact; a nightly job runs the 10-minute fuzz and the load test. Add the status badge to each README. Then back-port the workflow to projects 1–5 and make their `main` branches require it.
3. Semantic versioning: `MAJOR.MINOR.PATCH`; bump MINOR for features, PATCH for fixes, MAJOR for breaking API changes (relevant now that `servlet` depends on the others' headers). Maintain `CHANGELOG.md` in Keep-a-Changelog format, generated from conventional commits (`git log --pretty=format:'- %s' v0.5.0..HEAD`) and edited by hand.
4. Release: `git tag -a v1.0.0 -m ...`, `git push --follow-tags`, `gh release create v1.0.0 --notes-file RELEASE_NOTES.md build/release/servlet`. `git archive --format=tar.gz --prefix=servlet-1.0.0/ v1.0.0` produces a source tarball without `.git`.
5. Internals revisited: `git count-objects -v`, `git gc`, then look at `.git/objects/pack/`; `git verify-pack -v` shows delta chains—six months of history in a few megabytes, because blobs are delta-compressed against similar blobs. Compare with the loose-object count you recorded in project 1.
6. Final push checklist: all six repos at a tagged release, CI green, READMEs with headline figures, six blog posts linked from a top-level `systems-from-csapp` README that explains how the pieces fit.

Appendix A

Git command reference

Grouped by the question you are asking. Each project's Git lesson introduces the commands in context; this is the lookup table afterwards.

Inspecting

```
git status -sb                      # short status with branch/upstream
git log --oneline --graph --decorate --all
git log -p -- path                  # history of one file with diffs
git log -S'needle' / -G'regex'      # commits that add/remove a string / match a regex
    in the diff
git show <rev>[:path]               # a commit, or a file at a commit
git diff / --staged / <a>..<b> / --stat
git blame -w -C -C path             # ignore whitespace, follow moved code
git cat-file -t|-p <hash>           # object type / pretty-print
git ls-tree HEAD [path]             # tree entries (gitlinks for submodules)
git reflog                          # where HEAD has been (local, expires)
git describe --tags --dirty         # nearest tag + distance + hash
git count-objects -v                # loose vs packed objects
```

Making commits

```
git add -p                          # stage hunks interactively
git commit [-m msg] [--amend] [--fixup <rev>]
git restore --staged path           # unstage (HEAD -> index)
git restore path                    # discard working-tree change (index -> worktree)
git rm --cached path                # stop tracking, keep file
git stash push -m "msg" / list / pop / drop
```

Branches and history surgery

```
git switch -c name [start]          # create+switch; git switch - toggles
git branch -d name / -D             # delete merged / force
git merge --ff-only other           # refuse to create a merge commit
```

```
git merge --no-ff other              # always create one
git rebase main                      # replay branch onto main
git rebase -i main                   # pick/reword/edit/squash/fixup/drop
git rebase --onto main A B           # transplant commits (A, B] onto main
git rebase --autosquash              # apply --fixup commits automatically
git cherry-pick <rev>...             # copy commits onto current branch
git revert <rev>                     # new commit undoing <rev> (safe on public history)
git reset --soft|--mixed|--hard <rev>   # move branch; keep index / keep worktree /
    discard
git range-diff main old new          # compare two versions of a series
git bisect start [bad] [good]; git bisect run script   # exit 0 good, 1 bad, 125 skip
git worktree add ../dir branch / list / remove
git config rerere.enabled true       # reuse recorded conflict resolutions
```

Remotes

```
git remote -v / add origin url / set-url origin url
git fetch [--prune] [--tags]         # update remote-tracking refs, never touches your
    branches
git pull --rebase                    # fetch + rebase (set pull.rebase=true)
git push -u origin branch            # set upstream on first push
git push --force-with-lease          # safe force: refuses if remote moved
git push --follow-tags               # push annotated tags reachable from pushed commits
git push origin --delete branch
```

Tags, releases, submodules

```
git tag -a v1.2.0 -m "..."           # annotated (object); git tag v1.2.0 is lightweight
     (ref only)
gh release create v1.2.0 --generate-notes [files...]
git archive --format=tar.gz --prefix=name-1.2.0/ v1.2.0 > name-1.2.0.tar.gz
git submodule add url path; git submodule update --init --recursive
git submodule status; git submodule foreach 'git fetch'
git clone --recurse-submodules url
```

Hooks, attributes, signing

```
git config core.hooksPath .githooks       # versioned hooks
git config gpg.format ssh; git config user.signingkey ~/.ssh/id_ed25519.pub
git config commit.gpgsign true
# .gitattributes:  * text=auto eol=lf   |  *.bin binary   |  path linguist-language=
    Python
```

Appendix B

Performance measurement cheat sheet

```
# Allow perf for non-root (per boot):  sudo sysctl kernel.perf_event_paranoid=1
perf stat -e cycles,instructions,cache-misses,cache-references,\
  L1-dcache-load-misses,LLC-load-misses,dTLB-load-misses,page-faults -r 5 ./bench
perf record -g --call-graph fp ./bench     # frame-pointer call graphs (why -fno-omit-
    frame-pointer)
perf report --no-children --sort sym
perf annotate <symbol>                     # instruction-level hot spots
perf list | grep -i -e tlb -e fma          # available events on this CPU
valgrind --tool=cachegrind --cache-sim=yes ./bench ; cg_annotate cachegrind.out.*
objdump -d -M intel --no-show-raw-insn -S build/release/x.o | less
# Pin and quiet the machine for microbenchmarks:
taskset -c 2 ./bench ;  echo performance | sudo tee /sys/devices/system/cpu/cpu*/
    cpufreq/scaling_governor
```

Reporting rules used throughout. Report the median of ≥ 5 runs and the spread; state the build flags, the commit (`git describe`), and the machine (`docs/machine.md`); plot the bound you derived next to the measurement; never report a speedup without the absolute numbers it came from.

Cycle counting. `rdtsc` counts at a constant reference frequency, not core cycles; calibrate against `clock_gettime(CLOCK_MONOTONIC)` once at startup and convert to nanoseconds, or use `perf_event_open` with `PERF_COUNT_HW_CPU_CYCLES` for true cycles (project 2's `roofline.h`). Warm up before timing; time enough iterations to exceed 100 ms; subtract loop overhead measured with an empty body.

Appendix C

Reference constants

Bit patterns (hexadecimal)

	fp32	fp16	bf16	E5M2	E4M3
+0	00000000	0000	0000	00	00
+1.0	3F800000	3C00	3F80	3C	38
max finite	7F7FFFFF	7BFF	7F7F	7B	7E
min normal	00800000	0400	0080	04	08
min subnormal	00000001	0001	0001	01	01
$+\infty$	7F800000	7C00	7F80	7C	—
canonical NaN	7FC00000	7E00	7FC0	7E	7F

Transformer configurations for testing

Model	d	L	h	h_{kv}	F	V	θ_0
`stories15M` (llama2.c)	288	6	6	6	768	32000	10000
`stories110M`	768	12	12	12	2048	32000	10000
TinyLlama-1.1B	2048	22	32	4	5632	32000	10000
Llama-3-8B	4096	32	32	8	14336	128256	500000

x86-64 System V register roles

Integer arguments 1–6	`rdi rsi rdx rcx r8 r9`
Return	`rax` (`rdx` for 128-bit)
Callee-saved	`rbx rbp r12 r13 r14 r15` (and `rsp`)
Caller-saved	`rax rcx rdx rsi rdi r8--r11`, all `xmm/ymm/zmm`
FP arguments / return	`xmm0--7` / `xmm0`
Stack alignment at `call`	16 bytes; red zone 128 bytes below `rsp`

Register numbers for encoding

`rax`=0, `rcx`=1, `rdx`=2, `rbx`=3, `rsp`=4, `rbp`=5, `rsi`=6, `rdi`=7, `r8`–`r15`=8–15 (REX.`B`/`R`/`X` set for 8–15). `ymm0`–`ymm15` likewise. Note `rsp` as `rm=100` means "SIB follows" and `rbp` as `base=101` with

`mod=00` means "no base, disp32" (RIP-relative in 64-bit mode when `rm=101`, `mod=00`); both need special-casing in the encoder.

Appendix D

Blog series overview

#	Post	Central figure
1	Every inference number format, from the bits up	SQNR vs bits/weight, all formats, real weights
1b	Verifying a float converter against all four billion inputs	runtime and findings of the exhaustive sweep
2	A roofline for LLM decode, measured from scratch on one CPU core	roofline with the optimization ladder
2b	Online softmax, explained with one invariant	ULP distance online vs offline
3	What does PyTorch actually ask the CUDA runtime for?	cudaMalloc/launch timeline
3b	A sampling profiler is a statistics problem	estimate vs truth with confidence intervals
4	Paging a KV cache on a CPU: what the TLB thinks of PagedAttention	latency vs pool size, 4K vs 2M pages
4b	Belady's MIN for KV blocks: how far is LRU from optimal?	miss ratio vs pool size, LRU/LFU/OPT
5	An AVX2 JIT for fused tensor ops in 2000 lines of C	fused speedup across the cache hierarchy
5b	How I test a code generator	(method post; table of oracles)
6	Continuous batching from first principles	latency vs load, four configurations
6b	Finding data races in a server with ThreadSanitizer	(method post; the lock-order diagram)

Each post follows the same shape: the figure first, the mechanism that explains it second, the code third, and a "what this predicts elsewhere" paragraph last (GPUs, Tenstorrent, the serving stack you actually use). Link every post to the tagged release that produced its numbers.